



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Pranav Pol** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1
13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 1

AIM: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Tasks:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.
2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.
3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.
4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

THEORY:

1. Cryptography Hash Function in Blockchain

A cryptographic hash function is a mathematical algorithm that transforms input data of arbitrary size into a fixed-size output called a hash or digest. In blockchain technology, hash functions serve as the fundamental cryptographic primitive that ensures data integrity, immutability, and security.

Definition and Core Concept: Cryptographic hashing is one of the most notable uses of cryptography in blockchain systems. It enables immutability by creating unique digital fingerprints of data. The encryption in cryptographic hashing does not involve the use of keys, unlike symmetric or asymmetric encryption methods. When a transaction is verified, a hash algorithm adds the hash to the block, and a new unique hash is generated from the original transaction data.

Essential Properties:

1. **Deterministic Nature:** For a particular message, the hash function always produces the same output. This consistency is crucial for verification processes in blockchain networks.

2. **Avalanche Effect:** Every minor change in input data results in a drastically different hash value. Even changing a single bit in the input produces a completely different output, making it easy to detect any tampering.
3. **Pre-image Resistance:** The input value cannot be guessed or derived from the output hash function. This one-way property ensures that the original data cannot be reconstructed from the hash.
4. **Collision Resistance:** It is computationally infeasible to find two different inputs that produce the same hash output. This property prevents fraudulent transactions from being disguised as legitimate ones.
5. **Computational Efficiency:** Hash functions are fast and efficient as they largely rely on bitwise operations, allowing quick computation even for large datasets.

Commonly Used Hash Functions:

- **SHA-256 (Secure Hash Algorithm 256-bit):** Used extensively in Bitcoin and many other cryptocurrencies
- **Keccak-256:** Employed in Ethereum for hashing operations
- **MD5 and SHA-1:** Older algorithms, though considered less secure for modern applications

Benefits in Blockchain:

- **Bandwidth Reduction:** Hashing reduces the amount of data that needs to be transmitted across the network
- **Data Integrity:** Prevents modification in the data block by creating tamper-evident records
- **Simplified Verification:** Makes verification of transactions easier and faster without processing entire datasets
- **Chain Linking:** Hashing continues to combine or make new hashes, but the original footprint remains accessible, creating an immutable chain

Practical Implementation: When a transaction is verified through a hash algorithm, it is added to the blockchain. As the transaction becomes confirmed, it is added to the network, making a chain of blocks. The hash of each block includes the hash of the previous block, creating a cryptographically secure chain. Any attempt to modify historical data would require recalculating all subsequent hashes, which is computationally impractical.

2. Merkle Tree

A Merkle tree, also known as a hash tree or binary hash tree, is a sophisticated data structure used for efficient data verification and synchronization in blockchain and distributed systems. Named after Ralph Merkle, who proposed this concept in his 1987 paper titled "A Digital

Signature Based on a Conventional Encryption Function," it represents a fundamental innovation in cryptographic data structures.

Fundamental Definition: A Merkle tree is a hash-based data structure that generalizes the concept of a hash list. It is a tree data structure where each leaf node contains a hash of a block of data (typically a transaction), and each non-leaf node contains a hash of its children nodes. This hierarchical arrangement creates an "infinite tree" structure where information is processed through cryptographic functions to generate unique identifiers.

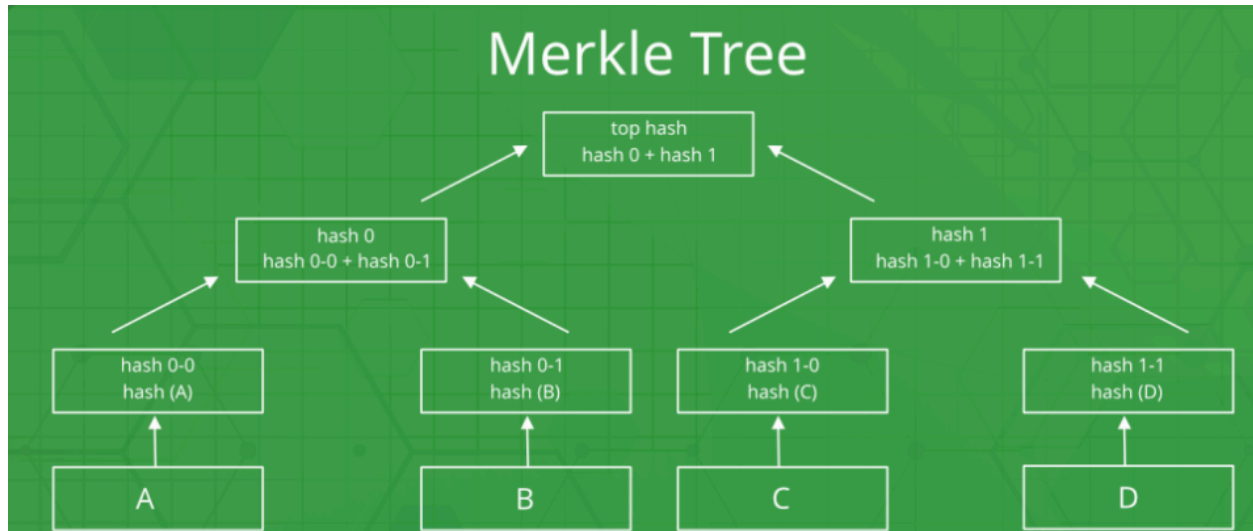
Core Characteristics:

1. **Hierarchical Organization:** The tree consists of multiple levels - leaf nodes at the bottom containing transaction hashes, intermediate nodes (branches) in the middle, and a single root node at the top.
2. **Hash-Based Construction:** Each node's value is derived by hashing the concatenated values of its child nodes, creating a cryptographically secure relationship throughout the tree.
3. **Binary Structure:** Merkle trees typically have a branching factor of 2, meaning each node has up to 2 children, though n-ary trees with n children per node are also possible.
4. **Depth Uniformity:** All leaf nodes are at the same depth and are positioned as far left as possible, ensuring balanced tree structure.

Purpose and Significance: Merkle trees securely encode blockchain transaction data and dramatically improve validation efficiency and speed. They allow users to verify specific transactions without downloading entire blockchain files, which can be hundreds of gigabytes in size. This capability is essential for lightweight clients, mobile wallets, and efficient network synchronization.

Data Integrity Mechanism: The structure maintains data integrity using hash functions. The root hash serves as a fingerprint for the entire dataset, and any modification to underlying data propagates upward through the tree, changing the root hash. This makes tampering immediately detectable and ensures the reliability of data stored in the blockchain.

Historical Context: The alphanumeric sequence (hash) produced is set up so that the same input always generates the same output, but the output cannot be reversed to generate the original information. This creates a mechanism to compare files and verify their identity without manually checking contents, which is crucial for distributed blockchain systems.



3. Structure of Merkle Tree

The structure of a Merkle tree follows a hierarchical, bottom-up architecture that efficiently organizes and cryptographically links transaction data. Understanding this structure is essential for comprehending how blockchain achieves its security and efficiency properties.

Layer-by-Layer Breakdown:

1. Leaf Nodes (Bottom Layer - Transaction Level):

- Contain hashes of individual transactions (designated as T1, T2, T3, T4, etc. in diagrams)
- Each transaction is independently hashed using cryptographic hash functions like SHA-256
- Represent the raw data inputs that need to be verified and secured
- Form the foundation upon which the entire tree is built
- In Bitcoin, each leaf node is a hash of a single transaction, typically 64 bytes (256 bits)

2. Intermediate Nodes (Middle Layers - Branch Level):

- Created by concatenating and hashing pairs of child node hashes
- Mathematical operation: $\text{Hash}(\text{Hash}(\text{Left Child}) + \text{Hash}(\text{Right Child}))$
- Example: If $H(A)$ and $H(B)$ are two child hashes, their parent becomes $H(AB) = \text{Hash}(H(A) + H(B))$
- Each successive level reduces the number of nodes by approximately half
- These nodes are called "branches" and serve as intermediate verification points
- Enable efficient traversal and verification without examining all leaf nodes

3. Root Node (Top Layer - Merkle Root):

- Single hash value that cryptographically represents all transactions in the block
- Calculated by recursively hashing pairs of nodes until only one remains
- Stored in the block header along with timestamp, previous block hash, nonce, and other metadata
- Serves as the ultimate verification point for all data in the tree
- Typically 32 bytes (256 bits) in Bitcoin and similar systems

Construction Process and Rules:

Pairing Mechanism: When building the tree from bottom to top, adjacent hashes are paired and combined. The process follows these steps:

1. Hash all individual transactions to create leaf nodes
2. If the number of nodes at any level is odd, duplicate the last node to create an even pair
3. Concatenate each pair of hashes and hash the result
4. Continue this process recursively until only one hash remains

Handling Odd Numbers: If there is an odd number of transactions, the last transaction is doubled and its hash is concatenated with itself. This padding ensures that every level of the tree maintains proper binary structure. The duplicated nodes are marked as "padding" in the implementation.

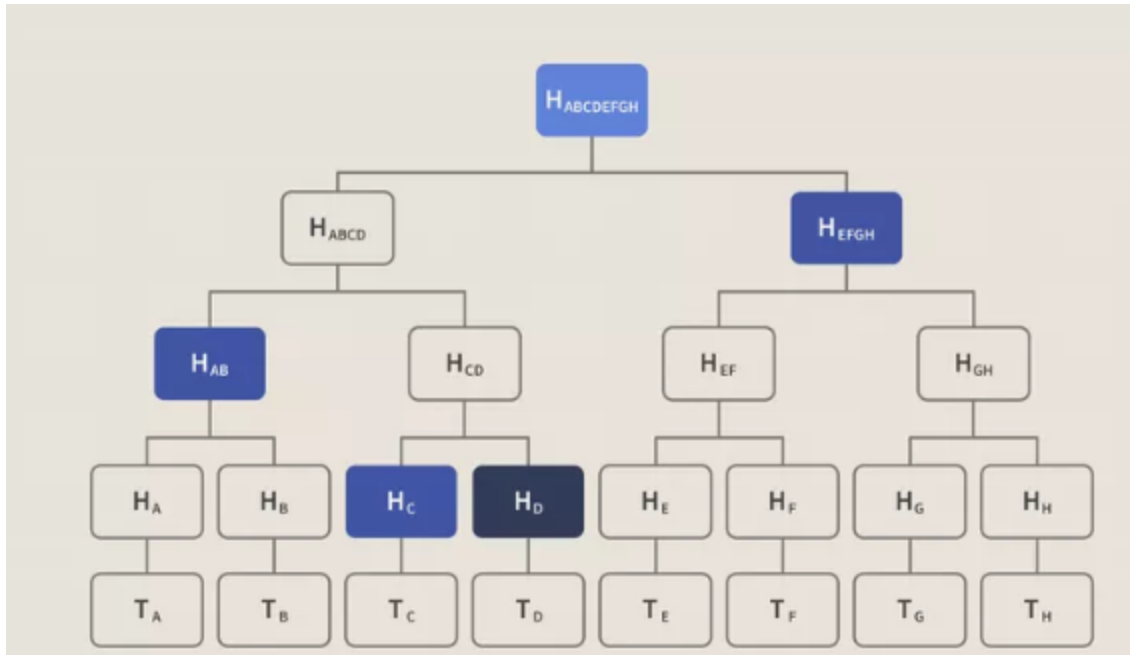
Visual Representation:

Structural Properties:

Depth and Balance:

- The depth of a binary Merkle tree is logarithmic: $\log_2(n)$ where n is the number of transactions
- For 1,000 transactions, depth ≈ 10 levels
- For 1,000,000 transactions, depth ≈ 20 levels
- This logarithmic scaling is what makes Merkle trees so efficient

Space Characteristics: A Merkle tree's size depends on the number of transactions. For example, Bitcoin block #854,473 had 2,530 transactions, and each transaction's hash is 64 bytes. The block's Merkle tree occupied only 161.92 KB of the block's total 1.54 MB, demonstrating the structure's efficiency.



4. Merkle Root

The Merkle root is the cornerstone of the Merkle tree structure and represents one of the most critical components in blockchain architecture. It serves as the ultimate cryptographic summary of all transactions within a block.

Comprehensive Definition: The Merkle root is the single hash value at the apex of the Merkle tree that cryptographically represents all transactions contained within a block. It is generated through a recursive hashing process that combines all transaction hashes into one unified hash. This hash acts as a compact digital fingerprint for potentially thousands of individual transactions.

Generation Process: The Merkle root is created through the following systematic process:

1. Each transaction is individually hashed to create leaf nodes
2. Pairs of leaf hashes are concatenated and hashed to create the next level
3. This pairing and hashing continues recursively through each level
4. The process terminates when only a single hash remains - this is the Merkle root
5. The combined hash represents all transactions through a cryptographic chain of dependencies

Unique Properties and Characteristics:

1. Cryptographic Uniqueness: The Merkle root is absolutely unique to its set of transactions. Any modification to any transaction, no matter how minor, will cascade upward through the tree

structure, producing a completely different Merkle root. This property makes the blockchain tamper-evident.

2. Compact Representation: Despite potentially representing thousands of transactions, the Merkle root occupies only 32 bytes (256 bits) in Bitcoin. This extreme compression allows efficient storage and transmission while maintaining complete cryptographic integrity. For example, the Merkle root of Bitcoin block #854,046 is:

`4c825b4e6a4fea2ea96a1dd879ceff1f854d5be51fa01bb5fd4d95853db9f1bc`

3. Block Header Integration: The Merkle root is stored in the block header alongside other critical information:

- Software version number
- Previous block hash (creating the blockchain linkage)
- Timestamp
- Difficulty target
- Nonce (for proof-of-work mining)

These elements are then hashed together to create the unique block hash, which serves as the block's identifier in the blockchain.

Functional Significance:

Immutability Assurance: The Merkle root ensures that once transactions are added to a block, they cannot be altered without detection. If someone attempts to modify a transaction:

1. The transaction's hash changes
2. This changes its parent node's hash
3. The change propagates upward, affecting all ancestor nodes
4. The Merkle root becomes completely different
5. The block hash becomes invalid
6. The blockchain's consensus mechanism rejects the block

Lightweight Verification: The Merkle root enables Simple Payment Verification (SPV), allowing lightweight clients to verify transactions without downloading entire blocks. Users only need:

- The block header (containing the Merkle root)
- Their transaction
- The Merkle proof (sibling hashes along the path to the root)

Chain Integrity: The Merkle root links the block header to the block body, ensuring complete data integrity. It becomes part of the next block's header through the block hash, creating an

unbreakable cryptographic chain. Merkle roots, trees, and block hashes together create the unalterable chain of transactions and blocks.

Distinction from Block Hash: It's important to understand that the Merkle root and block hash are different entities. The Merkle root represents the transactions within a block, while the block hash (created from the block header, which includes the Merkle root) serves as the block's unique identifier and appears in the subsequent block's header.

5. Working of Merkle Tree

The working mechanism of a Merkle tree involves a sophisticated yet elegant process of hierarchical hashing that enables efficient data verification and maintains cryptographic security throughout the blockchain network.

Detailed Construction Process:

Step 1: Initial Transaction Hashing The process begins at the leaf level where each transaction in a block is individually hashed using a cryptographic hash function (typically SHA-256 in Bitcoin). This creates the foundation layer of the tree. For instance, if there are transactions T1, T2, T3, and T4, each is processed: $H(T1)$, $H(T2)$, $H(T3)$, $H(T4)$. These hashes become the leaf nodes and represent the raw transaction data in cryptographically secured form.

Step 2: Pairing and Concatenation Adjacent leaf node hashes are paired together and concatenated. The concatenation operation combines two hashes into a single string. For example, $H(T1)$ and $H(T2)$ are concatenated to form $H(T1) \parallel H(T2)$, where $\parallel$ represents the concatenation operator. This combined string serves as input for the next hashing operation.

Step 3: Parent Node Generation The concatenated hashes are then hashed again to create parent nodes at the next level. For example:

- $H(AB) = \text{Hash}(H(A) \parallel H(B))$
- $H(CD) = \text{Hash}(H(C) \parallel H(D))$

This process creates the intermediate layer of the tree. Each parent node cryptographically represents both of its children through the hash function's properties.

Step 4: Recursive Upward Propagation The pairing and hashing process continues recursively upward through successive tree levels. Each level contains approximately half the nodes of the level below it:

- Level 0 (leaves): n nodes
- Level 1: $n/2$ nodes
- Level 2: $n/4$ nodes

- Continuing until only one node remains

Step 5: Merkle Root Culmination The recursive process terminates when only a single hash remains at the top of the tree. This final hash is the Merkle root, which cryptographically represents the entire dataset of transactions through a chain of hash dependencies.

Handling Edge Cases: When there's an odd number of nodes at any level, the implementation duplicates the last node to create an even pair. This ensures the binary tree structure remains consistent. The duplicated node is marked as padding to indicate it's a copy rather than unique data.

Verification Mechanism - The Core Innovation:

Transaction Verification Without Complete Data: The true power of Merkle trees lies in their verification capability. To verify that a specific transaction is included in a block, a verifier doesn't need all transactions—only:

1. **The transaction itself** (to hash and verify)
2. **The Merkle root** (from the block header)
3. **The Merkle path/proof** (sibling hashes along the path from transaction to root)

Verification Process Example: Suppose you want to verify that transaction TD exists in a block. The verification protocol works as follows:

1. You have the root hash (HABCDEFGH) from the trusted block header
2. You request proof for TD from the network
3. The network provides: HC, HAB, and HEFGH (the sibling hashes along the path)
4. You compute: $HD = \text{Hash}(TD)$
5. You compute: $HCD = \text{Hash}(HC \parallel HD)$
6. You compute: $HABCD = \text{Hash}(HAB \parallel HCD)$
7. You compute: $HABCDEFGH = \text{Hash}(HABCD \parallel HEFGH)$
8. You compare the computed root with the known root
9. If they match, TD is proven to be in the block

Computational Efficiency: This verification requires only $\log_2(n)$ hashes for n transactions. For 1,000 transactions, only about 10 hashes are needed. For 1,000,000 transactions, only about 20 hashes are required. This logarithmic complexity makes verification extremely efficient even for blocks containing thousands of transactions.

Network Protocol for Data Synchronization:

When distributed systems need to synchronize data, the Merkle tree enables an efficient protocol:

1. **Computer A sends its Merkle root hash to Computer B**
2. **Computer B compares against its own Merkle root**
3. **If roots match:** Data is identical, synchronization complete
4. **If roots differ:** Computer B requests the roots of the two subtrees
5. **Computer A sends the requested subtree roots**
6. **Repeat steps 4-5**, drilling down through the tree until the specific data blocks that differ are identified
7. **Only the inconsistent data blocks are transmitted and corrected**

This protocol minimizes network I/O by transmitting only hashes (small, fixed-size data) instead of complete files. The assumption is that network I/O is slower than local hash computation, which is valid because modern computers can calculate thousands of hashes per millisecond and can parallelize these operations.

Time Complexity Analysis: Modern computers can hash thousands of transactions in milliseconds. This computational efficiency, combined with the logarithmic structure, makes Merkle trees practical for verifying large datasets like those in blockchain systems.

6. Benefits of Merkle Tree

Merkle trees provide numerous critical advantages that make them indispensable in blockchain technology and distributed systems. These benefits address fundamental challenges in data verification, storage, and network efficiency.

1. Efficient Verification and Simplified Payment Verification (SPV):

The primary benefit of Merkle trees is enabling verification of specific transactions without downloading entire blockchain data. This allows users to verify transactions with only a few hashes (logarithmically proportional to the number of transactions).

For lightweight clients and mobile wallets, this is transformative. A user can verify their transaction is included in a block by:

- Downloading only the block header (approximately 80 bytes)
- Requesting the Merkle proof for their transaction ($\log_2(n)$ hashes)
- Verifying the path to the Merkle root

For example, verifying one transaction among 2,000 requires only about 11 hashes (32 bytes each) plus the block header, totaling less than 500 bytes instead of potentially megabytes of full block data. This enables blockchain applications on devices with limited bandwidth and storage.

2. Data Integrity and Tamper Detection:

Merkle trees provide robust data integrity guarantees through their cryptographic structure. Any modification to transaction data causes cascading changes:

- The modified transaction's hash changes
- Its parent node hash changes
- The change propagates upward through all ancestor nodes
- The Merkle root becomes completely different

This makes tampering immediately detectable. The blockchain's consensus mechanism will reject any block with an incorrect Merkle root, ensuring data reliability and preventing fraudulent modifications. This property is fundamental to blockchain's trustless security model.

3. Storage Efficiency and Scalability:

Merkle trees enable significant storage optimization:

- Full nodes can store only block headers containing Merkle roots instead of full transaction data
- A block header is approximately 80 bytes versus potentially megabytes of transaction data
- The Merkle tree itself occupies minimal space relative to the data it represents

For example, Bitcoin block #854,473 with 2,530 transactions had a Merkle tree occupying only 161.92 KB of the block's total 1.54 MB. This efficiency allows blockchain systems to scale while managing storage requirements. It allows limitless transactions to be recorded securely in the network through cryptographic compression.

4. Bandwidth Optimization and Network Efficiency:

Merkle trees dramatically reduce network bandwidth consumption:

- Nodes verify transactions by requesting only relevant Merkle proofs rather than entire blocks
- Synchronization between nodes requires transmitting only the differences identified through Merkle tree comparison
- Hash values (32 bytes) are transmitted instead of full transaction data (potentially kilobytes)

This bandwidth reduction is crucial for blockchain networks with thousands of nodes constantly synchronizing. The structure allows efficient mapping of huge datasets, and small changes can be easily identified without transmitting entire datasets.

5. Logarithmic Verification Complexity:

The time complexity for verification is $O(\log_2 n)$, providing excellent scalability:

- 10 transactions require 4 hashes to verify
- 1,000 transactions require 10 hashes
- 1,000,000 transactions require 20 hashes

This logarithmic scaling means that even as blockchain grows exponentially, verification remains practical. The complexity analysis shows:

Operation	Average Complexity	Worst Case
Space	$O(n)$	$O(n)$
Search	$O(\log_2 n)$	$O(\log_k n)$
Insertion	$O(\log_2 n)$	$O(\log_k n)$
Verification	$O(\log_2 n)$	$O(\log_2 n)$
Synchronization	$O(\log_2 n)$	$O(n)$ worst case

7. Use of Merkle Tree in Blockchain

Merkle trees are deeply integrated into blockchain architecture, serving multiple critical functions that enable blockchain's security, efficiency, and scalability properties. Their application spans from basic transaction organization to advanced state management.

1. Block Structure and Transaction Organization:

In blockchain networks like Bitcoin and Ethereum, the Merkle tree forms an essential component of block structure:

Block Header Integration:

- The Merkle root is included in each block header alongside timestamp, previous block hash, nonce, and difficulty target
- Block headers are approximately 80 bytes and can be easily transmitted and stored
- The Merkle root links all transactions in the block to the block header
- This creates a cryptographic connection between the block header (which chains blocks together) and the block body (which contains transactions)

Transaction Inclusion:

- All transactions in a block are organized into a Merkle tree structure
- Bitcoin hashes each transaction separately rather than all at once
- Each transaction is hashed, paired, and recursively hashed until one hash remains
- If there's an odd number of transactions, one transaction is doubled and its hash concatenated with itself

Block Hash Generation: The Merkle root combines with other block header elements and is hashed to create the unique block hash. For example, in Bitcoin block #854,046:

- Merkle root:
`4c825b4e6a4fea2ea96a1dd879ceff1f854d5be51fa01bb5fd4d95853db9f1bc`
- Combined with version, previous hash, time, difficulty, and nonce
- Hashed to create block hash:
`0000000000000000000000005d886429368c23489583edfd77d0bffffecef8b570d00`

This block hash appears in the next block's header, creating the blockchain linkage.

2. Simplified Payment Verification (SPV) - Lightweight Clients:

SPV is one of the most important applications of Merkle trees in blockchain:

Mobile and Lightweight Wallets:

- Users can verify their transactions without running full nodes
- Only block headers (80 bytes each) need to be downloaded
- To verify a transaction, users request the Merkle proof from full nodes
- The proof consists of $\log_2(n)$ sibling hashes along the path to the root

Verification Process:

1. Lightweight client receives a transaction
2. Requests Merkle proof from full node
3. Reconstructs the path from transaction to Merkle root
4. Compares computed root with the root in the block header
5. If they match, transaction is confirmed as included in the block

This makes cryptocurrency wallets practical on mobile devices with limited storage and bandwidth, enabling widespread blockchain adoption.

3. Peer-to-Peer Network Synchronization:

When nodes synchronize with the blockchain network, Merkle trees enable efficient data verification:

Initial Synchronization:

- New nodes can download block headers quickly (small size)
- Full transaction data can be verified against Merkle roots
- Only blocks relevant to the node's interests need full download
- Merkle proofs allow selective verification of specific transactions

Ongoing Synchronization:

- Nodes can efficiently verify new blocks as they're added
- Transaction verification requires only Merkle proofs, not full block downloads
- Network bandwidth is conserved through selective data requests

4. State Management in Ethereum - Merkle Patricia Trie:

Ethereum uses a modified Merkle tree called the Merkle Patricia Trie for sophisticated state management:

Three Separate Tries:

- **State Trie:** Stores account balances, nonces, contract code, and storage
- **Transaction Trie:** Organizes all transactions in a block
- **Receipt Trie:** Records transaction receipts and logs

Benefits for Ethereum:

- Enables efficient verification of account states without processing entire blockchain
- Supports complex smart contract interactions
- Allows light clients to query contract storage
- Facilitates efficient state updates across network

5. Fraud Proofs in Layer-2 Scaling Solutions:

Merkle trees enable fraud detection in blockchain scaling solutions:

Optimistic Rollups:

- Transaction batches are posted to Layer-1 with Merkle roots
- If invalid transactions are included, anyone can generate a fraud proof

- The proof consists of the invalid transaction and its Merkle path
- This allows efficient verification without processing all transactions

Validity Proofs:

- ZK-Rollups use Merkle trees to commit to state transitions
- Proofs reference specific positions in Merkle trees
- Enables succinct verification of large batches of transactions

8. Use Cases of Merkle Tree

Merkle trees have diverse applications across blockchain, distributed systems, and data verification domains. Their efficiency in handling large datasets and enabling secure verification makes them invaluable in numerous real-world scenarios.

1. Bitcoin and Cryptocurrency Transaction Verification:

Bitcoin Implementation: Bitcoin is the most prominent example of Merkle tree usage in cryptocurrency. The blockchain uses Merkle trees to organize thousands of transactions within each block efficiently.

Practical Application:

- Each block can contain hundreds to thousands of transactions
- All transactions are organized into a binary Merkle tree
- The Merkle root is included in the 80-byte block header
- SPV wallets verify payments with minimal data downloads
- Mobile Bitcoin wallets can operate without storing the entire blockchain (over 400 GB)

User Benefits: Users running lightweight wallets can verify their transactions are included in blocks by downloading only:

- Block headers (approximately 80 bytes each)
- Merkle proofs for their specific transactions (a few hundred bytes)
- Total: Less than 1 KB versus potentially megabytes for full blocks

This makes mobile cryptocurrency wallets practical and enables global financial inclusion for users with limited connectivity or device capabilities.

2. Git Version Control System:

Source Code Management: Git, the world's most popular version control system used by millions of programmers, employs Merkle tree structures to track code changes efficiently.

Implementation Details:

- Each commit contains a Merkle tree (though Git calls them "tree objects")
- Files are hashed and organized hierarchically
- Directory structures map to tree nodes
- The root hash represents the entire codebase state at that commit

Benefits for Developers:

- Enables efficient version comparison without examining every file
- Detects changes instantly through root hash comparison
- Ensures code integrity and prevents tampering
- Allows distributed collaboration with verification
- Makes repository cloning and synchronization efficient

Practical Example: When a developer commits changes, Git computes hashes only for modified files, then updates the affected branches of the tree, and generates a new root hash. Comparing two commits requires only comparing root hashes initially, then drilling down through the tree to identify specific changes.

3. Distributed File Systems - IPFS (InterPlanetary File System):

Content-Addressed Storage: IPFS uses Merkle DAGs (Directed Acyclic Graphs, a generalization of Merkle trees) to store and retrieve files in a distributed manner.

Mechanism:

- Files are split into chunks and hashed
- Chunks are organized into Merkle DAG structures
- The root hash serves as the file's content identifier
- Files are retrieved using their cryptographic hashes rather than location-based URLs

Advantages:

- Ensures content addressing (same content always has same hash)
- Enables deduplication (identical content stored once)
- Provides tamper-proof file distribution
- Allows partial file verification without downloading complete files
- Supports efficient peer-to-peer file sharing

4. Database Synchronization and Consistency:

Apache Cassandra and Distributed Databases: Distributed databases use Merkle trees for anti-entropy repairs and replica synchronization.

Application:

- Cassandra uses Merkle trees to detect inconsistencies between database replicas
- Different database nodes maintain potentially divergent data
- Merkle trees identify exactly which data ranges differ between replicas
- Only inconsistent data is transferred and synchronized

Process:

1. Each replica builds a Merkle tree of its data ranges
2. Replicas exchange root hashes
3. If roots differ, they exchange subtree hashes
4. Process continues until specific inconsistent data blocks are identified
5. Only those blocks are synchronized

Amazon DynamoDB: Similarly uses Merkle trees for replica comparison and consistency maintenance across globally distributed data centers.

Real-World Impact: Global websites and applications maintain databases across continents. When one database is updated, every other database needs identical updates. Merkle trees make this verification efficient, ensuring data consistency for millions of users worldwide.

5. Certificate Transparency and Public Key Infrastructure:

SSL/TLS Certificate Verification: Certificate Transparency logs use Merkle trees to maintain transparent, append-only logs of SSL/TLS certificates.

Implementation:

- All issued certificates are added to public Merkle tree logs
- Browsers can verify certificates against these logs
- Merkle proofs demonstrate certificate inclusion
- Detects fraudulently issued certificates

Security Benefits:

- Makes certificate issuance transparent and auditable
- Prevents certificate authorities from secretly issuing fraudulent certificates
- Enables efficient monitoring of the entire certificate ecosystem
- Protects users from man-in-the-middle attacks

•

CODE:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib

def calculate_sha256_hash(input_string):
    """
    Computes the SHA-256 hash of the given input string.

    Args:
        input_string (str): The string to be hashed.

    Returns:
        str: The SHA-256 hash of the input string in hexadecimal format.
    """
    # 2. Convert the input_string to bytes using .encode('utf-8')
    encoded_string = input_string.encode('utf-8')

    # 3. Compute the SHA-256 hash of the encoded string
    hash_object = hashlib.sha256(encoded_string)

    # 4. Convert the hash object to a hexadecimal string
    hex_dig = hash_object.hexdigest()

    # 5. Return the hexadecimal hash string
    return hex_dig

# Demonstrate the function with an example
example_string = "Hello, DSA!"
sha256_result = calculate_sha256_hash(example_string)

print(f"Original String: {example_string}")
print(f"SHA-256 Hash: {sha256_result}")
```

OUTPUT:

```
... Original String: Hello, DSA!  
SHA-256 Hash: b0917c6f50094c63bc805247b95f3269ca136d5abd0b46fd6ce159525022bf31
```

2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
def generate_hash_with_nonce(input_string, nonce):  
    """  
    Concatenates an input string with a nonce and computes its SHA-256  
    hash.  
  
    Args:  
        input_string (str): The base string data.  
        nonce (int or str): The nonce to be appended.  
  
    Returns:  
        str: The SHA-256 hash of the combined string.  
    """  
    # 2. Concatenate the input_string and the string representation of the  
    nonce  
    combined_string = input_string + str(nonce)  
  
    # 3. Utilize the previously defined calculate_sha256_hash function  
    combined_hash = calculate_sha256_hash(combined_string)  
  
    # 4. Return the computed SHA-256 hash  
    return combined_hash  
  
# 5. Demonstrate the function's usage  
example_data = "Blockchain Data"  
example_nonce = 12345  
  
combined_hash_result = generate_hash_with_nonce(example_data,  
example_nonce)  
  
print(f"Original Data: {example_data}")  
print(f"Nonce: {example_nonce}")  
print(f"Combined SHA-256 Hash: {combined_hash_result}")
```

OUTPUT:

Original Data: Blockchain Data Nonce: 12345 Combined SHA-256 Hash: 748632215d0392e2823bfa4432e5ec51420ea8097d4fe0da331dbc2952573a39

3. Proof-of-Work Puzzle Solving: Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```
def find_proof_of_work(input_string, difficulty):
    """
    Finds a nonce such that the SHA-256 hash of the combined input_string
    and nonce
    starts with a specified number of leading zeros.

    Args:
        input_string (str): The base string data.
        difficulty (int): The number of leading zeros required in the
    hash.

    Returns:
        tuple: A tuple containing the found nonce (int) and the resulting
    hash (str).
    """
    nonce = 0
    target_prefix = '0' * difficulty

    print(f"Searching for a nonce for '{input_string}' with difficulty
    {difficulty}...")

    while True:
        # 4. Call the generate_hash_with_nonce function
        current_hash = generate_hash_with_nonce(input_string, nonce)

        # 5. Check if the generated hash starts with the target number of
    leading zeros
        if current_hash.startswith(target_prefix):
            # 6. If it meets the difficulty, return the nonce and the hash
            return nonce, current_hash

        # 7. If not, increment the nonce and continue the loop
        nonce += 1
```

```
# 8. Demonstrate the function's usage
data = "Proof of Work Example"
difficulty_level = 4

found_nonce, final_hash = find_proof_of_work(data, difficulty_level)

print("\nProof of Work found!")
print(f"Input Data: {data}")
print(f"Difficulty (leading zeros): {difficulty_level}")
print(f"Nonce found: {found_nonce}")
print(f"Resulting Hash: {final_hash}")
```

OUTPUT:

```
*** Searching for a nonce for 'Proof of Work Example' with difficulty 4...

Proof of Work found!
Input Data: Proof of Work Example
Difficulty (leading zeros): 4
Nonce found: 22730
Resulting Hash: 00001cd4acd24473763fc7f4a7796d8f0e46ac9006545ddb8d4623ace64cdba8
```

4. Merkle Tree Construction: Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
def construct_merkle_tree(transactions):
    """
    Constructs a Merkle Tree from a list of transactions and returns the
    Merkle Root.

    Args:
        transactions (list of str): A list of transaction strings.

    Returns:
        str: The SHA-256 Merkle Root of the transactions.
    """
    # 1. Initialize leaf_hashes by applying calculate_sha256_hash to each
    transaction
    current_level_hashes = [calculate_sha256_hash(tx) for tx in
    transactions]
```

```

print(f"Initial leaf hashes: {current_level_hashes}")

# 2. Implement a while loop that continues as long as there's more
than one hash
while len(current_level_hashes) > 1:
    # a. Check if the number of hashes is odd, and duplicate the last
one if it is
    if len(current_level_hashes) % 2 != 0:
        current_level_hashes.append(current_level_hashes[-1])

    # b. Initialize an empty list for the next level of hashes
    next_level_hashes = []

    # c. Iterate through current_level_hashes in steps of two
    for i in range(0, len(current_level_hashes), 2):
        hash1 = current_level_hashes[i]
        hash2 = current_level_hashes[i+1]

        # i. Concatenate hash1 and hash2
        combined_hash_input = hash1 + hash2

        # ii. Compute the SHA-256 hash of the combined string
        new_hash = calculate_sha256_hash(combined_hash_input)

        # iii. Append the resulting hash to next_level_hashes
        next_level_hashes.append(new_hash)

    current_level_hashes = next_level_hashes
    print(f"Next level hashes: {current_level_hashes}")

# 4. Once the loop finishes, return the single hash, which is the
Merkle root
return current_level_hashes[0]

# 5. Demonstrate the function's usage
example_transactions = [
    "transaction A",
    "transaction B",
    "transaction C",
    "transaction D",

```



```

        "transaction E"
    ]

merkle_root = construct_merkle_tree(example_transactions)

# 6. Print the example transactions and the calculated Merkle root
print("\n--- Merkle Tree Construction Result ---")
print(f"Example Transactions: {example_transactions}")
print(f"Calculated Merkle Root: {merkle_root}")

```

OUTPUT:

```

... Initial leaf hashes: ['d9d7f1e9b9b882a23f57b5a2395acab1aba9b6795ca11fb648a11857f3c719d8', '81fbaccaedbf55887ae389359b01630d5c3501e737f20d51f5baa34337
Next level hashes: ['1e0f12c87d8fc5a2373f91158c695cc3bd1a1ab8288ff32e2afb92c73f71dd3f', '3a9227889b51d36adc15a47df811aa3b9005da3b5d11f3c4efdc859a4118a
Next level hashes: ['218db2acc809f2124704f5765b6cec388da45578388a79d19f3a2f62bd2d2f13', 'b5603ae8f80594929afbca7832167f0f823f931ee5281cee7f96567dea504
Next level hashes: ['006bb45e777f2ad28ff55ff31a7c0a97638d84f0455c4742857430238e2c2249']

--- Merkle Tree Construction Result ---
Example Transactions: ['transaction A', 'transaction B', 'transaction C', 'transaction D', 'transaction E']
Calculated Merkle Root: 006bb45e777f2ad28ff55ff31a7c0a97638d84f0455c4742857430238e2c2249

```

Conclusion:

Merkle trees' versatility stems from their fundamental properties: efficient verification, data integrity, and logarithmic scalability. These use cases demonstrate that Merkle trees are useful in distributed systems where the same data should exist in multiple places, and they can be used to check inconsistencies efficiently. From cryptocurrency transactions to version control, from database synchronization to supply chain verification, Merkle trees provide the cryptographic foundation for trust and efficiency in distributed systems. Their continued relevance spans beyond blockchain into any application requiring verifiable data management at scale.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 2

AIM: Create a Blockchain using Python.

THEORY:

1. What is Blockchain?

Blockchain is a **decentralized, distributed digital ledger** that securely records transactions across multiple computers (nodes). Transactions are grouped into blocks, and each block is linked to the previous one using **cryptographic hashes**, forming a continuous chain.

Once data is recorded and validated, it becomes **immutable**, meaning it is extremely difficult to alter or delete.

Key Features of Blockchain:

- **Decentralization:** No single authority controls the blockchain; it is maintained by a network of nodes.
- **Transparency:** Transactions are visible to all authorized participants.
- **Immutability:** Any attempt to modify data changes the block's hash and breaks the chain.
- **Security:** Transactions are verified by majority consensus.
- **Distributed Ledger:** Every node maintains a copy of the blockchain.

2. What is a Block in Blockchain?

A **block** is a fundamental unit of a blockchain, similar to a link in a chain. It stores a group of verified transactions along with metadata and is securely connected to the previous block using cryptographic hashing.

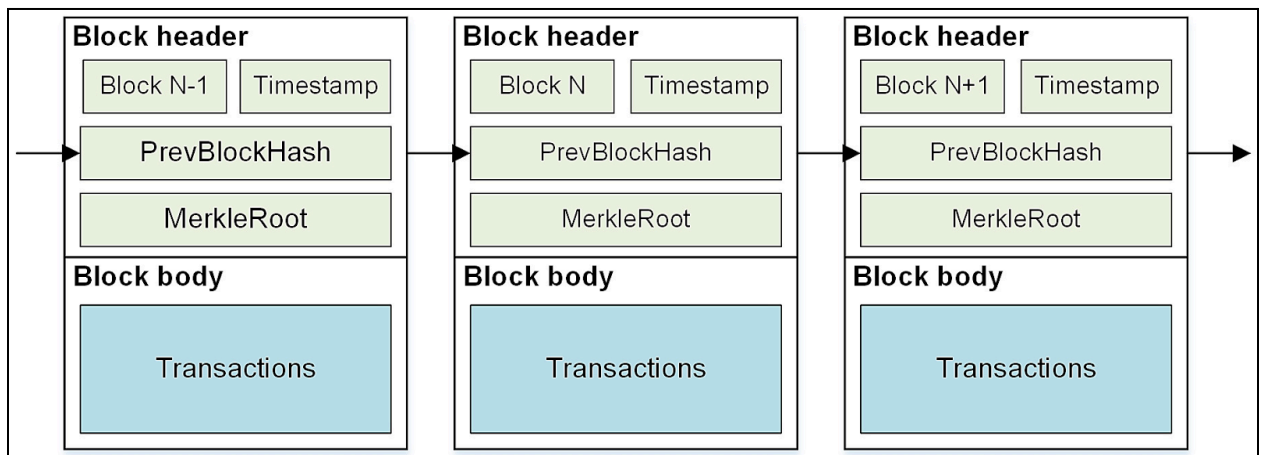
Blocks help manage and track the massive number of transactions occurring globally by organizing them into a secure and structured format.

What are the Components of a Block

A block consists of the following components:

1. **Block Header**
Identifies the block within the blockchain. It contains essential metadata and is repeatedly hashed by miners during mining by changing the nonce value.
2. **Previous Block Hash**
A reference to the hash of the previous block, linking blocks together and ensuring chain integrity.

3. **Timestamp**
Records the exact date and time when the block was created, ensuring chronological order.
4. **Nonce**
A number used only once. Miners modify the nonce during Proof-of-Work to generate a valid hash that meets the difficulty target.
5. **Merkle Root**
A cryptographic hash representing all transactions in the block. It allows efficient and secure verification of transactions.



Example of a Block Structure

Block Number: 105678

Block Header:

- Previous Block Hash: 000000000000000000a3f2c9e7b1d45c8fa9a1e7b6d9c2f0a4e3b1c8d9f
- Merkle Root: 4f3c2a1b9e8d7c6b5a4f3e2d1c9b8a7f6e5d4c3b2a1f9e8d7c6b5a4
- Timestamp: 2026-01-29 14:32:10 UTC
- Nonce: 845932
- Block Hash: 00000000000000000005d6e8f3c2a9b1e7d4f6c8a2e9b5d3f1c7...

Transactions:

1. Alice → Bob : 0.5 BTC
2. Charlie → Dave : 1.2 BTC
3. Eve → Frank : 0.3 BTC

3. What is the Process of Mining in Blockchain?

Mining is the process of **adding a new block to the blockchain** using the Proof-of-Work (PoW) mechanism.

Steps in Mining:

Step 1: Collect Transactions

- Pending transactions are gathered into a block.
- Example: Alice sends 50 coins to Bob.

Step 2: Create Block Header

- Includes previous hash, transaction data, timestamp, and nonce.

Step 3: Proof of Work (PoW)

- Miners repeatedly change the nonce to generate a hash.
- The hash must satisfy the difficulty condition (e.g., leading zeroes).

SHA-256(block data + nonce) → 0000ab34cd...

Step 4: Validate and Broadcast

- The valid block is broadcast to the network.
- Other nodes verify the hash, nonce, and previous hash link.

Step 5: Add to Blockchain

- Once verified, the block is added permanently.
- The miner receives a cryptocurrency reward.

4. How is the Validity of a Block Checked in Blockchain

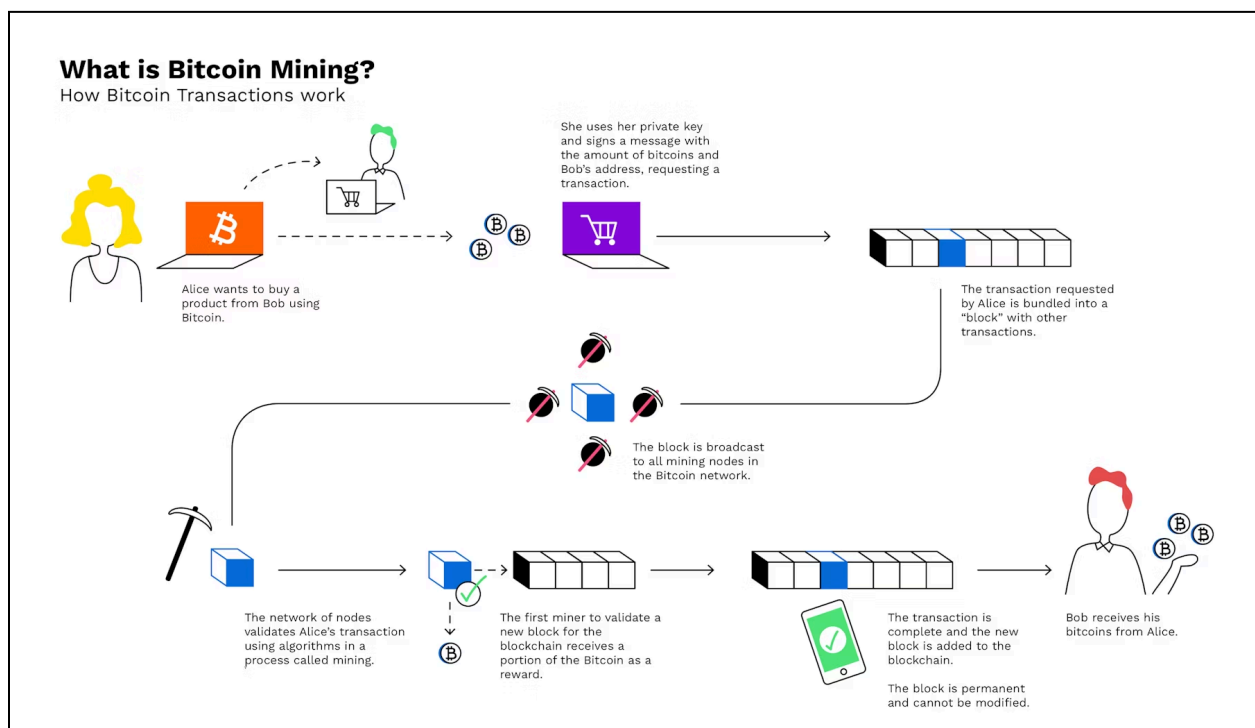
A block is validated through multiple verification steps:

1. **Verify Block Hash**
 - Recalculate the hash and compare it with the stored hash.
2. **Check Proof-of-Work**
 - Ensure the hash meets the difficulty requirement (leading zeroes).
3. **Validate Previous Block Hash**
 - Confirm the previous hash matches the last block's hash.
4. **Verify Transactions**
 - Check transaction authenticity, digital signatures, and prevent double spending.
5. **Verify Merkle Root**

- Recalculate and compare the Merkle root from transactions.
6. **Check Timestamp**
 - Timestamp must be valid and not in the future.
 7. **Check Block Size and Format**
 - Block must follow protocol-defined size and structure.
 8. **Validate Genesis Block**
 - The first block is hardcoded with previous hash = “0” and must remain unchanged.

Final Result:

- All checks pass → Block is valid
- Any check fails → Block is rejected



CODE:

```
import datetime
import hashlib
import json
from flask import Flask, jsonify
```

```
class Blockchain:
```

```
    def __init__(self):
        self.chain = []
        self.transactions = []
        self.create_block(0, '0')
```

```
    def create_block(self, proof, previous_hash):
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,                # Golden nonce
            'transactions': self.transactions,
            'previous_hash': previous_hash
        }
        self.transactions = []            # Clear transactions
        self.chain.append(block)
        return block
```

```
    def get_previous_block(self):
        return self.chain[-1]
```

```
# Transactions created internally
```

```
    def create_transaction(self):
        self.transactions.append({
            'sender': 'SYSTEM',
            'receiver': 'MINER',
            'amount': 10
        })
```

```
# Proof of Work: hash must start with "000"
```

```
    def proof_of_work(self, previous_hash, transactions):
        nonce = 0
        while True:
            data = {
                'nonce': nonce,
                'transactions': transactions,
                'previous_hash': previous_hash
            }
            hash_val = hashlib.sha256(
                json.dumps(data, sort_keys=True).encode()
            ).hexdigest()
            if hash_val[:3] == '000':
```

```

        return nonce
        nonce += 1

def hash(self, block):
    return hashlib.sha256(
        json.dumps(block, sort_keys=True).encode()
    ).hexdigest()

def is_chain_valid(self):
    for i in range(1, len(self.chain)):
        prev = self.chain[i - 1]
        curr = self.chain[i]

        if curr['previous_hash'] != self.hash(prev):
            return False

    data = {
        'nonce': curr['proof'],
        'transactions': curr['transactions'],
        'previous_hash': curr['previous_hash']
    }
    hash_val = hashlib.sha256(
        json.dumps(data, sort_keys=True).encode()
    ).hexdigest()

    if hash_val[:3] != '000':
        return False

    return True

# =====
# Flask App
# =====

app = Flask(__name__)
blockchain = Blockchain()

@app.route('/')
def home():
    return "Blockchain API is running"

@app.route('/add_transaction', methods=['GET'])
def add_transaction():
    blockchain.create_transaction()

```



```

    return jsonify({'message': 'Transaction added internally'}), 200

@app.route('/mine_block', methods=['GET'])
def mine_block():
    if len(blockchain.transactions) == 0:
        return jsonify({'message': 'No transactions to mine'}), 400

    prev_block = blockchain.get_previous_block()
    prev_hash = blockchain.hash(prev_block)

    proof = blockchain.proof_of_work(prev_hash, blockchain.transactions)
    block = blockchain.create_block(proof, prev_hash)

    return jsonify({
        'message': 'Block mined successfully!',
        'block': block
    }), 200

@app.route('/get_chain', methods=['GET'])
def get_chain():
    return jsonify({
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }), 200

@app.route('/is_valid', methods=['GET'])
def is_valid():
    return jsonify({
        'valid': blockchain.is_chain_valid()
    }), 200

# =====
# Run App
# =====

if __name__ == '__main__':
    app.run(debug=True)

```

OUTPUT:

← ↻ ⓘ 127.0.0.1:5000/add_transaction

Pretty-print ☐

```
{
  "message": "Transaction added internally"
}
```

← ↻ ⓘ 127.0.0.1:5000/mine_block

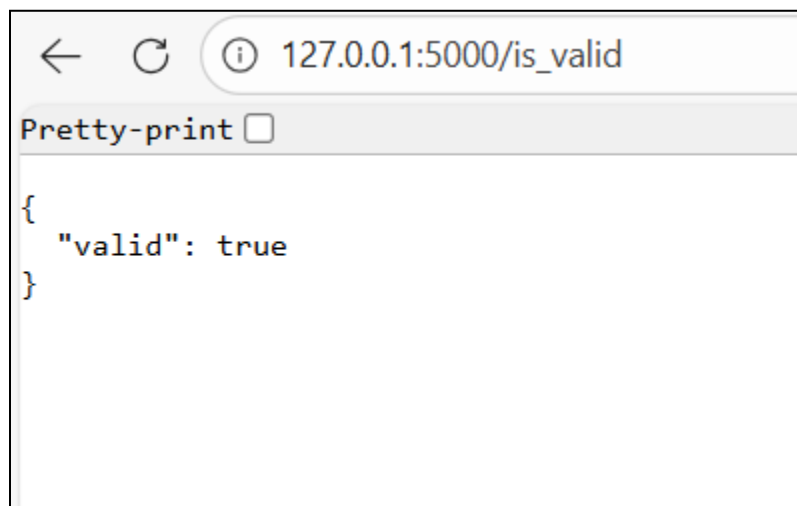
Pretty-print ☐

```
{
  "block": {
    "index": 2,
    "previous_hash": "207828c62cd49627f6edb41c7eebbb51914eb892cb0b2c5ba71296c54ad228aa",
    "proof": 14602,
    "timestamp": "2026-02-03 03:06:07.797595",
    "transactions": [
      {
        "amount": 10,
        "receiver": "MINER",
        "sender": "SYSTEM"
      }
    ]
  },
  "message": "Block mined successfully!"
}
```



```
← 127.0.0.1:5000/get_chain
Pretty-print ☐

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 0,
      "timestamp": "2026-02-03 03:04:38.829887",
      "transactions": []
    },
    {
      "index": 2,
      "previous_hash": "207828c62cd49627f6edb41c7eebbb51914eb892cb0b2c5ba71296c54ad228aa",
      "proof": 14602,
      "timestamp": "2026-02-03 03:06:07.797595",
      "transactions": [
        {
          "amount": 10,
          "receiver": "MINER",
          "sender": "SYSTEM"
        }
      ]
    }
  ],
  "length": 2
}
```



```
← 127.0.0.1:5000/is_valid
Pretty-print ☐

{
  "valid": true
}
```

CONCLUSION:

In this experiment, a basic blockchain was successfully implemented using Python. The blockchain demonstrates core concepts such as block structure, hashing, Proof-of-Work mining, and chain validation. By linking blocks through cryptographic hashes and verifying them using difficulty rules, the system ensures data integrity and immutability. This experiment provides a clear understanding of how blockchain works in a decentralized and secure manner.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 3

AIM: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

THEORY:

1. Blockchain Overview

A **blockchain** is a **distributed, decentralized, and cryptographically secured digital ledger** used to record transactions in a chronological and immutable manner. Unlike traditional centralized databases that rely on a trusted third party, blockchain operates on a **peer-to-peer network**, where trust is established through cryptographic verification and consensus protocols.

Each block in a blockchain contains:

- A **set of verified transactions**
- A **timestamp** indicating the time of block creation
- The **hash of the previous block**, which links blocks together
- A **nonce**, used in the mining process
- The block's **own cryptographic hash**

The linking of blocks using cryptographic hashes creates a chain structure. Any attempt to modify data in a block changes its hash, which invalidates all subsequent blocks. This feature ensures **immutability, data integrity, and tamper resistance**. Because copies of the blockchain are stored across multiple nodes, transparency and fault tolerance are also achieved.

2. Mining

Mining is the process by which new blocks are created, validated, and added to the blockchain. It plays a crucial role in maintaining the security and reliability of the blockchain network. Mining performs two major functions:

1. Verification and validation of transactions
2. Securing the blockchain against malicious attacks

Mining is based on the **Proof-of-Work (PoW)** algorithm. In PoW, miners compete to solve a

computationally complex mathematical puzzle. This involves repeatedly modifying a value called a **nonce** and recalculating the block's hash until it meets the network's difficulty criteria (such as a specific number of leading zeroes).

The difficulty level dynamically adjusts to control the rate at which blocks are mined. Once a miner successfully finds a valid hash, the block is broadcast to all other nodes for verification. Due to the high computational cost involved, PoW makes attacks like double spending and data manipulation extremely difficult.

3. Multi-Node Blockchain Network

A **multi-node blockchain network** consists of several independent nodes participating in transaction validation and block propagation. In this experiment, three nodes are deployed on different ports (5001, 5002, and 5003) to simulate a real-world decentralized environment.

Each node in the network:

- Operates autonomously
- Maintains a complete copy of the blockchain
- Communicates with peer nodes using a peer-to-peer protocol

This architecture eliminates the need for a central server and ensures **decentralization**, **data redundancy**, and **high availability**. Even if one node fails or behaves maliciously, the network continues to function correctly. This setup clearly demonstrates how blockchain systems maintain reliability and consistency across distributed systems.

4. Consensus Mechanism

A **consensus mechanism** is a protocol that enables all nodes in a blockchain network to agree on a single, consistent version of the ledger. Since nodes operate independently and blocks may be mined simultaneously, temporary inconsistencies or forks can occur.

In this system, the **Longest Chain Rule** is used as the consensus mechanism. According to this rule:

- The blockchain with the maximum number of blocks is considered valid
- The longest chain represents the greatest cumulative computational effort
- Nodes discard shorter chains and adopt the longest valid chain

This rule ensures eventual consistency and prevents permanent divergence in the blockchain. Consensus mechanisms are essential for maintaining trust, preventing double spending, and ensuring the integrity of distributed ledgers.

5. Transactions and Mining Reward

A **transaction** is a digitally signed record representing the transfer of value between participants in a blockchain network. Each transaction contains:

- Sender's address
- Receiver's address
- Transaction amount

Transactions are verified using cryptographic techniques and temporarily stored in a transaction pool until they are included in a block.

To incentivize miners, the system introduces a **mining reward mechanism**. When a block is successfully mined, a special transaction called a **coinbase transaction** is created. This transaction generates new cryptocurrency and awards it to the miner. The reward mechanism encourages honest participation and provides economic security to the network.

6. Chain Replacement

Chain replacement is a synchronization mechanism that ensures all nodes in the blockchain network maintain a consistent and up-to-date ledger. Due to network latency or simultaneous mining, nodes may temporarily have different versions of the blockchain.

When the `/replace_chain` endpoint is invoked:

1. A node requests blockchain data from all peer nodes
2. The received chains are validated for correctness and integrity
3. The node replaces its own blockchain if a longer and valid chain is found

This mechanism resolves conflicts, eliminates forks, and ensures that all nodes converge to a single authoritative blockchain. Chain replacement is essential for maintaining decentralization without central coordination.

CODE


```

import datetime

import hashlib

import json

from flask import Flask,jsonify,request

import requests

from uuid import uuid4

from urllib.parse import urlparse

class Blockchain:

    def __init__(self):

        self.chain=[]

        self.transactions=[]

        self.create_block(proof=1,previous_hash='0')

        self.nodes=set()

    def create_block(self,proof,previous_hash):

        block={'index':len(self.chain)+1,'timestamp':str(datetime.datetime.now()),'proof':proof,'previous_hash':previous_hash,'transactions':self.transactions}

        self.transactions=[]

        self.chain.append(block)

        return block

    def get_previous_block(self):

        return self.chain[-1]

    def proof_of_work(self,previous_proof):

        new_proof=1

        check_proof=False

```

```

while not check_proof:

    hash_operation=hashlib.sha256(str(new_proof**2-previous_proof**2).encode()).hexdigest()

    if hash_operation[:4]=='0000':

        check_proof=True

    else:

        new_proof+=1

return new_proof

def hash(self,block):

    encoded_block=json.dumps(block,sort_keys=True).encode()

    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self,chain):

    previous_block=chain[0]

    block_index=1

    while block_index<len(chain):

        block=chain[block_index]

        if block['previous_hash']!=self.hash(previous_block):

            return False

        previous_proof=previous_block['proof']

        proof=block['proof']

        hash_operation=hashlib.sha256(str(proof**2-previous_proof**2).encode()).hexdigest()

        if hash_operation[:4]!='0000':

            return False

        previous_block=block

        block_index+=1

```

```

    return True

def add_transaction(self,sender,receiver,amount):

    self.transactions.append({'sender':sender,'receiver':receiver,'amount':amount})

    previous_block=self.get_previous_block()

    return previous_block['index']+1

def add_node(self,address):

    parsed_url=urlparse(address)

    self.nodes.add(parsed_url.netloc)

def replace_chain(self):

    network=self.nodes

    longest_chain=None

    max_length=len(self.chain)

    for node in network:

        response=requests.get(f'http://{node}/get_chain')

        if response.status_code==200:

            length=response.json()['length']

            chain=response.json()['chain']

            if length>max_length and self.is_chain_valid(chain):

                max_length=length

                longest_chain=chain

    if longest_chain:

        self.chain=longest_chain

    return True

return False

```

```

app=Flask(__name__)

node_address=str(uuid4()).replace('-', '')

blockchain=Blockchain()

@app.route('/mine_block',methods=['GET'])

def mine_block():

    previous_block=blockchain.get_previous_block()

    previous_proof=previous_block['proof']

    proof=blockchain.proof_of_work(previous_proof)

    previous_hash=blockchain.hash(previous_block)

    blockchain.add_transaction(sender=node_address,receiver='Richard',amount=1)

    block=blockchain.create_block(proof,previous_hash)

    response={'message':'Congratulations, you just mined a
    block!','index':block['index'],'timestamp':block['timestamp'],'proof':block['proof'],'previous_hash':block['previous_ha
    sh'],'transactions':block['transactions']}

    return jsonify(response),200

@app.route('/add_transaction',methods=['POST'])

def add_transaction():

    json_data=request.get_json()

    transaction_keys=['sender','receiver','amount']

    if not all(key in json_data for key in transaction_keys):

        return 'Some elements of the transaction are missing',400

    index=blockchain.add_transaction(json_data['sender'],json_data['receiver'],json_data['amount'])

    response={'message':f'This transaction will be added to Block {index}'}

    return jsonify(response),201

@app.route('/connect_node',methods=['POST'])

```

```

def connect_node():

    json_data=request.get_json()

    nodes=json_data.get('nodes')

    if nodes is None:

        return "No node",400

    for node in nodes:

        blockchain.add_node(node)

    response={'message':'All the nodes are now connected.','total_nodes':list(blockchain.nodes)}

    return jsonify(response),201

@app.route('/replace_chain',methods=['GET'])

def replace_chain():

    is_chain_replaced=blockchain.replace_chain()

    if is_chain_replaced:

        response={'message':'The chain was replaced by the longest one.','new_chain':blockchain.chain}

    else:

        response={'message':'The chain is already the largest one.','actual_chain':blockchain.chain}

    return jsonify(response),200

app.run(host='0.0.0.0',port=5000)

```

OUTPUT:

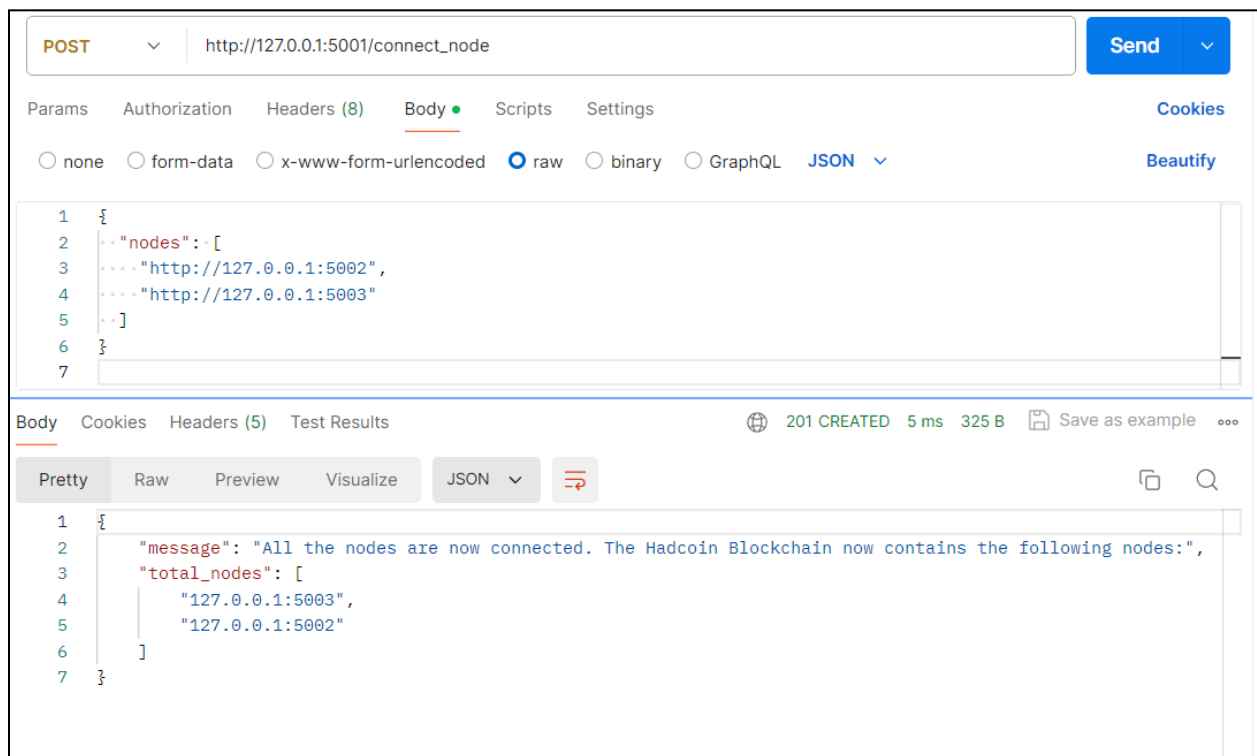
1. Connect Nodes (POST)

URL (from any node):

POST `http://127.0.0.1:5001/connect_node`

Body → raw → JSON

```
{
  "nodes": [
    "http://127.0.0.1:5002",
    "http://127.0.0.1:5003"
  ]
}
```



2. Add Transaction (POST)

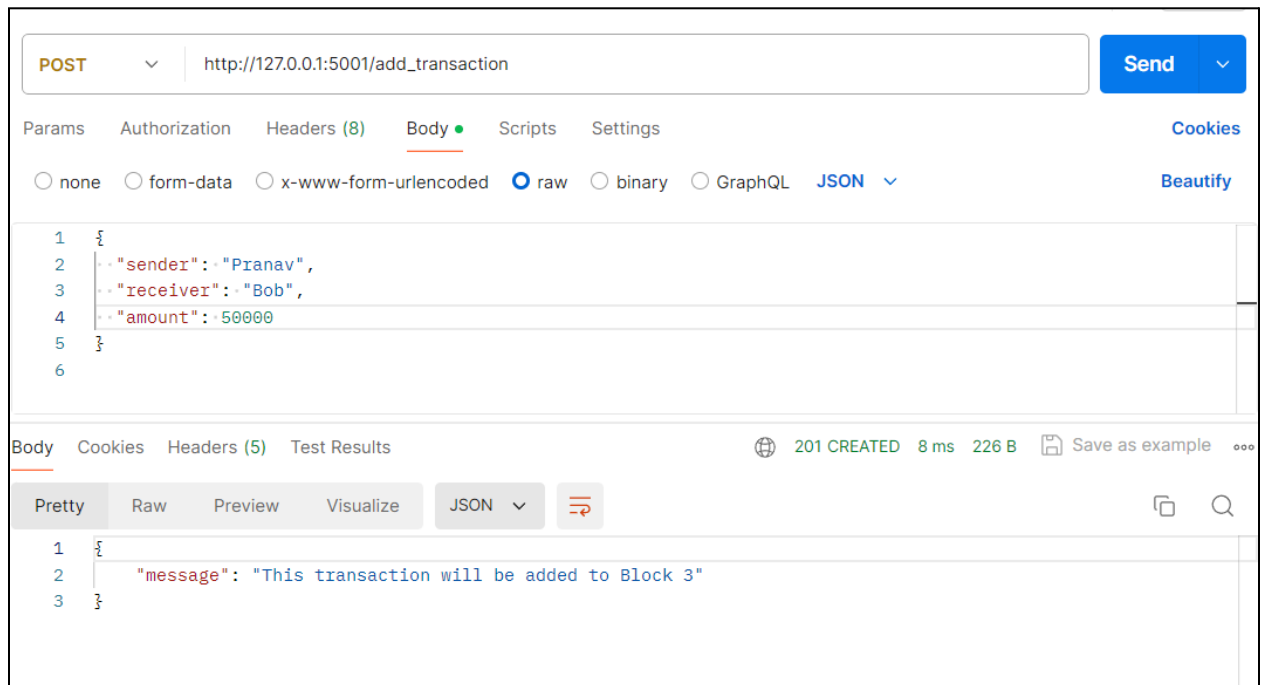
URL

POST `http://127.0.0.1:5001/add_transaction`

Body

```
{  
  "sender": "Pranav",  
  "receiver": "Bob",  
  "amount": 50  
}
```

Transaction goes into **mempool**, NOT block yet.



3. Mine Block (GET)

GET `http://127.0.0.1:5001/mine_block`

GET http://127.0.0.1:5001/mine_block Send

Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description	Bulk Edit
-----	-------	-------------	-----------

Body Cookies Headers (5) Test Results 200 OK 141 ms 516 B Save as example

Pretty Raw Preview Visualize JSON

```
1 {
2   "index": 3,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "0b7845288777c3c2bac0c29dd458999b4f84ac501604d6cffa99dddec805f705",
5   "proof": 45293,
6   "timestamp": "2026-02-03 03:28:51.323676",
7   "transactions": [
8     {
9       "amount": 50000,
10      "receiver": "Bob",
11      "sender": "Pranav"
12    },
13    {
14      "amount": 1,
15      "receiver": "Richard",
16      "sender": "699a990230044a66be45ae642f1b26db"
17    }
18  ]
19 }
```

4. Get Blockchain (GET)

GET http://127.0.0.1:5001/get_chain

You'll see:

- Transactions inside blocks
- `transactions: [ ]` for new pending list


```
GET http://127.0.0.1:5001/get_chain

Status: 200 OK Time: 7 ms Size: 830 B Save as example

Pretty Raw Preview Visualize JSON

[{"index": 1, "previous_hash": "39b87dbcccc1d15b97adf96631518a5181908b47eb555930ee08dc9eb12aa502", "proof": 533, "timestamp": "2026-02-03 03:16:59.331634", "transactions": []}, {"index": 2, "previous_hash": "0b7845288777c3c2bac0c29dd458999b4f84ac501604d6cffa99dddec805f705", "proof": 45293, "timestamp": "2026-02-03 03:21:02.753581", "transactions": [{"amount": 1, "receiver": "Richard", "sender": "699a990230044a66be45ae642f1b26db"}]}, {"index": 3, "previous_hash": "0b7845288777c3c2bac0c29dd458999b4f84ac501604d6cffa99dddec805f705", "proof": 45293, "timestamp": "2026-02-03 03:28:51.323676", "transactions": [{"amount": 50000, "receiver": "Bob", "sender": "Pranav"}]}
```

5. Replace Chain (Consensus)

GET http://127.0.0.1:5002/replace_chain

Shorter chains get replaced by the longest valid one.

```
1 {
2   "message": "The nodes had different chains so the chain was replaced by the longest one.",
3   "new_chain": [
4     {
5       "index": 1,
6       "previous_hash": "0",
7       "proof": 1,
8       "timestamp": "2026-02-03 03:16:59.331634",
9       "transactions": []
10    },
11    {
12      "index": 2,
13      "previous_hash": "39b87dbcccc1d15b97adf96631518a5181908b47eb555930ee08dc9eb12aa502",
14      "proof": 533,
15      "timestamp": "2026-02-03 03:21:02.753581",
16      "transactions": [
17        {
18          "amount": 1,
19          "receiver": "Richard",
20          "sender": "699a990230044a66be45ae642f1b26db"
21        }
22      ]
23    },
24    {
25      "index": 3,
26      "previous_hash": "0b7845288777c3c2bac0c29dd458999b4f84ac501604d6cffa99dddec805f705",
27      "proof": 45293,
```

CONCLUSION:

In this experiment, a simple cryptocurrency was successfully created using Python by implementing core blockchain concepts such as block creation, hashing, Proof-of-Work mining, transactions, and decentralization. The blockchain system was developed using Flask, allowing multiple nodes to communicate and maintain their own copies of the blockchain.

Mining was performed by solving the Proof-of-Work algorithm, through which new blocks were added to the blockchain and miners were rewarded with cryptocurrency. The experiment also demonstrated transaction handling, where transactions were verified and included in mined blocks. Overall, this experiment provided a practical understanding of how blockchain technology works, including mining, decentralization, and consensus, and highlighted how cryptocurrencies operate in real-world distributed systems.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT 4

AIM: Hands on Solidity Programming Assignments for creating Smart Contracts

THEORY:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- `uint` / `int`: unsigned and signed integers of different sizes (e.g., `uint256`, `int128`).
- `bool`: represents logical values (true or false).
- `address`: holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- `bytes` / `string`: store binary data or textual data.

Variables in Solidity can be state variables (stored on the blockchain permanently), local variables (temporary, created during function execution), or global variables (special predefined variables such as `msg.sender`, `msg.value`, and `block.timestamp`).

Functions allow execution of contract logic. Special types of functions include:

- `pure`: cannot read or modify blockchain state; they work only with inputs and internal computations.
- `view`: can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- Function Visibility defines who can access a function:
 - o `public`: available both inside and outside the contract.
 - o `private`: only accessible within the same contract.
 - o `internal`: accessible within the contract and its child contracts.
 - o `external`: can be called only by external accounts or other contract
- Modifiers are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (`onlyOwner`).

- Constructors are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays:** Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings:** Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs:** Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums:** Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

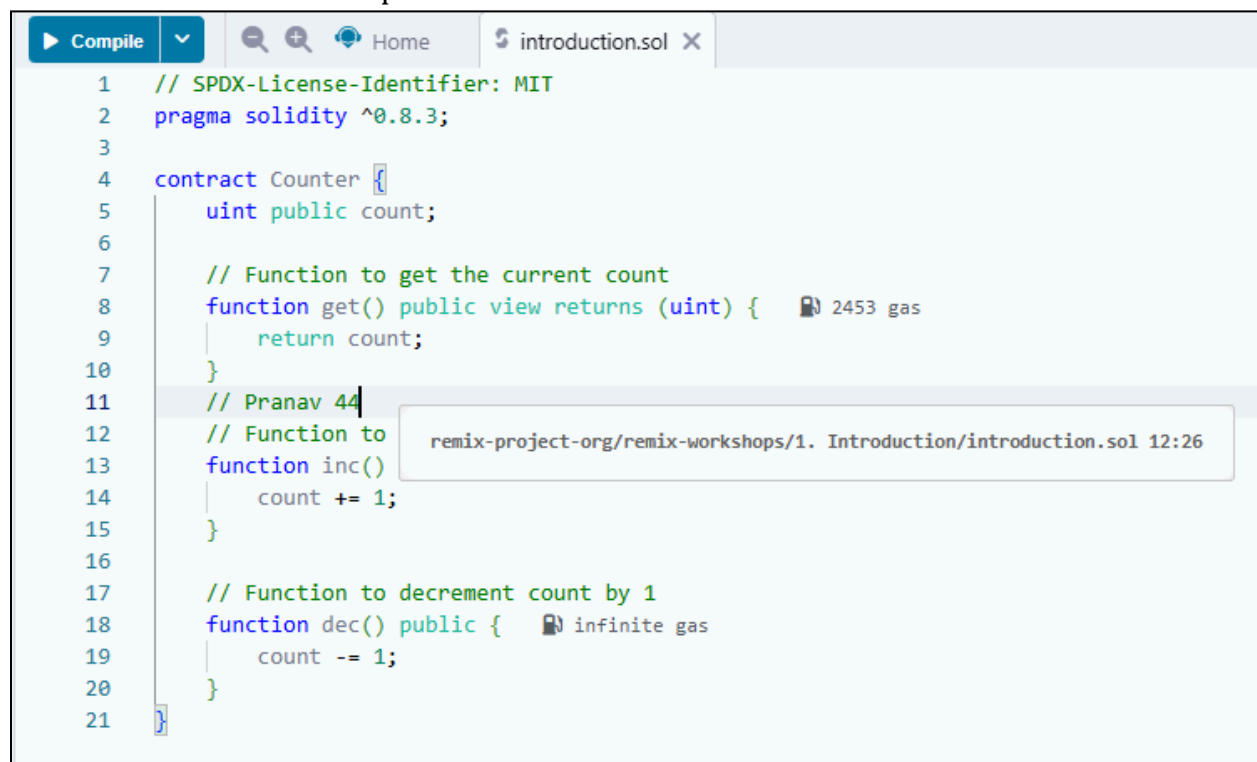
7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

OUTPUT:

Tutorial 1:

- Tutorial no. 1 – Compile the code



```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) { 2453 gas
9         return count;
10    }
11    // Pranav 44
12    // Function to
13    function inc()
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { infinite gas
19        count -= 1;
20    }
21 }
```

- DEPLOY & RUN
TRANSACTIONS

At Address Load contract from Address

Transactions recorded 1 i >

Deployed Contracts 1 🗑

▼ COUNTER AT 0X7EF...8CB47 (lv 📄 🔗 ✕)

Balance: 0 ETH

dec

inc

count

get

✓ Compiled ▼ 🔍 🏠 Home 🔄 introduction.sol ✕

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) {
9         return count;
10    }
11    // Pranav 44
12    // Function to increment count by 1
13    function inc() public { ⛽ infinite gas
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public { ⛽ infinite gas
19        count -= 1;
20    }
21 }
```

Decrement

The image displays three sequential screenshots of the Etherscan interface, specifically the 'Deployed Contracts' section for a contract named 'COUNTER AT 0X7EF...8CB47'. Each screenshot shows the contract's balance as 0 ETH and a set of buttons: 'dec', 'inc', 'count', and 'get'. The first screenshot shows the 'get' button being clicked, with a tooltip 'get - call' and a balance of 0 ETH. The second screenshot shows the 'get' button being clicked, with a tooltip 'get - call' and a balance of 0 ETH. The third screenshot shows the 'get' button being clicked, with a tooltip 'get - call' and a balance of 0 ETH.

Tutorial 2:

LEARNETH

[Tutorials list](#)

[Syllabus](#)

<

2. Basic Syntax

2 / 19

>

following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer

Show answer

Next

Well done! No errors.

Compile

introduction.sol

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than
3 pragma solidity ^0.8.3;
4 // pranav 44
5 contract MyContract {
6     string public name = "Alice";
7 }
```

Explain contract

0 ☐ Listen on all tra

Tutorial 3:

LEARNETH

[Tutorials list](#)

[Syllabus](#)

<

3. Primitive Data Types

3 / 19

>

and **Structs**.

[Watch a video tutorial on Primitive Data Types.](#)

★ Assignment

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer

Show answer

Next

Well done! No errors.

Compile

introduction.sol

basicSyntax.sol

primitiveDataTy

```
12
13
14
15 uint8 public u8 = 1;
16 uint public u256 = 456;
17 uint public u = 123; // uint is an alias for uint256
18 uint public newU = 0;
19 // Pranav 44
20
21 /*
22 Negative numbers are allowed for int types.
23 Like uint, different ranges are available from int8 to int256
24 */
25 int8 public i8 = -1;
26 int public i256 = 456;
27 int public i = -123; // int is same as int256
28 int public neg=-29;
29
30 address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c;
31 address public newAddr = 0x0000000000000000000000000000000000000000;
32
```

Explain contract

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 4:

LEARNETH

Tutorials list

Syllabus

4. Variables

4 / 19

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compile

variables.sol 3

variab

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8     uint public blockNumber;
9
10    function doSomething() public { 22334 gas
11        // Local variables are not saved to the blockchain.
12
13        uint i = 456;
14        // Pranav 44
15        // Here are some global variables
16        uint timestamp = block.timestamp; // Current block timestamp
17        address sender = msg.sender; // address of the caller
18        blockNumber = block.number;
19    }
20 }
```

Explain contract

0 ☐ Listen on all transactions

data: 0x6d4...ce63c

Tutorial 5:

LEARNETH

Tutorials list

Syllabus

5.1 Functions - Reading and Writing to a State Variable

5 / 19

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

[Watch a video tutorial on Functions.](#)

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

variables.sol 3

variables_answer.sol 3

res

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7     bool public b=true;
8
9     function get_b() public view returns (bool) { 2539 gas
10         return b;
11     }
12     // Pranav 44
13     // You need to send a transaction to write to a state variable.
14     function set(uint _num) public { 22536 gas
15         num = _num;
16     }
17
18     // You can read from a state variable without sending a transaction.
19     function get() public view returns (uint) { 2475 gas
20         return num;
21     }
22 }
```

Explain contract

0 ☐ Listen on all transactions

data: 0x6d4...ce63c

Tutorial 6:

LEARNETH

Tutorials list

Syllabus

5.2 Functions - View and Pure

6 / 19

you can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

[Watch a video tutorial on View and Pure Functions.](#)

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile

riables.sol 3

variables_answer.sol 3

readAndWrite.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) {
9         return x + y;
10    }
11    // Pranav 44
12    function addToX2(uint y) public {
13        x=x+y;
14    }
15
16    // Promise not to modify or read from the state.
17    function add(uint i, uint j) public pure returns (uint) {
18        return i + j;
19    }
20 }
21
```

Explain contract

0 ☐ Listen on all transactions

Filter with transaction hash

data: 0xd4...ce63c

Tutorial 7:

LEARNETH

Tutorials list

Syllabus

5.3 Functions - Modifiers and Constructors

7 / 19

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

[Watch a video tutorial on Function Modifiers.](#)

★ Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

Compile

readAndWrite.sol

viewAndPure.sol

modifiersAndConstruct

```
43     locked = true;
44     _;
45     locked = false;
46 }
47
48 function decrement(uint i) public noReentrancy {
49     x -= i;
50
51     if (i > 1) {
52         decrement(i - 1);
53     }
54 }
55 // Pranav 44
56 function increaseX(uint i) public noReentrancy {
57     x += i;
58
59     if (i < 1) {
60         increaseX(i + 1);
61     }
62 }
63
```

Explain contract

0 ☐ Listen on all transactions

Filter with transaction hash or ad

data: 0xd4...ce63c

Tutorial 8:

LEARNETH

Tutorials list

Syllabus

5.4 Functions - Inputs and Outputs

8 / 19

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

[Watch a video tutorial on Function Outputs.](#)

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile

Constructors.sol

inputsAndOutputs.sol

```
11  bool,
12  uint
13  )
14  {
15      return (1, true, 2);
16  }
17  function returnTwo() 472 gas
18  public
19  pure
20  returns (
21      int i,
22      bool b
23  )
24  {
25      i = -2;
26      b = true;
27  }
28  // Pranav 44
29  // Return values can be named.
30  function named() infinite gas
31  public
```

Explain contract

0 ☐ Listen on all transactions

data: 0x6d4...ce63c

Tutorial 9:

LEARNETH

Tutorials list

Syllabus

6. Visibility

9 / 19

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

[Watch a video tutorial on Visibility.](#)

★ Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile

IConstructors.sol

inputsAndOutputs.sol

visibility.sol

visibility_answer.s

```
52  // string external externalVar = "my external variable";
53  }
54
55  contract Child is Base {
56      // Inherited contracts do not have access to private functions
57      // and state variables.
58      // function testPrivateFunc() public pure returns (string memory) {
59      //     return privateFunc();
60      // }
61
62      // Internal function call be called inside child contracts.
63      function testInternalFunc() public pure override returns (string memory) {
64          return internalFunc();
65      }
66      // Pranav 44
67      function testInternalVar() public view returns (string memory, string memory) {
68          return (internalVar, publicVar);
69      }
70  }
```

Explain contract

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 10:

7.1 Control Flow - If/Else
10 / 19

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.
[Watch a video tutorial on the If/Else statement.](#)

★ Assignment

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check AnswerShow answer

Next

Well done! No errors.

```
11     return 2;
12 }
13 }
14
15 function ternary(uint _x) public pure returns (uint) {
16     // if (_x < 10) {
17     //     return 1;
18     // }
19     // return 2;
20
21     // shorthand way to write if / else statement
22     return _x < 10 ? 1 : 2;
23 }
24
25 // Pranav 44
26 function evenCheck(uint y) public pure returns (bool) {
27     return y%2 == 0 ? true : false;
28 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 11:

Tutorials list

Syllabus

7.2 Control Flow - Loops
11 / 19

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.
[Watch a video tutorial on Loop statements.](#)

★ Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.

2. At the end of the for loop, increment the count variable by 1.

3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check AnswerShow answer

Next

Well done! No errors.

```
4 contract Loop {
5     uint public count;
6     function loop() public {
7         // for loop
8         for (uint i = 0; i < 10; i++) {
9             if (i == 5) {
10                // Skip to next iteration with continue
11                continue;
12            }
13            if (i == 5) {
14                // Exit loop with break
15                break;
16            }
17            count++;
18        }
19    }
20    // Pranav 44
21    // while loop
22    uint j;
23    while (j < 10) {
24        j++;
25    }
26 }
```

Explain contract

AI copilot

0 ☐ Listen on all transactions

Filter with transact

data: 0x6d4...ce63c

Tutorial 12:

8.1 Data Structures - Arrays
12 / 19

important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

[Watch a video tutorial on Arrays.](#)

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1,
2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

```
14 }
15
16 // Solidity can return the entire array.
17 // But this function should be avoided for
18 // arrays that can grow indefinitely in length.
19 function getArr() public view returns (uint[3] memory) { infinite gas
20     return arr3;
21 }
22 // Pranav 44
23
24 function push(uint i) public { 46820 gas
25     // Append to array
26     // This will increase the array length by 1.
27     arr.push(i);
28 }
29
30 function pop() public { 29462 gas
31     // Remove last element from array
32 }
```

AI cop

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 13:

8.2 Data Structures - Mappings
13 / 19

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

[Watch a video tutorial on Mappings.](#)

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

```
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7     // Pranav 44
8     function get(address _addr) public view returns (uint) { 2872 gas
9         // Mapping always returns a value.
10        // If the value was never set, it will return the default value.
11        return balances[_addr];
12    }
13
14    function set(address _addr) public { 25256 gas
15        // Update the value at this address
16        balances[_addr] = _addr.balance;
17    }
18
19    function remove(address _addr) public { 5566 gas
20        // Reset the value to the default value.
21        delete balances[_addr];
22    }
```

AI cop

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 14:

8.3 Data Structures - Structs
14 / 19

members and access a member. We initialize an empty struct member and then access its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

```
34 }
35
36 // update text
37 function update(uint _index, string memory _text) public { infinite gas
38     Todo storage todo = todos[_index];
39     todo.text = _text;
40 }
41
42 // update completed
43 function toggleCompleted(uint _index) public { 28995 gas
44     Todo storage todo = todos[_index];
45     todo.completed = !todo.completed;
46 }
47 // Pranav 44
48 function remove(uint _index) public { infinite gas
49     delete todos[_index];
50 }
51 }
```

AI cop

0 ☐ Listen on all transactions

Filter with transaction hash or ad...

data: 0x6d4...ce63c

Tutorial 15:

8.4 Data Structures - Enums
15 / 19

Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

```
22 Status public Status;
23 Size public sizes;
24
25 function get() public view returns (Status) {
26     return status;
27 }
28
29 function getSize() public view returns (Size) {
30     return sizes;
31 }
32 // Pranav 44
33
34 // Update status by passing uint into input
35 function set(Status _status) public {
36     status = _status;
37 }
38
39 // You can update to a specific enum like this
40 function cancel() public {
```

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or address

data: 0x6d4...ce63c

Tutorial 16:

9. Data Locations
16 / 19

★ Assignment

1. Change the value of the `myStruct` member `foo`, inside the function `f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

```
24 myMemStruct3.foo = 3;
25 return (myStruct, myMemStruct2, myMemStruct3);
26 }
27
28 function f(
29     uint[] storage _arr,
30     mapping(uint => address) storage _map,
31     MyStruct storage _myStruct
32 ) internal {
33     // do something with storage variables
34 }
35 // Pranav 44
36
37 // You can return memory variables
38 function g(uint[] memory _arr) public returns (uint[] memory) {
39     // do something with memory array
40     _arr[0] = 1;
41 }
42
```

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or address

data: 0x6d4...ce63c

Tutorial 17:

10.1 Transactions - Ether and Wei
17 / 19

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

[Watch a video tutorial on Ether and Wei.](#)

★ Assignment

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

```
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     // 1 wei is equal to 1
7     bool public isOneWei = 1 wei == 1;
8
9     uint public oneEther = 1 ether;
10    // 1 ether is equal to 10^18 wei
11    bool public isOneEther = 1 ether == 1e18;
12    // Pranav 44
13    uint public oneGwei = 1 gwei;
14    // 1 ether is equal to 10^9 wei
15    bool public isOneGwei = 1 gwei == 1e9;
16 }
```

AI copilot

0 ☐ Listen on all transactions

Filter with transaction hash or address

data: 0x6d4...ce63c

Tutorial 18 :

run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

```
3
4 contract Gas {
5   uint public i = 0;
6   uint public cost = 170367;
7   // Pranav 44
8   // Using up all of the gas that you send causes your transaction to fail.
9   // State changes are undone.
10  // Gas spent are not refunded.
11  function forever() public { infinite gas
12    // Here we run a loop until all of the gas are spent
13    // and the transaction fails
14    while (true) {
15      i += 1;
16    }
17  }
18 }
```

Explain contract AI copilot

0 ☐ Listen on all transactions

data: 0x6d4...ce63c

Tutorial 19:

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer **Show answer**

Next

Well done! No errors.

```
51 }
52
53 contract Charity {
54   address public owner;
55
56   constructor() { 165452 gas 141000 gas
57     owner = msg.sender;
58   }
59
60   function donate() public payable {} 141 gas
61   // Pranav 44
62   function withdraw() public { infinite gas
63     uint amount = address(this).balance;
64
65     (bool sent, bytes memory data) = owner.call{value: amount}("");
66     require(sent, "Failed to send Ether");
67   }
68 }
```

Explain contract AI copilot

0 ☐ Listen on all transactions

data: 0x6d4...ce63c

CONCLUSION:

Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 5

AIM: Deploying a Voting/Ballot Smart Contract

Tasks:

1. Open [Remix IDE](#)
2. Under **Workspaces**, open **contracts** folder
3. Open **Ballot.sol**, contract.
4. Understand **Ballot.sol** contract.
5. Deploy the contract by changing the Proposal name from **bytes32** → **string**

THEORY:

1. Relevance of require Statements in Solidity Programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt the blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(`require(voters[msg.sender].weight > 0, "Has no right to vote");`).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They also allow developers to attach error messages, making debugging and contract interaction easier for users.

2. Keywords: mapping, storage, and memory

mapping:

- A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is `mapping(keyType => valueType)`. For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful for contracts like Ballot, where you need to associate voters with their data (whether they voted, which proposal they chose, etc.). Unlike arrays, mappings do not have a length property and cannot be iterated over directly, making them gas efficient for lookups but limited for enumeration.

storage:

- In Solidity, storage refers to the permanent memory of the contract, stored on the Ethereum blockchain. Variables declared at the contract level are stored in storage by default. Data stored in storage is persistent across transactions, which means once written, it remains available unless explicitly modified. However, because writing to blockchain storage consumes gas, it is more expensive. For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle.

memory:

- In contrast, memory is temporary storage, used only for the lifetime of a function call. When the function execution ends, the data stored in memory is discarded. Memory is mainly used for temporary variables, function arguments, or computations that don't need to be permanently stored on the blockchain. It is cheaper than storage in terms of gas cost. For instance, when handling proposal names or temporary string manipulations, memory is often used.

Thus, a smart contract developer must balance between storage and memory to ensure efficiency and cost-effectiveness.

- **bytes32** is a fixed-size type, meaning it always stores exactly 32 bytes of data. This makes storage simple, comparison operations faster, and gas costs lower. However, it limits proposal names to 32 characters, which is not very flexible for user-friendly names.
- **string** is a dynamically sized type, meaning it can store text of variable length. While it is easier for developers and users (since names can be written normally), it requires more complex handling inside the Ethereum Virtual Machine (EVM). This increases gas usage and may slow down comparisons or manipulations.

To make the system more user-friendly, modern implementations of the Ballot contract often convert from bytes32 to string. Tools like the Web3 Type Converter help developers easily switch between these two types for deployment and testing.

CODE:

```
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */
contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
        string name;    // CHANGED from bytes32 → string
        uint voteCount;
    }

    address public chairperson;
    mapping(address => Voter) public voters;
    Proposal[] public proposals;

    /**
     * @dev Create a new ballot
     * @param proposalNames names of proposals
     */
    constructor(string[] memory proposalNames) {
        chairperson = msg.sender;
        voters[chairperson].weight = 1;
    }
}
```

```

for (uint i = 0; i < proposalNames.length; i++) {
    proposals.push(
        Proposal({
            name: proposalNames[i],
            voteCount: 0
        })
    );
}
}

```

```

function giveRightToVote(address voter) external {
    require(msg.sender == chairperson, "Only chairperson can give right to vote");
    require(!voters[voter].voted, "The voter already voted");
    require(voters[voter].weight == 0, "Voter already has right");

    voters[voter].weight = 1;
}

```

```

function delegate(address to) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");
    require(to != msg.sender, "Self-delegation not allowed");

    while (voters[to].delegate != address(0)) {
        to = voters[to].delegate;
        require(to != msg.sender, "Delegation loop detected");
    }

    Voter storage delegate_ = voters[to];
    require(delegate_.weight >= 1, "Delegate has no right");

    sender.voted = true;
}

```

```

sender.delegate = to;

if (delegate_.voted) {
    proposals[delegate_.vote].voteCount += sender.weight;
} else {
    delegate_.weight += sender.weight;
}
}

function vote(uint proposal) external {
    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0, "No right to vote");
    require(!sender.voted, "Already voted");

    sender.voted = true;
    sender.vote = proposal;

    proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() external view returns (string memory) {
    return proposals[winningProposal()].name;
}

```

```
}  
}  
}
```

OUTPUT:

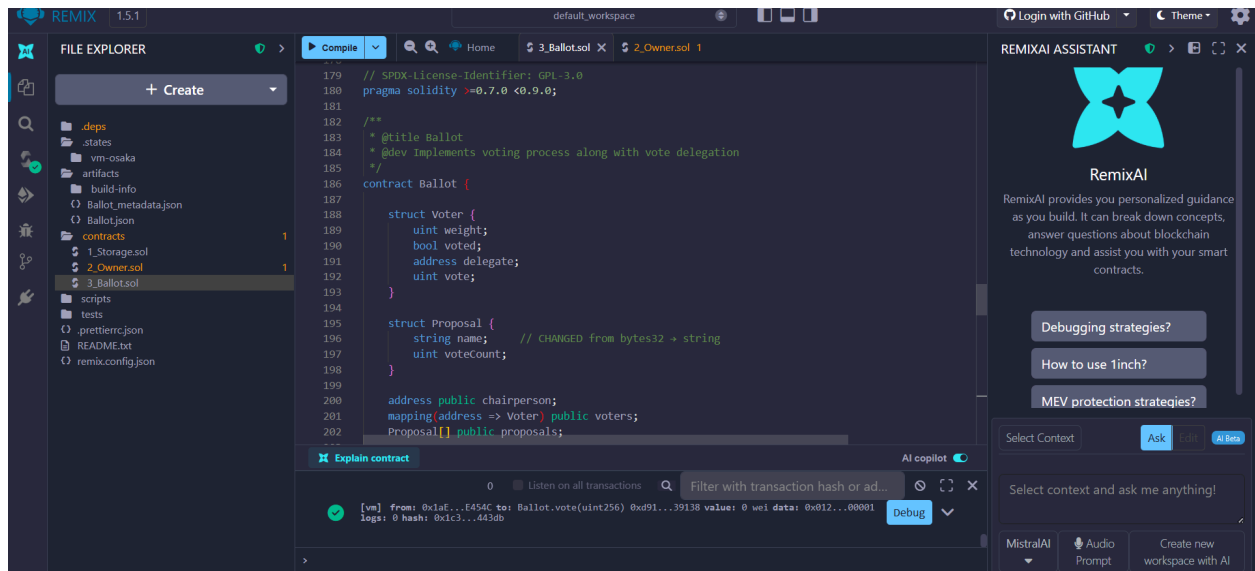
Step 1: Open Remix

Go to 🖱️ <https://remix.ethereum.org>

No install needed.

Step 2: Create the file

- In File Explorer
Ballot.sol



Step 3: Compile the contract

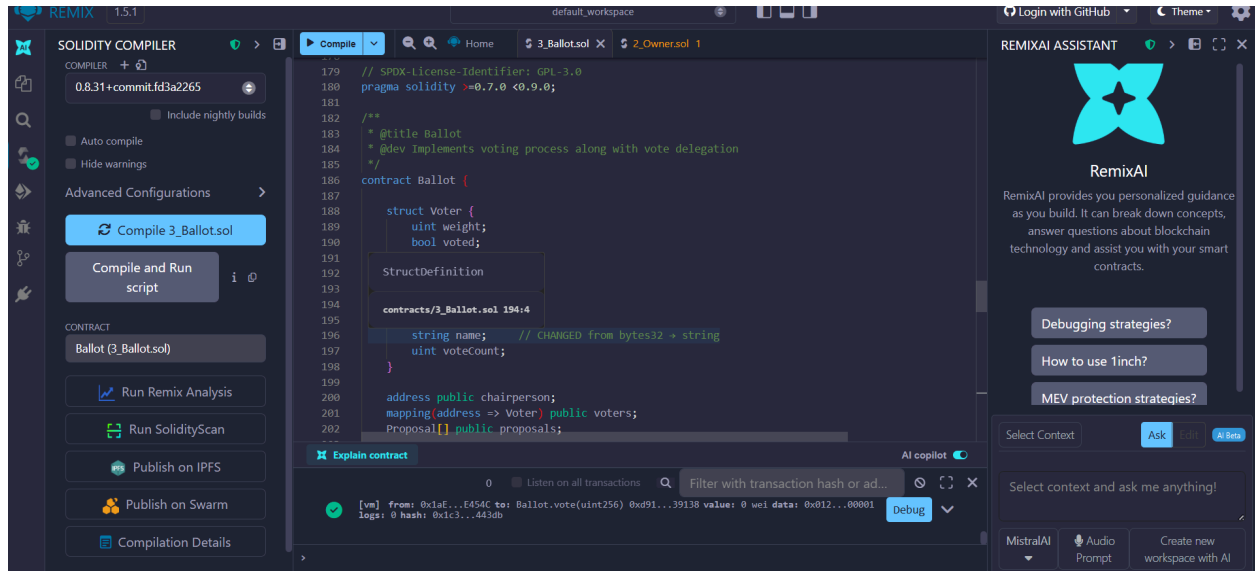
1. Open Solidity Compiler (left sidebar)
2. Click Compile Ballot.sol
3. Make sure green tick appears (no errors)

Step 4: Deploy the contract

1. Open Deploy & Run Transactions

2. Set:

- Environment → **Remix VM**
- Account → keep default
- Contract → **Ballot**



Step 5: Enter constructor input (IMPORTANT)

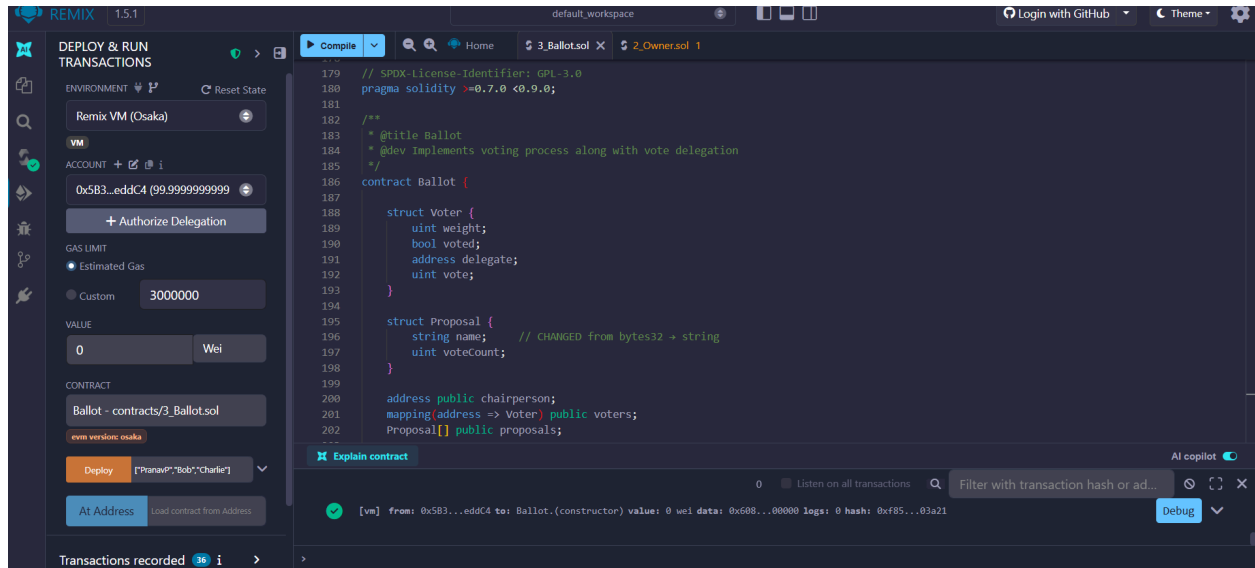
Your constructor is:

constructor(string[] memory proposalNames)

So input must be a string array 🙌

Example:

["PranavP", "Bob", "Charlie"]



Step 6: Click Deploy

- Click Deploy
- Contract appears under Deployed Contracts 🎉
- The deployer is automatically the chairperson

giveRightToVote (address voter)

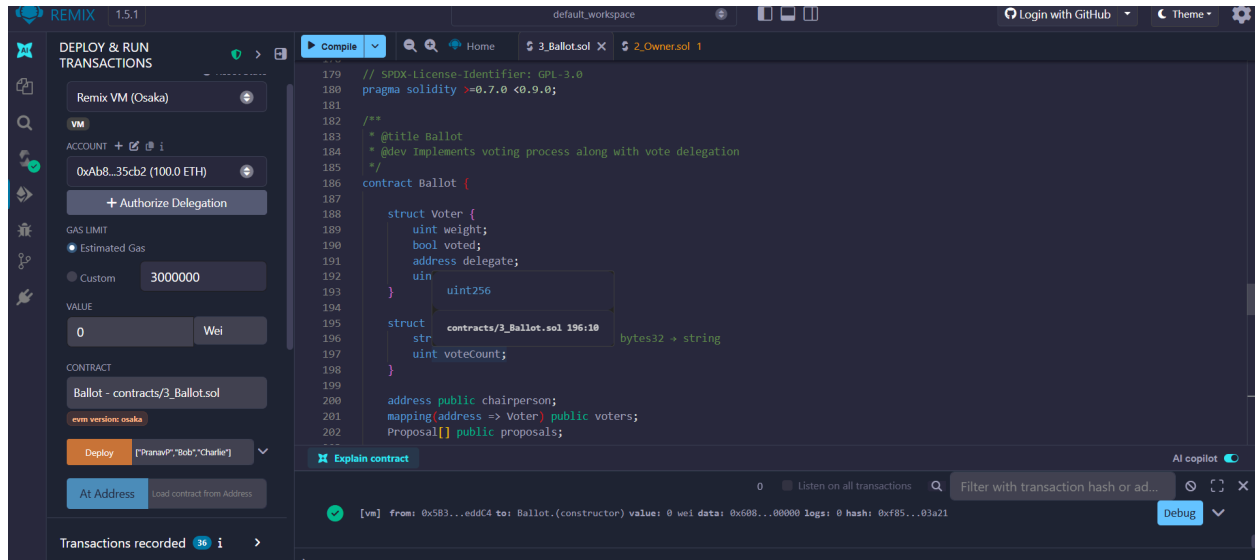
What to add:

Paste a Remix account address (NOT the chairperson).

Important:

- You must be using Account 1 (chairperson) when clicking this
- Switch account at the top if needed

Click Transact



Then vote

2 vote (uint256 proposal)

What to add:

A number, based on proposal index:

0 → PranavP

1 → Bob

2 → Charlie

Example:

0

Important:

- Switch to the voter account (the one you gave rights to)
- Then click Transact

REMIX 1.5.1 default_workspace Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

Deployed Contracts 1

BALLOT AT 0xD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote 0xA6B483F649C6d1Ed9b

vote uint256 proposal

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions

CALLDATA

Transact

```
179 // SPDX-License-Identifier: GPL-3.0
180 pragma solidity >=0.7.0 <0.9.0;
181
182 /**
183  * @title Ballot
184  * @dev Implements voting process along with vote delegation
185  */
186 contract Ballot {
187
188     struct Voter {
189         uint weight;
190         bool voted;
191         address delegate;
192         uint vote;
193     }
194
195     struct Proposal {
196         string name; // CHANGED from bytes32 + string
197         uint voteCount;
198     }
199
200     address public chairperson;
201     mapping(address => Voter) public voters;
202     Proposal[] public proposals;
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0x5B3...eddC4 to: Ballot.(constructor) value: 0 wei data: 0x608...00000 logs: 0 hash: 0xf85...03a21

Debug

REMIX 1.5.1 default_workspace Login with GitHub Theme

DEPLOY & RUN TRANSACTIONS

Transactions recorded 30

Deployed Contracts 1

BALLOT AT 0xD91...39138 (ME)

Balance: 0 ETH

delegate address to

giveRightToVote 0xA6B483F649C6d1Ed9b

vote 0

chairperson

proposals uint256

voters address

winnerName

winningProposal

Low level interactions

CALLDATA

Transact

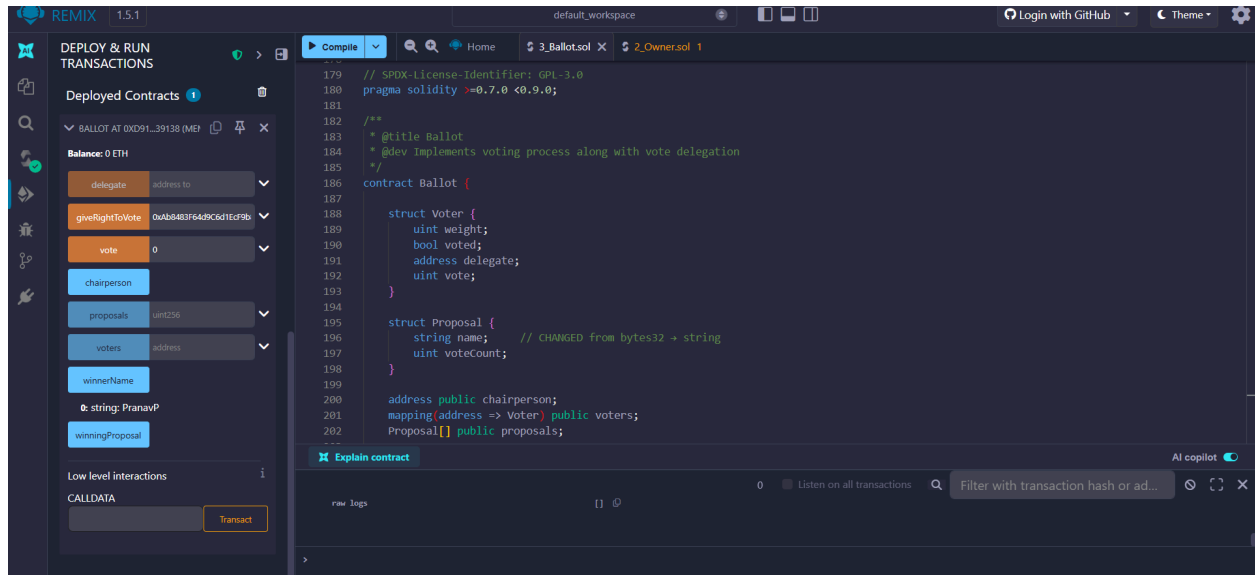
```
179 // SPDX-License-Identifier: GPL-3.0
180 pragma solidity >=0.7.0 <0.9.0;
181
182 /**
183  * @title Ballot
184  * @dev Implements voting process along with vote delegation
185  */
186 contract Ballot {
187
188     struct Voter {
189         uint weight;
190         bool voted;
191         address delegate;
192         uint vote;
193     }
194
195     struct Proposal {
196         string name; // CHANGED from bytes32 + string
197         uint voteCount;
198     }
199
200     address public chairperson;
201     mapping(address => Voter) public voters;
202     Proposal[] public proposals;
```

Explain contract

0 Listen on all transactions Filter with transaction hash or address

[vm] from: 0xA6B...35cb2 to: Ballot.vote(uint256) 0xd91...39138 value: 0 wei data: 0x012...00000 logs: 0 hash: 0xc66...d2510

Debug



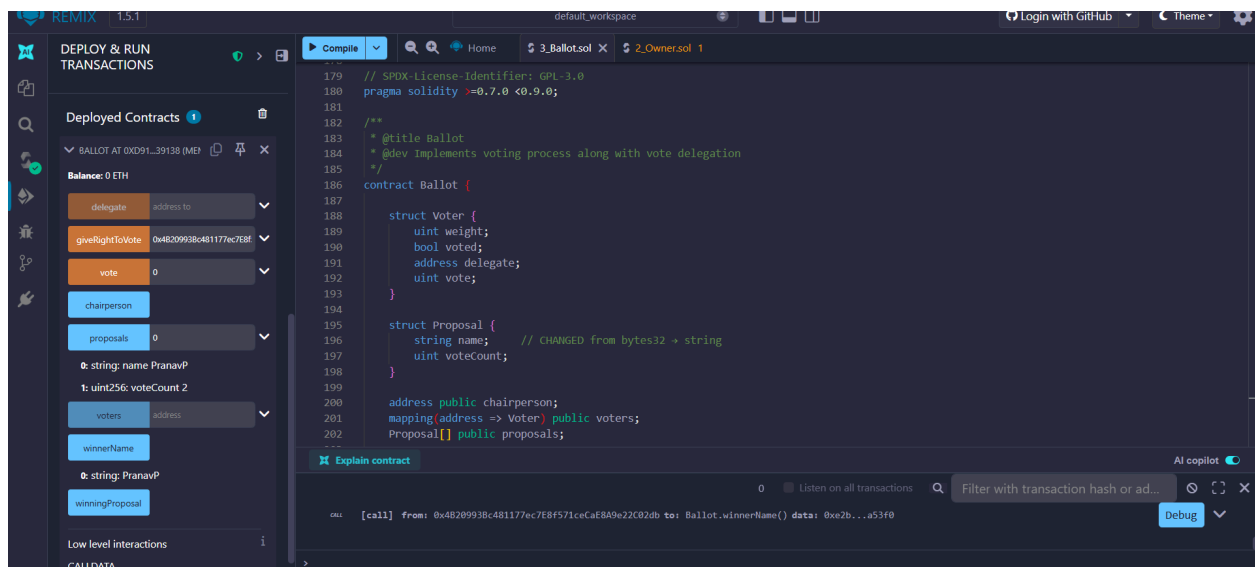
proposals (uint256)

What to add:

Proposal index:

0

Returns proposal details.



Conclusion:

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

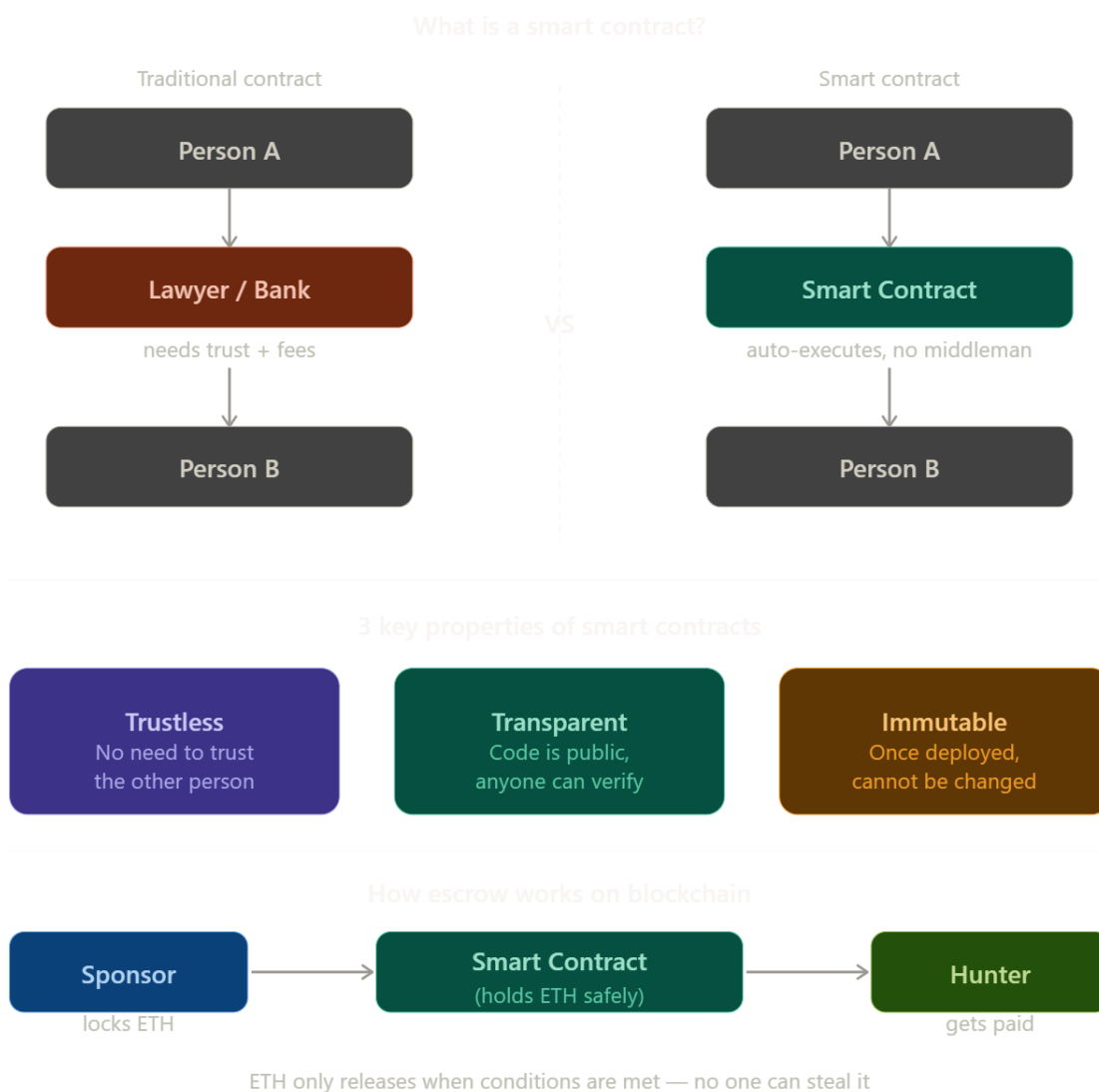
Batch: C

EXPERIMENT 6

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:-

Smart contracts are self-executing programs that live on a blockchain. Think of them like a vending machine — you put in money, the machine checks the rules, and automatically gives you what you paid for. No human in the middle.



Decentralized Bug Bounty Escrow System

What is this project?

This is a blockchain-based platform that connects **companies (sponsors)** who want their software tested, with **security researchers (hunters)** who find vulnerabilities — all without trusting a middleman.

When a sponsor posts a bounty and locks ETH, the money is held securely by a smart contract on Ethereum. The hunter only gets paid when their bug report is approved. If there's a disagreement, a neutral arbiter resolves the dispute. Everything is automated, transparent, and tamper-proof.

The Problem it Solves

Traditional bug bounty platforms like HackerOne or Bugcrowd charge high fees, can delay payments for months, and hunters have to trust the company will actually pay. With this system, the ETH is locked upfront — the hunter knows the money is there before they even start working.

The 3 Smart Contracts

BugBountyToken (BBT) — A reward token given to hunters every time they get paid. Acts like a reputation badge — the more bugs you find, the more BBT you earn.

BugBountyEscrow — The heart of the system. It locks ETH, manages the full bounty lifecycle (create → fund → submit → approve/dispute), and automatically pays the hunter when approved.

BugBountyRegistry — A public leaderboard and directory. Stores bounty listings, severity tags, and tracks each hunter's reputation score on-chain for everyone to see.

Who uses it?

Role	What they do
 Sponsor	Posts a bounty, locks ETH, reviews reports
 Hunter	Finds bugs, submits reports, earns ETH + BBT tokens
 Arbiter	Resolves disputes fairly between sponsor and hunter



Platform Owner

Collects a small fee, manages the system

Tech Stack

Built with **Solidity 0.8.19** on the **Ethereum** blockchain. Deployable on any EVM-compatible network (Ethereum, Polygon, Base, etc.) and fully testable on **Remix IDE** with no real money needed.

Implementation:-

Smart Contracts - BugBountyToken

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.19;

/**

* @title BugBountyToken

* @dev Simple ERC20-style reward token for Bug Bounty hunters.

* Minted by the BugBountyEscrow contract when a bounty is paid out.

* Hunters accumulate BBT as reputation/reward tokens.

*/

contract BugBountyToken {

string public name = "Bug Bounty Token";

string public symbol = "BBT";

uint8 public decimals = 18;

uint256 public totalSupply;

// Token balances

mapping(address => uint256) public balanceOf;

// Allowances for delegated transfers

mapping(address => mapping(address => uint256)) public allowance;

// Only the escrow contract (set at deploy time) can mint tokens

address public minter;

address public owner;

// — Events

event Transfer(address indexed from, address indexed to, uint256 value);

event Approval(address indexed owner_, address indexed spender, uint256 value);

event MinterUpdated(address indexed oldMinter, address indexed newMinter);

// — Modifiers

```
modifier onlyOwner() {
    require(msg.sender == owner, "BBT: not owner");
    _;
}
modifier onlyMinter() {
    require(msg.sender == minter, "BBT: caller is not the minter");
    _;
}
// — Constructor
```

```
constructor() {
    owner = msg.sender;
    minter = msg.sender; // owner acts as minter until escrow is deployed
}
// — Admin
```

```
/**
 * @notice Set the escrow contract address as the authorised minter.
 *      Call this after deploying BugBountyEscrow.
 */
function setMinter(address _minter) external onlyOwner {
    emit MinterUpdated(minter, _minter);
    minter = _minter;
}
// — ERC-20 Core
```

```
function transfer(address to, uint256 amount) external returns (bool) {
    _transfer(msg.sender, to, amount);
    return true;
}
function approve(address spender, uint256 amount) external returns (bool) {
    allowance[msg.sender][spender] = amount;
    emit Approval(msg.sender, spender, amount);
    return true;
}
```

```

}
function transferFrom(address from, address to, uint256 amount) external returns (bool) {
    require(allowance[from][msg.sender] >= amount, "BBT: allowance exceeded");
    allowance[from][msg.sender] -= amount;
    _transfer(from, to, amount);
    return true;
}
// — Minting

```

```

/**
 * @notice Mint BBT reward tokens to a hunter's wallet.
 *      Only callable by the BugBountyEscrow contract (the minter).
 */
function mint(address to, uint256 amount) external onlyMinter {
    totalSupply += amount;
    balanceOf[to] += amount;
    emit Transfer(address(0), to, amount);
}
// — Internal

```

```

function _transfer(address from, address to, uint256 amount) internal {
    require(to != address(0), "BBT: transfer to zero address");
    require(balanceOf[from] >= amount, "BBT: insufficient balance");
    balanceOf[from] -= amount;
    balanceOf[to] += amount;
    emit Transfer(from, to, amount);
}
}

```

Step 1:- Deploy Contract 1: BugBountyToken

Step 2:- Deploy Contract 2: BugBountyEscrow

In the **Contract** dropdown, select BugBountyEscrow

Fill in the constructor arguments:

- `_rewardToken`: paste the BugBountyToken address from Step 3
- `_platformFeeBps`: enter 200 (= 2% platform fee)

Click **Deploy**

Copy the BugBountyEscrow deployed address

Step 3:- Deploy Contract 3: BugBountyRegistry

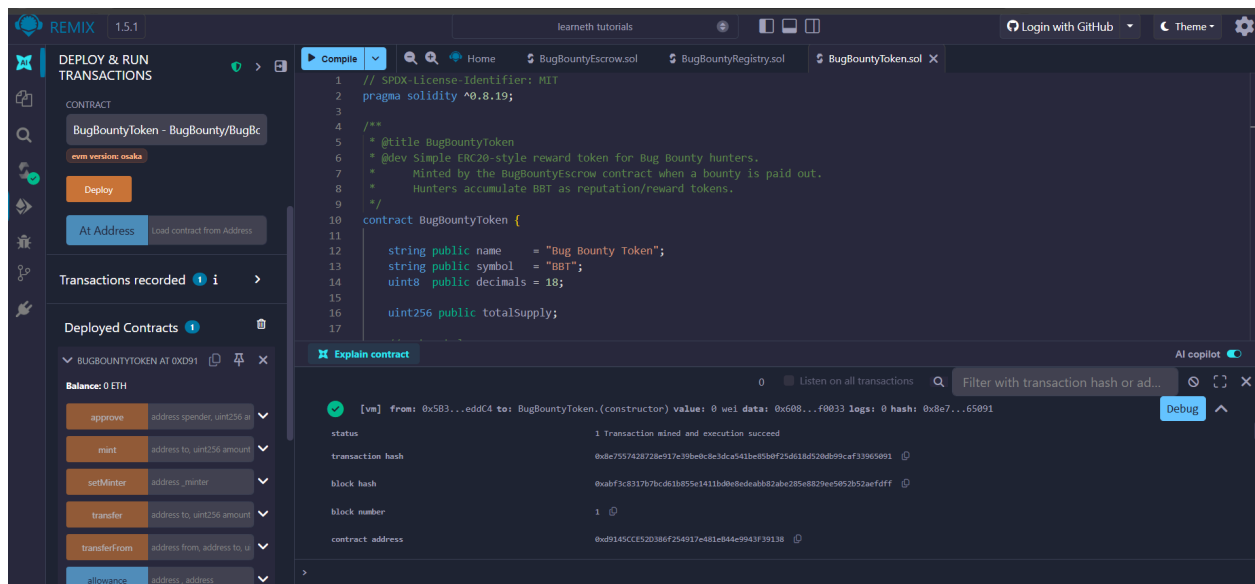
In the **Contract** dropdown, select BugBountyRegistry

Constructor argument:

- `_escrowContract`: paste the BugBountyEscrow address from Step 4

Click **Deploy**

Copy the BugBountyRegistry deployed address



Step 4:- Paste the BugBountyEscrow address to mint BBT tokens then click the transact button

- Expand the deployed BugBountyToken contract in the left panel
- Find the setMinter function
- Paste the BugBountyEscrow address
- Click **transact**

REMIX 1.5.1

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT Reset State

Remix VM (Cancel)

ACCOUNT 0x583...eddC4 (99.999999999)

GAS LIMIT Estimated Gas

Custom 3000000

VALUE 0 Wei

CONTRACT BugBountyRegistry - BugBounty/Bug

Deploy address_escrowContract

At Address Load contract from Address

Transactions recorded 1

```

22 contract BugBountyRegistry {
23
24     // -- Types
25
26     enum Severity { None, Informational, Low, Medium, High, Critical }
27
28     struct HunterProfile {
29         uint256 totalSubmissions;
30         uint256 totalApprovals;
31         uint256 totalEarnedWei;
32         uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
33     }
34
35     struct BountyRecord {
36         uint256 bountyId;
37         address sponsor;
38         Severity severity; // set by sponsor on creation or after approval

```

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x168...784f3 Debug

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18 Debug

Step 5:- Continuing from the step 4

REMIX 1.5.1

DEPLOY & RUN TRANSACTIONS

ENVIRONMENT Reset State

Remix VM (Cancel)

ACCOUNT 0x583...eddC4 (99.999999999)

GAS LIMIT Estimated Gas

Custom 3000000

VALUE 0 Wei

CONTRACT BugBountyRegistry - BugBounty/Bug

Deploy address_escrowContract

At Address Load contract from Address

Transactions recorded 5

Deployed Contracts 4

BUGBOUNTYTOKEN AT 0XD91

Balance: 0 ETH

approve address spender, uint256 a

mint address to, uint256 amount

setMinter 0xd8b9345801e35a11858c

transfer address to, uint256 amount

transferFrom address from, address to, si

```

22 contract BugBountyRegistry {
23
24     // -- Types
25
26     enum Severity { None, Informational, Low, Medium, High, Critical }
27
28     struct HunterProfile {
29         uint256 totalSubmissions;
30         uint256 totalApprovals;
31         uint256 totalEarnedWei;
32         uint256 reputationScore; // simple formula: approvals * 10 + totalEarned / 1e15
33     }
34
35     struct BountyRecord {
36         uint256 bountyId;
37         address sponsor;
38         Severity severity; // set by sponsor on creation or after approval

```

Explain contract

0 Listen on all transactions Filter with transaction hash or ad...

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x168...784f3 Debug

creation of BugBountyRegistry pending...

[vm] from: 0x583...eddC4 to: BugBountyRegistry.(constructor) value: 0 wei data: 0x608...33fa8 logs: 0 hash: 0x82a...6fd18 Debug

transact to BugBountyToken.setMinter pending ...

[vm] from: 0x583...eddC4 to: BugBountyToken.setMinter(address) 0xd91...39138 value: 0 wei data: 0xfca...33fa8 logs: 1 hash: 0xe95...a1cc8 Debug

Step 6:- Call fundBountywith id = 1 and set Value to 1 ETH

Step 7 :- You need 3 fake people (accounts). At the **top of the Deploy panel** in Remix, there's a dropdown that says "**Account**" Click it — you'll see **10 fake accounts**, each has **100 ETH** (free test money!)

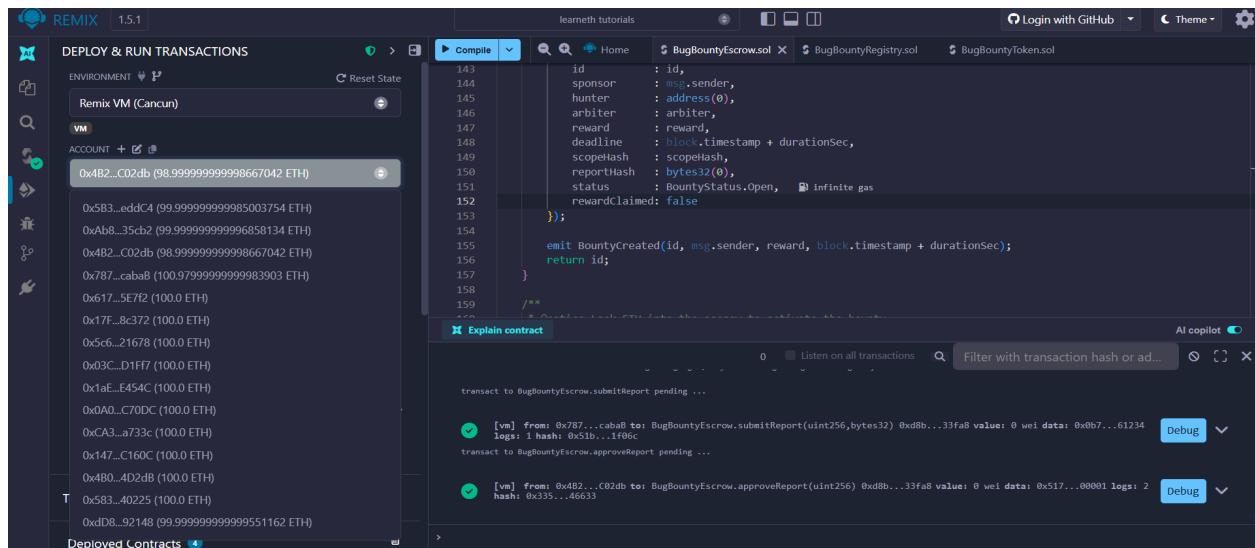
Step 8:- Sponsor creates a bounty

Switch to Account 2 (the Sponsor) from the Account dropdown. Now go to your deployed **BugBountyEscrow** contract and click **createBounty**

Fill in that exactly like this

reward → 10000000000000000000
durationSec → 2592000
arbiter → [paste Account 1 address]
scopeHash → 0xabcd....

Step 9:- Register bounty in the registry, then there are 3 people in the accounts part there are 10 fake accounts, then one owner at the top, then there is sponsor, the sponsor will create bounty, sponsor then locks the bounty, hunter submits their bug report and the sponsor approves and then the hunter gets paid, **then the value of that hunter raises**



Conclusion

The Decentralized Bug Bounty Escrow System demonstrates how blockchain technology can solve real trust problems in the cybersecurity industry. By replacing middlemen with smart contracts, it ensures that hunters always get paid fairly and sponsors only release funds when satisfied all enforced automatically by code, not promises.

With three interconnected contracts handling payments, reputation, and dispute resolution, this system creates a transparent and tamper-proof environment where security researchers can work with confidence. The addition of BBT reward tokens further incentivizes hunters by building their on-chain reputation with every successful submission.

This project proves that decentralized systems are not just a concept they are practical, deployable tools that can reshape how industries handle trust, payments, and accountability. As Web3 continues to grow, platforms like this represent the future of fair and transparent collaboration between companies and the security community.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 7

AIM: Implement the embedding wallet (Metamask) and transaction using Solidity

Theory:

Blockchain is a decentralized and distributed digital ledger that securely records transactions across multiple systems. It ensures transparency, security, and immutability, meaning that once data is recorded, it cannot be altered.

Ethereum is a widely used blockchain platform that enables the execution of smart contracts. Smart contracts are self-executing programs written in Solidity that automatically carry out actions when predefined conditions are met, eliminating the need for intermediaries.

MetaMask is a cryptocurrency wallet and browser extension used to interact with blockchain networks. It securely stores user accounts and private keys, and allows users to send, receive, and sign transactions. It also serves as a bridge between Remix IDE and blockchain networks.

Test networks such as Sepolia and RSK Testnet are used for development and testing. They provide free tokens, enabling developers to deploy and test smart contracts without using real cryptocurrency, thereby reducing financial risk.

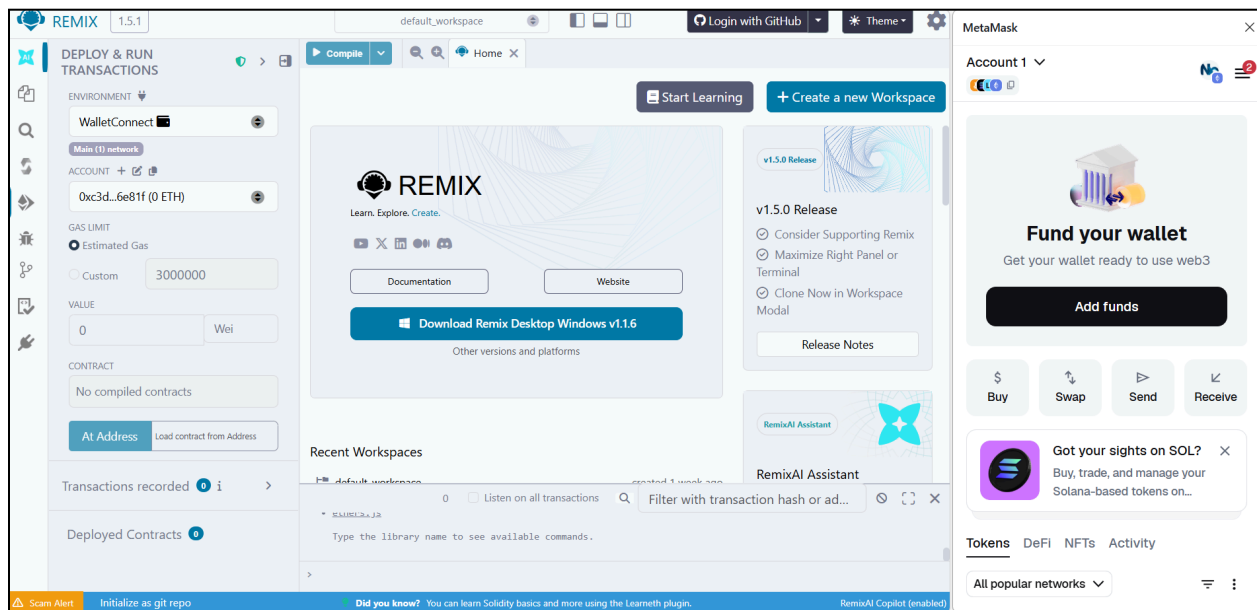
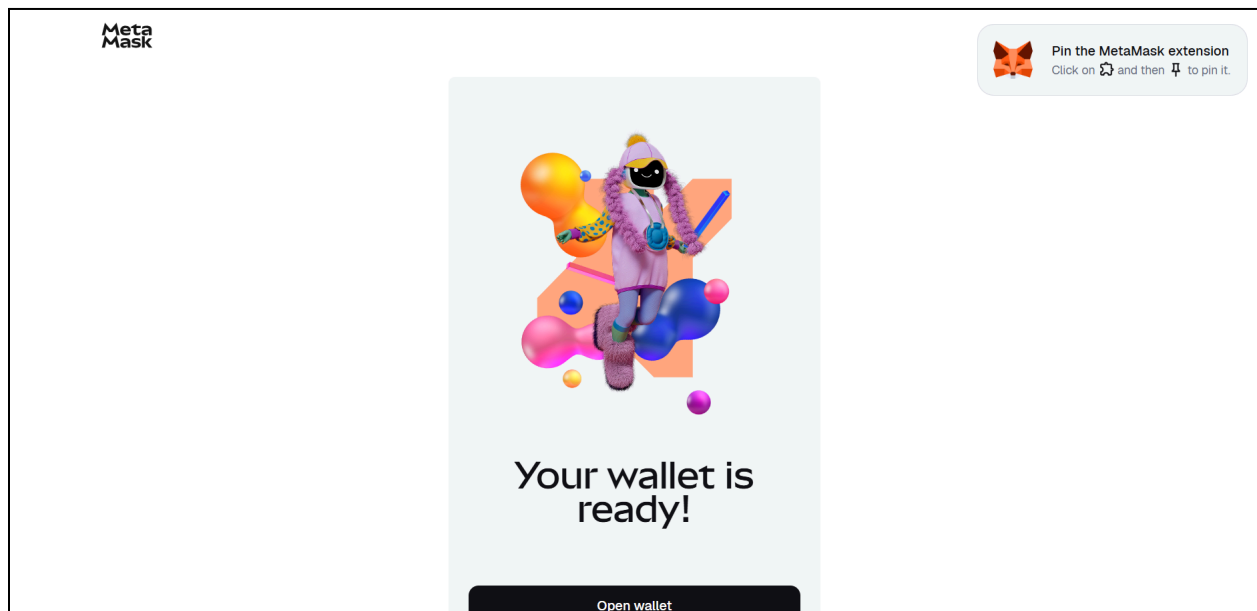
Remix IDE is a web-based development environment used to write, compile, deploy, and test smart contracts. It offers a user-friendly interface and integrates directly with MetaMask for deploying contracts on blockchain networks.

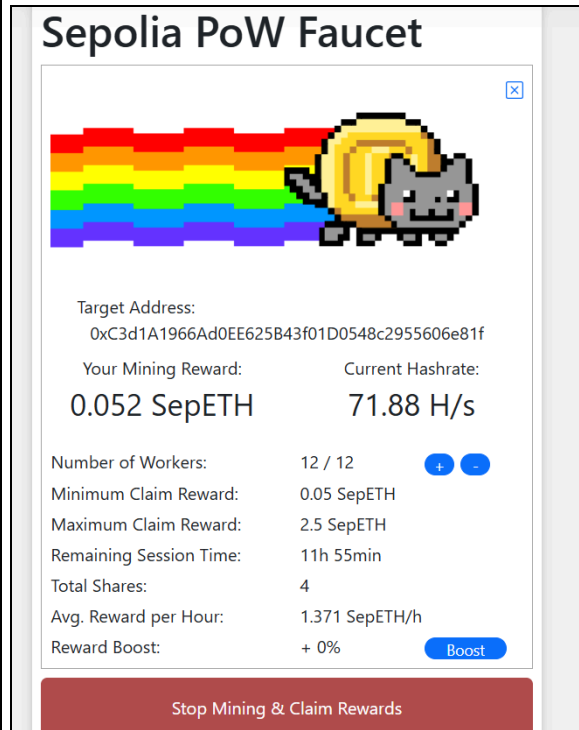
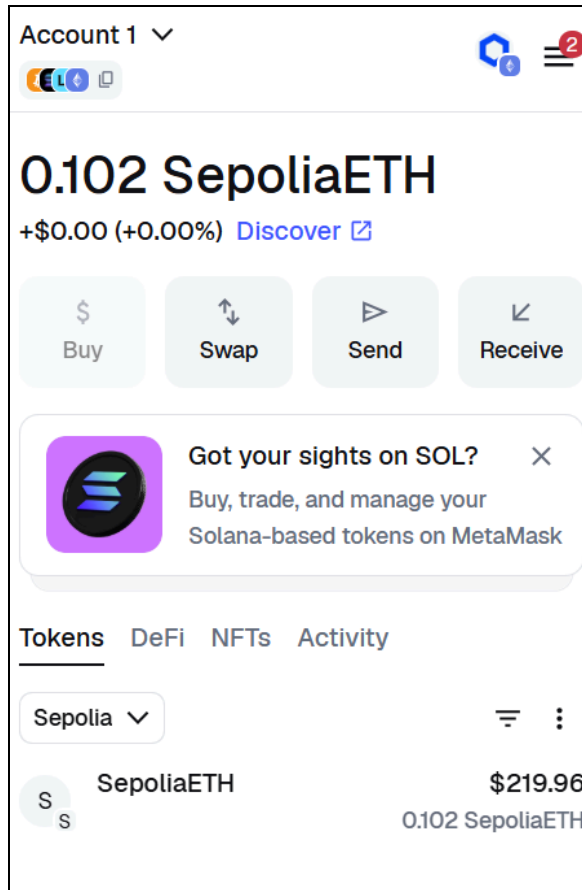
The smart contract deployment process involves writing the contract in Solidity, compiling it to eliminate errors, connecting MetaMask to Remix, and deploying the contract on a testnet by paying gas fees with test tokens.

A blockchain transaction refers to any interaction with the network, such as deploying a contract or executing its functions. Each transaction requires authentication through a digital signature and is permanently recorded on the blockchain. These transactions can be verified using blockchain explorers.

Step 1 —Install MetaMask

Install Metamask and create wallet





Step 2 — Deploy BugBountyToken First

1. In Remix, go to the **Deploy tab** (Ethereum icon on left)
2. Make sure Environment is "**Injected Provider - MetaMask**"
3. In the **Contract dropdown**, select "**BugBountyToken**"
4. Click **Deploy**
5. MetaMask will pop up → click **Confirm**
6. Wait a few seconds → you'll see it appear under "**Deployed Contracts**"
7. **Copy and save its address** (click the copy icon next to it)

REMITX1.5.1

default_workspace

Login with GitHubTheme

DEPLOY & RUN TRANSACTIONS

ACCOUNT0xc3d...6e81f (0 ETH)

GAS LIMIT

Estimated Gas

Custom3000000

VALUE

0Wei

CONTRACT

BugBountyToken - BugBountyToken.s

even version: osaka

Verify Contract on Explorers

Deploy & Verify

At AddressLoad contract from Address

Transactions recorded1

Deployed Contracts0

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

69

70

71

72

73

74

75

76

77

78

79

80

81

82

83

84

85

86

87

88

89

90

91

92

93

94

95

96

97

98

99

100

101

102

103

104

105

106

107

108

109

110

111

112

113

114

115

116

117

118

119

120

121

122

123

124

125

126

127

128

129

130

131

132

133

134

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

167

168

169

170

171

172

173

174

175

176

177

178

179

180

181

182

183

184

185

186

187

188

189

190

191

192

193

194

195

196

197

198

199

200

201

202

203

204

205

206

207

208

209

210

211

212

213

214

215

216

217

218

219

220

221

222

223

224

225

226

227

228

229

230

231

232

233

234

235

236

237

238

239

240

241

242

243

244

245

246

247

248

249

250

251

252

253

254

255

256

257

258

259

260

261

262

263

264

265

266

267

268

269

270

271

272

273

274

275

276

277

278

279

280

281

282

283

284

285

286

287

288

289

290

291

292

293

294

295

296

297

298

299

300

301

302

303

304

305

306

307

308

309

310

311

312

313

314

315

316

317

318

319

320

321

322

323

324

325

326

327

328

329

330

331

332

333

334

335

336

337

338

339

340

341

342

343

344

345

346

347

348

349

350

351

352

353

354

355

356

357

358

359

360

361

362

363

364

365

366

367

368

369

370

371

372

373

374

375

376

377

378

379

380

381

382

383

384

385

386

387

388

389

390

391

392

393

394

395

396

397

398

399

400

401

402

403

404

405

406

407

408

409

410

411

412

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

430

431

432

433

434

435

436

437

438

439

440

441

442

443

444

445

446

447

448

449

450

451

452

453

454

455

456

457

458

459

460

461

462

463

464

465

466

467

468

469

470

471

472

473

474

475

476

477

478

479

480

481

482

483

484

485

486

487

488

489

490

491

492

493

494

495

496

497

498

499

500

501

502

503

504

505

506

507

508

509

510

511

512

513

514

515

516

517

518

519

520

521

522

523

524

525

526

527

528

529

530

531

532

533

534

535

536

537

538

539

540

541

542

543

544

545

546

547

548

549

550

551

552

553

554

555

556

557

558

559

560

561

562

563

564

565

566

567

568

569

570

571

572

573

574

575

576

577

578

579

580

581

582

583

584

585

586

587

588

589

590

591

592

593

594

595

596

597

598

599

600

601

602

603

604

605

606

607

608

609

610

611

612

613

614

615

616

617

618

619

620

621

622

623

624

625

626

627

628

629

630

631

632

633

634

635

636

637

638

639

640

641

642

643

644

645

646

647

648

649

650

651

652

653

654

655

656

657

658

659

660

661

662

663

664

665

666

667

668

669

670

671

672

673

674

675

676

677

678

679

680

681

682

683

684

685

686

687

688

689

690

691

692

693

694

695

696

697

698

699

700

701

702

703

704

705

706

707

708

709

710

711

712

713

714

715

716

717

718

719

720

721

722

723

724

725

726

727

728

729

730

731

732

733

734

735

736

737

738

739

740

741

742

743

744

745

746

747

748

749

750

751

752

753

754

755

756

757

758

759

760

761

762

763

764

765

766

767

768

769

770

771

772

773

774

775

776

777

778

779

780

781

782

783

784

785

786

787

788

789

790

791

792

793

794

795

796

797

798

799

800

801

802

803

804

805

806

807

808

809

810

811

812

813

814

815

816

817

818

819

820

821

822

823

824

825

826

827

828

829

830

831

832

833

834

835

836

837

838

839

840

841

842

843

844

845

846

847

848

849

850

851

852

853

854

855

856

857

858

859

860

861

862

863

864

865

866

867

868

869

870

871

872

873

874

875

876

877

878

879

880

881

882

883

884

885

886

887

888

889

890

891

892

893

894

895

896

897

898

899

900

901

902

903

904

905

906

907

908

909

910

911

912

913

914

915

916

917

918

919

920

921

922

923

924

925

926

927

928

929

930

931

932

933

934

935

936

937

938

939

940

941

942

943

944

945

946

947

948

949

950

951

952

953

954

955

956

957

958

959

960

961

962

963

964

965

966

967

968

969

970

971

972

973

974

975

976

977

978

979

980

981

982

983

984

985

986

987

988

989

990

991

992

993

994

995

996

997

998

999

1000

1001

1002

1003

1004

1005

1006

1007

1008

1009

1010

1011

1012

1013

1014

1015

1016

1017

1018

1019

1020

1021

1022

1023

1024

1025

1026

1027

1028

1029

1030

1031

1032

1033

1034

1035

1036

1037

1038

1039

1040

1041

1042

1043

1044

1045

1046

1047

1048

1049

1050

1051

1052

1053

1054

1055

1056

1057

1058

1059

1060

1061

1062

1063

1064

1065

1066

1067

1068

1069

1070

1071

1072

1073

1074

1075

1076

1077

1078

1079

1080

1081

1082

1083

1084

1085

1086

1087

1088

1089

1090

1091

1092

1093

1094

1095

1096

1097

1098

1099

1100

1101

1102

1103

1104

1105

1106

1107

1108

1109

1110

1111

1112

1113

1114

1115

1116

1117

1118

1119

1120

1121

1122

1123

1124

1125

1126

1127

1128

1129

1130

1131

1132

1133

1134

1135

1136

1137

1138

1139

1140

1141

1142

1143

1144

1145

1146

1147

1148

1149

1150

1151

1152

1153

1154

1155

1156

1157

1158

1159

1160

1161

1162

1163

1164

1165

1166

1167

1168

1169

1170

1171

1172

1173

1174

1175

1176

1177

1178

1179

1180

1181

1182

1183

1184

1185

1186

1187

1188

1189

1190

1191

1192

1193

1194

1195

1196

1197

1198

1199

1200

1201

1202

1203

1204

1205

1206

1207

1208

1209

1210

1211

1212

1213

1214

1215

1216

1217

1218

1219

1220

1221

1222

1223

1224

1225

1226

1227

1228

1229

1230

1231

1232

1233

1234

1235

1236

1237

1238

1239

1240

1241

1242

1243

1244

1245

1246

1247

1248

1249

1250

1251

1252

1253

1254

1255

1256

1257

1258

1259

1260

1261

1262

1263

1264

1265

1266

1267

1268

1269

1270

1271

1272

1273

1274

1275

1276

1277

1278

1279

1280

1281

1282

1283

1284

1285

1286

1287

1288

1289

1290

1291

1292

1293

1294

1295

1296

1297

1298

1299

1300

1301

1302

1303

1304

1305

1306

1307

1308

1309

1310

1311

1312

1313

1314

1315

1316

1317

1318

1319

1320

1321

1322

1323

1324

1325

1326

1327

1328

1329

1330

1331

1332

1333

1334

1335

1336

1337

1338

1339

1340

1341

1342

1343

1344

1345

1346

1347

1348

1349

1350

1351

1352

1353

1354

1355

1356

1357

1358

1359

1360

1361

1362

1363

1364

1365

1366

1367

1368

1369

1370

1371

1372

1373

1374

1375

1376

1377

1378

1379

1380

1381

1382

1383

1384

1385

1386

1387

1388

1389

1390

1391

1392

1393

1394

1395

1396

1397

1398

1399

1400

1401

1402

1403

1404

1405

1406

1407

1408

1409

1410

1411

1412

1413

1414

1415

1416

1417

1418

1419

1420

1421

1422

1423

1424

1425

1426

1427

1428

1429

1430

1431

1432

1433

1434

1435

1436

1437

1438

1439

1440

1441

1442

1443

1444

1445

1446

1447

1448

1449

1450

1451

1452

1453

1454

1455

1456

1457

1458

1459

1460

1461

1462

1463

1464

1465

1466

1467

1468

1469

1470

1471

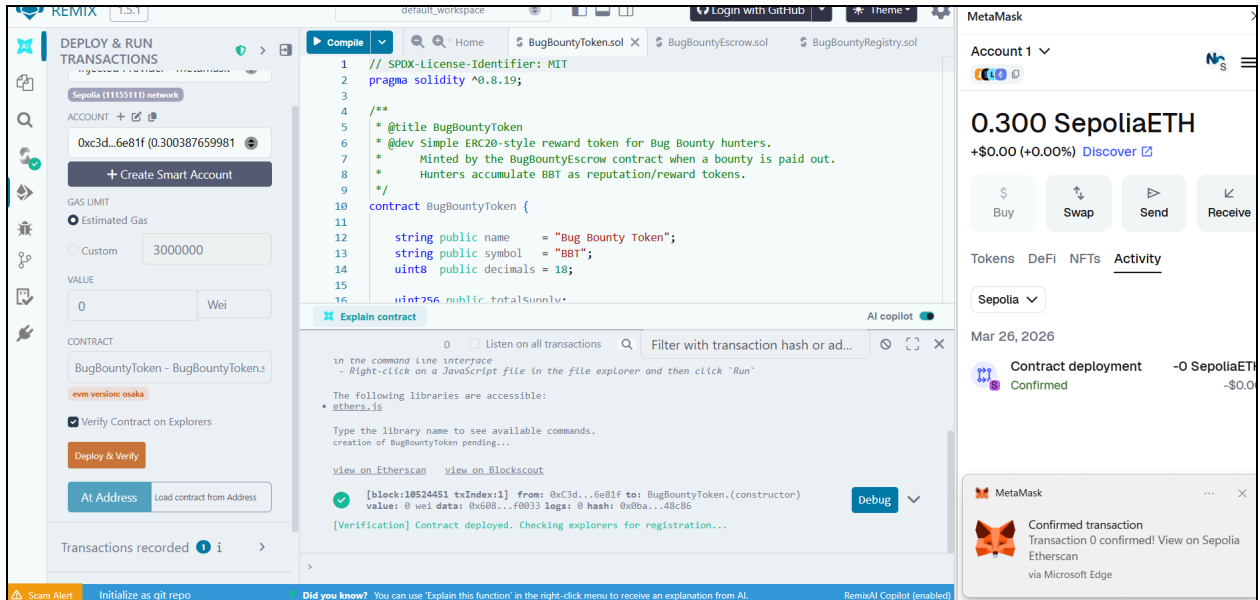
1472

1473

1474

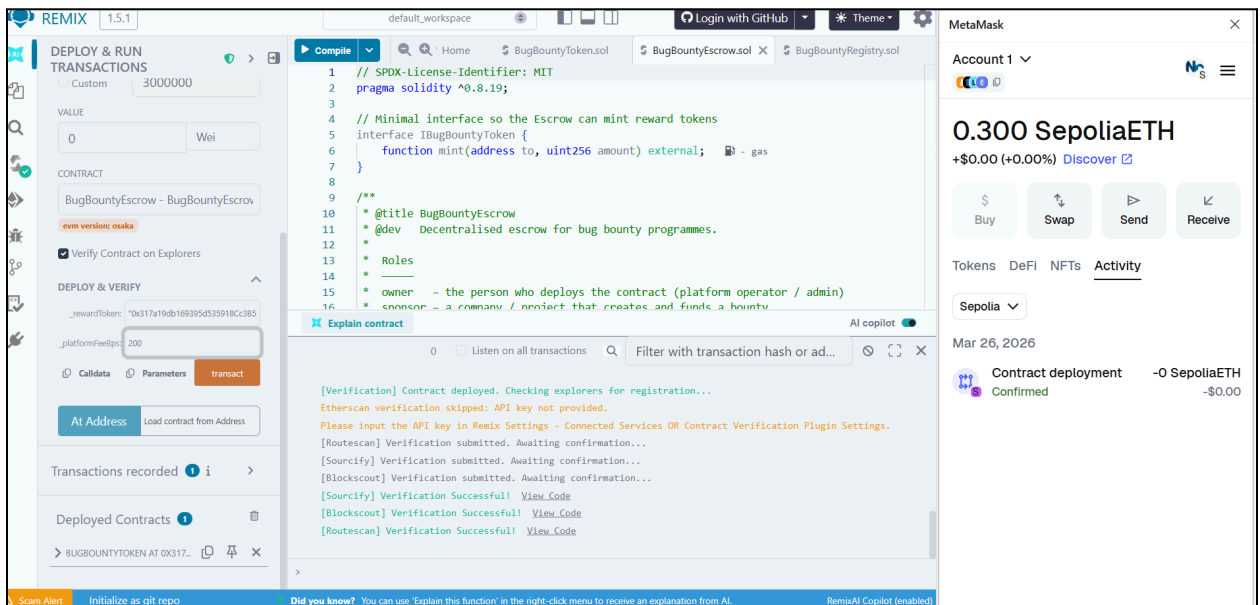
1475

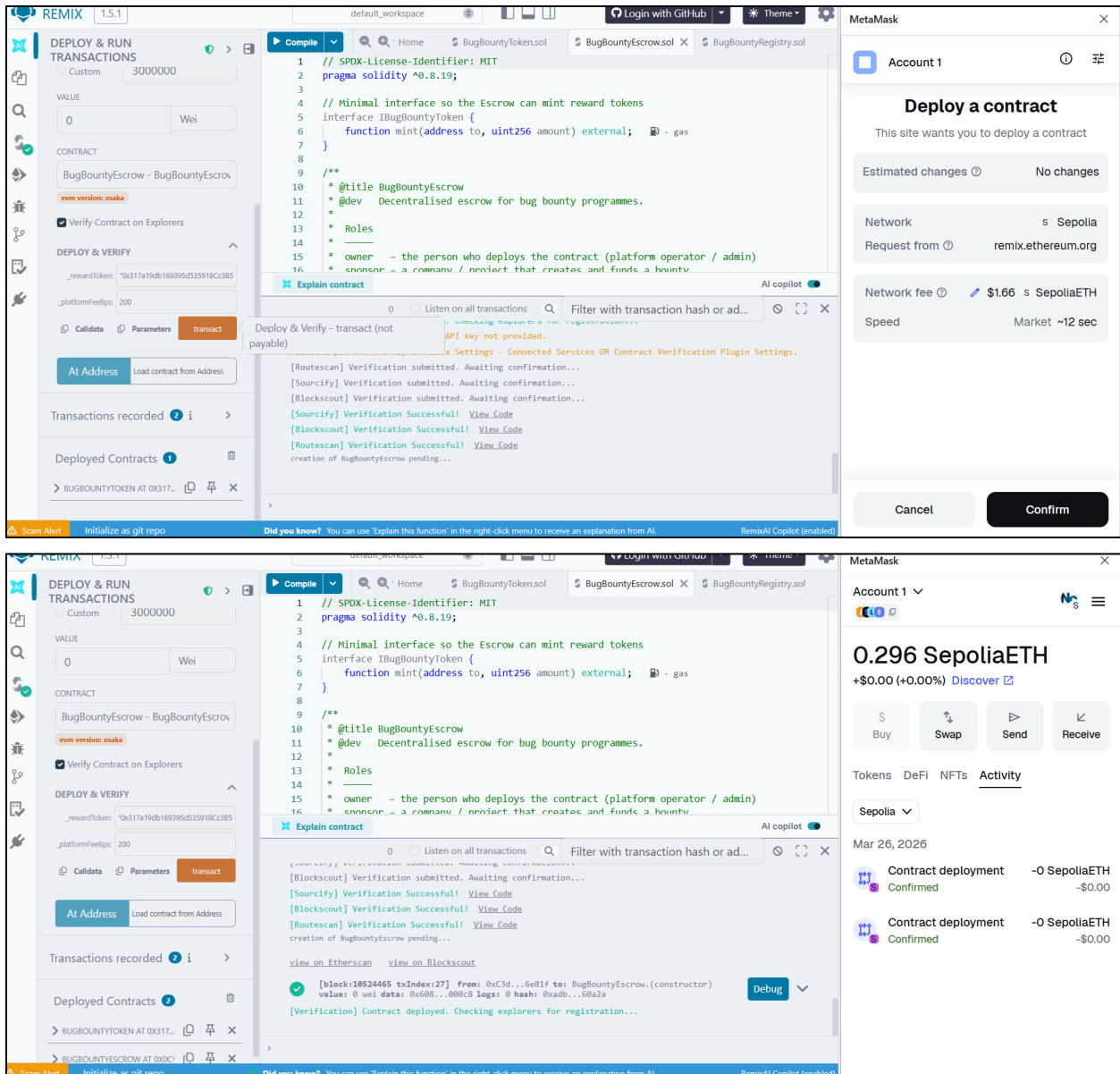
1476</



Step 3 — Deploy BugBountyEscrow

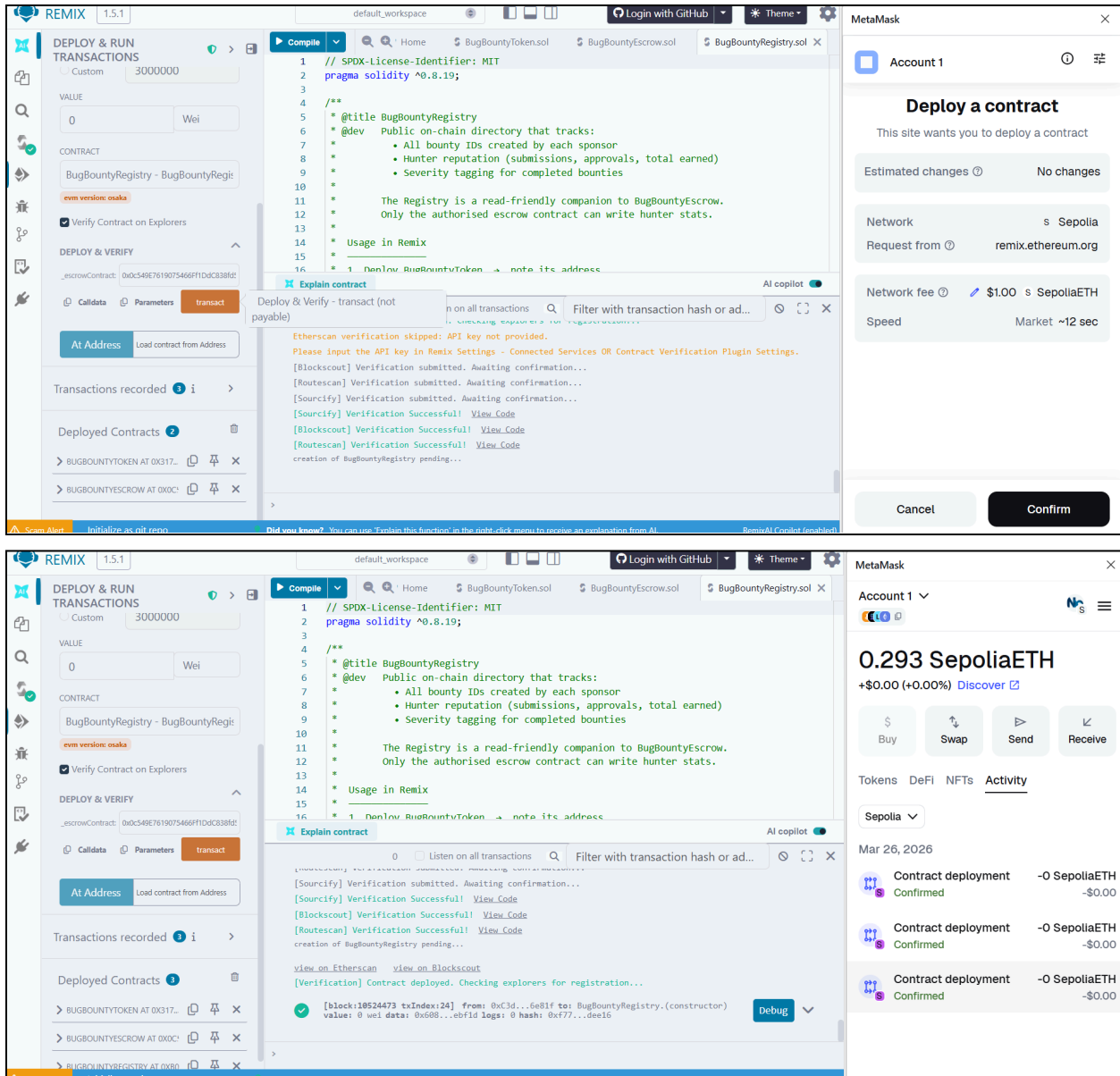
1. In the Contract dropdown, select "**BugBountyEscrow**"
2. You'll see two input fields before deploying:
 - `_rewardToken` → paste the **BugBountyToken** address from Step 1
 - `_platformFeeBps` → type **200** (means 2% fee)
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**





Step 4 — Deploy BugBountyRegistry

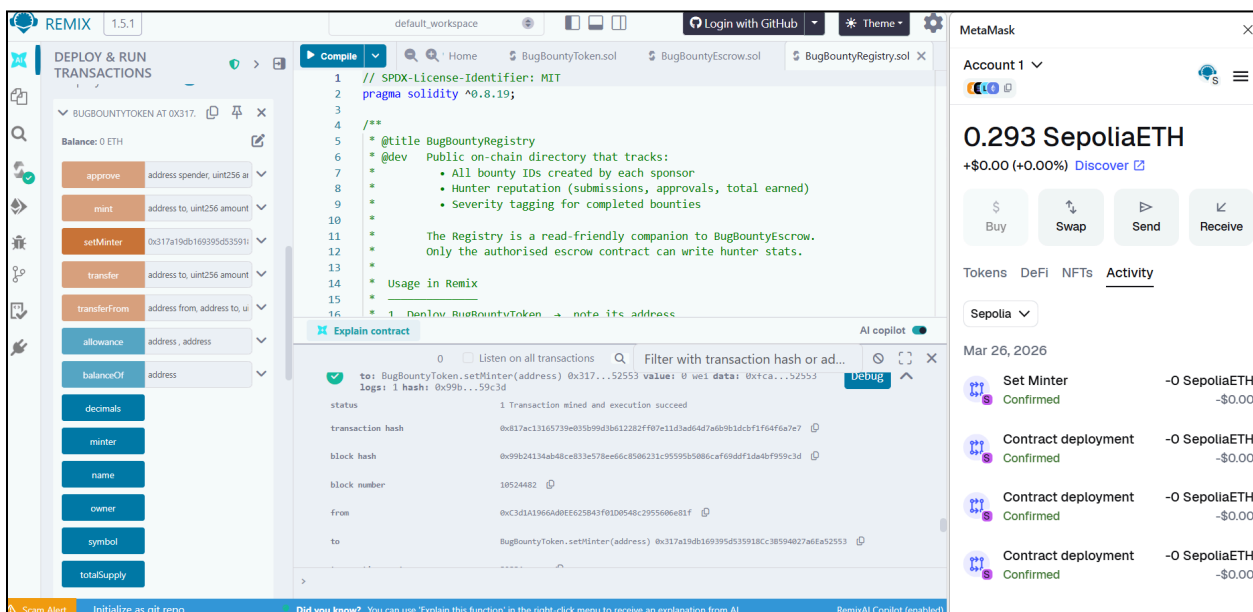
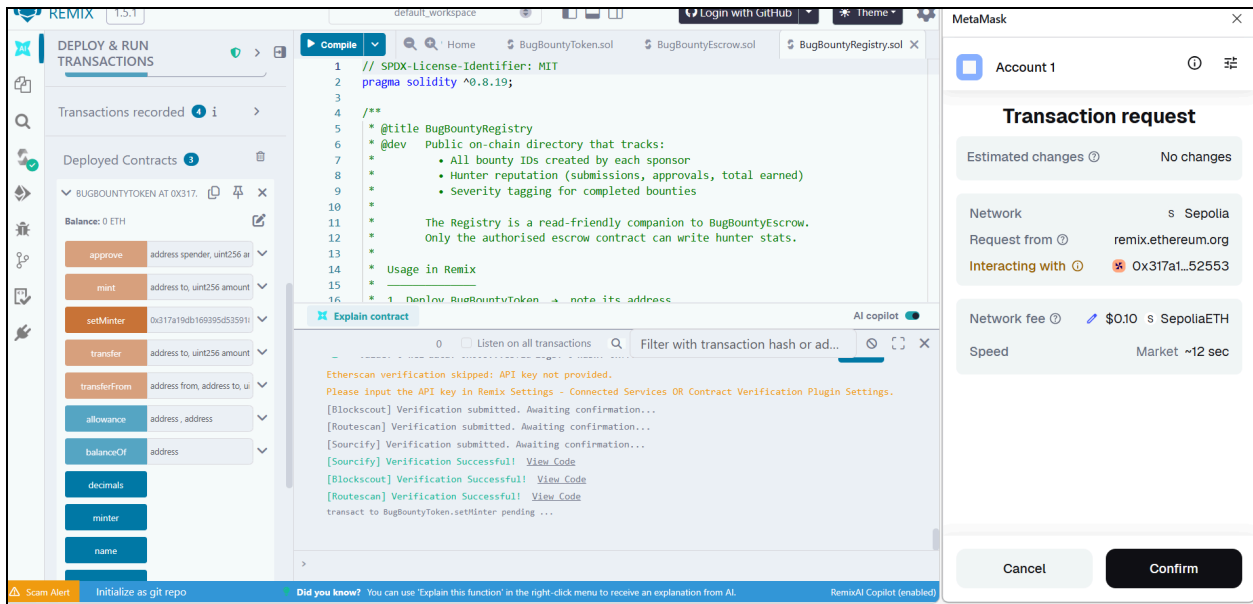
1. Select "**BugBountyRegistry**" in the dropdown
2. Input field:
 - `_escrowContract` → paste the **BugBountyEscrow** address from Step 2
3. Click **Deploy** → Confirm in MetaMask
4. **Copy and save its address**



Step 5 — Link the Contracts Together

On BugBountyToken:

1. Expand it under Deployed Contracts
2. Find **setMinter** function
3. Paste the **BugBountyEscrow** address → click it → Confirm in MetaMask



Conclusion

The experiment successfully demonstrated MetaMask integration with Remix IDE and execution of blockchain transactions. It provided hands-on experience with smart contracts, ensuring secure and transparent interactions on a testnet



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 8

Aim - To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Theory - Ganache is a personal Ethereum blockchain used for testing and development. It allows developers to deploy contracts, develop decentralized applications (DApps), and run tests in a safe and deterministic environment.

Ganache provides:

- Instant mining of blocks
- Pre-funded test accounts
- Transaction history view
- Graphical interface to monitor blockchain activities

It is part of the Truffle Suite and is commonly used with Remix and Metamask for smart contract deployment.

Steps to Connect Ganache with Metamask and Remix IDE:

1. Install Ganache:
 - Download and install Ganache from the Truffle Suite website.
 - Launch it to start a local blockchain network with test Ether accounts.
2. Connect Ganache Accounts with Metamask:
 - Open Metamask and switch to a Custom RPC Network.
 - Enter the Ganache RPC server URL (e.g., [HTTP://127.0.0.1:7545](http://127.0.0.1:7545)).
 - Import private keys of Ganache accounts into Metamask.
3. Connect Remix IDE with Metamask:
 - Open Remix IDE in the browser.
 - Choose Injected Web3 as the environment in the “Deploy & Run Transactions” tab.
 - Remix will automatically connect to the Metamask account (linked with Ganache).
4. Create a Simple Solidity Smart Contract:
 - Write a Solidity program related to your mini project (e.g., Student Record, Donation Tracker, etc.).

```
pragma solidity ^0.8.0;
contract SimpleStorage {
    uint public data;
    function set(uint x) public {
        data = x;
    }
}
```

5. Compile and Deploy the Contract:
Compile using Remix.
 - Deploy via the Metamask account connected to Ganache.
 - Approve the transaction in Metamask.
6. Check Transaction Details on Ganache:
 - View mined transactions, block details, and balances in the Ganache interface.
7. Interact with the Smart Contract:
 - Use Remix to call contract functions (**set()** or **get()**).
 - Observe the state changes and corresponding transactions on Ganache.

Code -

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Escrow {

    address public buyer;
    address payable public seller;
    address public arbiter;

    bool public buyerApproved;
    bool public sellerApproved;

    constructor(address payable _seller, address _arbiter) payable {
        buyer = msg.sender;
        seller = _seller;
        arbiter = _arbiter;
    }

    function approve() public {
        require(msg.sender == buyer || msg.sender == seller, "Not authorized");

        if (msg.sender == buyer) {
            buyerApproved = true;
        }

        if (msg.sender == seller) {
            sellerApproved = true;
        }

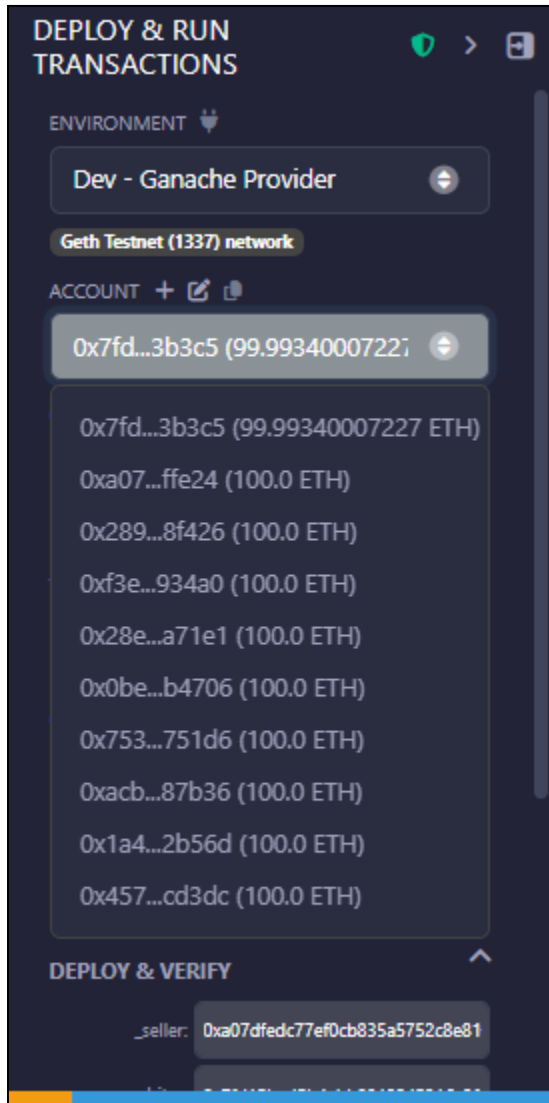
        if (buyerApproved && sellerApproved) {

            uint amount = address(this).balance;

            (bool success, ) = seller.call{value: amount}("");
```

```
        require(success, "Transfer failed");
    }
}
```

Output -



CURRENT BLOCK
2

GAS PRICE
2000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

TX HASH

0xb85506ee1d0385b8a59d33a0e97cbdc501667e7d45ef7ac1f5beec6aa9a79243

CONTRACT CREATION

FROM ADDRESS

0x7fd15bad5b4AbB0912345316A3fcD453B2a3b3C5

CREATED CONTRACT ADDRESS

0x7b10fEE11188F75915ce12A07120d8442A7638af

GAS USED

3000000

VALUE

0

TX HASH

0xa4f74fed3fc1e12bcc5777e0337466b545001a6ecac86fdea5c9e04f4f7aa085

CONTRACT CREATION

FROM ADDRESS

0x7fd15bad5b4AbB0912345316A3fcD453B2a3b3C5

CREATED CONTRACT ADDRESS

0x018b4fb3336095E9cdf99880d8f8C7D22e24ECC

GAS USED

3000000

VALUE

0

Add wallet

Private key

Imported private keys are backed up to your account and sync automatically when you sign in with the same Google or Apple login.
[Learn more](#) about how imported keys work

Select type

Private key ▾

Enter your private key string here:

.....

Cancel

Import

ACCOUNT INFORMATION

ACCOUNT ADDRESS

0xA07DFedc77EF0cB835a5752c8e8108E7cA4ffe24

PRIVATE KEY

0xfeb942cbaeb4cc850a33f5c87a3e5af51e76d998538bff2fad3f0c643128ed24

Do not use this private key on a public blockchain; use it for development purposes only!

DONE

Conclusion:

Ganache, Metamask, and Remix together create a powerful local blockchain development setup. Ganache provides a personal Ethereum blockchain with instant mining and pre-funded accounts, Metamask acts as the wallet interface to manage those accounts, and Remix IDE allows you to write, compile, and deploy Solidity smart contracts. By connecting Ganache to Metamask and using Remix's Injected Web3 environment, developers can safely test contracts, observe transactions, and validate DApp functionality before moving to a public testnet or mainnet. In short, this workflow offers a secure sandbox for building and refining decentralized applications efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 9

Aim - To implement a Private Ethereum Blockchain using Geth.

Theory -

Ethereum is an open-source, decentralized blockchain platform that enables developers to build and deploy **smart contracts** and **decentralized applications (DApps)**. Unlike public blockchains such as the Ethereum mainnet, a **Private Ethereum Blockchain** is restricted to specific participants, making it ideal for **development, testing, and enterprise applications** where confidentiality, control, and performance are essential.

Geth (Go-Ethereum) is one of the most widely used Ethereum clients, written in the Go programming language. It allows users to run an Ethereum node, mine Ether, and interact with the blockchain through **JSON-RPC, command-line, or JavaScript consoles**.

Setting up a **Private Ethereum Network** involves creating a customized blockchain environment isolated from the public Ethereum network. It enables developers to test smart contracts, consensus mechanisms, and network behavior without incurring gas costs or depending on public validators.

Steps Involved:

1. **Choosing a Network ID:**
2. Each private blockchain must have a unique network ID to differentiate it from public or other private networks.
3. **Choosing a Consensus Algorithm:**
4. Ethereum supports different consensus mechanisms such as **Proof of Work (PoW)** and **Proof of Authority (PoA)**. In private networks, **Clique (PoA)** is often used for faster block confirmation.
5. **Creating a Genesis Block:**
6. The genesis block is the first block of the blockchain, defining network parameters such as difficulty, gas limit, initial account balances, and consensus settings.
7. **Initializing the Geth Database:**
8. The `geth init` command initializes the blockchain using the `genesis.json` file, preparing the data directory for node operation.
9. **Setting up Networking:**
10. Nodes communicate using **enodes** (Ethereum Node URLs) that contain IP addresses and public keys for peer discovery and connection.
11. **Running the Member Nodes:**

12. Each participant runs a Geth node that connects to the private network and participates in block validation and transaction processing.
13. **Running a Signer (in Clique):**
14. In the **Clique consensus**, designated **signers** validate and sign blocks. These accounts are pre-configured in the genesis file to ensure trusted block production.

Outputs -

1. First install all the necessary dependencies like curl, git, build-essential.
`sudo apt install wget curl git build-essential -y`

```
pranav@DESKTOP-OSHSLJ1:~$ sudo apt install wget curl git build-essential -y
[sudo] password for pranav:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
curl is already the newest version (8.5.0-2ubuntu10.8).
git is already the newest version (1:2.43.0-1ubuntu7.3).
build-essential is already the newest version (12.10ubuntu1).
0 upgraded, 0 newly installed, 0 to remove and 11 not upgraded.
```

2. Install geth using wget
`cd /usr/local/bin && sudo wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz`

```
pranav@DESKTOP-OSHSLJ1:~$ cd /usr/local/bin && sudo wget https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
--2026-03-31 13:59:45-- https://gethstore.blob.core.windows.net/builds/geth-linux-amd64-1.12.0-e501b3b0.tar.gz
Resolving gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)... 20.60.40.164
Connecting to gethstore.blob.core.windows.net (gethstore.blob.core.windows.net)[20.60.40.164]:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 26856444 (26M) [application/octet-stream]
Saving to: 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz.1'

geth-linux-amd64-1.12.0-e501b 100%[=====] 25.61M 66.3KB/s in 2m 40s
2026-03-31 14:02:30 (164 KB/s) - 'geth-linux-amd64-1.12.0-e501b3b0.tar.gz.1' saved [26856444/26856444]
```

3. Extract the tar file and move the geth into /usr/local/bin/geth path & check the geth version
`sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz`
`sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth`
`geth version`

```
pranav@DESKTOP-OSHSLJ1:/usr/local/bin$ sudo tar -xvf geth-linux-amd64-1.12.0-e501b3b0.tar.gz
sudo mv geth-linux-amd64-1.12.0-e501b3b0/geth /usr/local/bin/geth
geth version
geth-linux-amd64-1.12.0-e501b3b0/
geth-linux-amd64-1.12.0-e501b3b0/COPYING
geth-linux-amd64-1.12.0-e501b3b0/geth
Geth
Version: 1.12.0-stable
Git Commit: e501b3b05db8e169f67dc78b7b59bc352b3c638d
Git Commit Date: 20230525
Architecture: amd64
Go Version: go1.20.3
Operating System: linux
GOPATH=
GOROOT=
```

4. Create a folder named private_ethereum_setup and create another 2 directories naming node1 and node2 individually inside private_ethereum_setup directory.
`mkdir private_ethereum_setup`
`cd private_ethereum_setup && mkdir node1 node2`

```
pranav@DESKTOP-OSHSLJI:~$ mkdir private_ethereum_setup
cd private_ethereum_setup && mkdir node1 node2
pranav@DESKTOP-OSHSLJI:~/private_ethereum_setup$ ls
node1  node2
```

5. Check the files

6. Create account for node1

```
geth --datadir "node1" account new
```

```
pranav@DESKTOP-OSHSLJI:~/private_ethereum_setup$ geth --datadir "node1" account new
INFO [03-31|14:05:28.888] Maximum peer count      ETH=50 LES=0 total=50
INFO [03-31|14:05:28.888] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0xC99cf64ef49425Bf0e814c9d51364281a61747d2
Path of the secret key file: node1/keystore/UTC--2026-03-31T14-05-36.977302946Z--c99cf64ef49425bf0e814c9d51364281a61747d2

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

7. Do the same for node2

```
geth --datadir "node2" account new
```

```
pranav@DESKTOP-OSHSLJI:~/private_ethereum_setup$ geth --datadir "node2" account new
INFO [03-31|14:05:46.530] Maximum peer count      ETH=50 LES=0 total=50
INFO [03-31|14:05:46.531] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such file or directory"
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x19F1609855626D5551A589497aCb92966c69C778
Path of the secret key file: node2/keystore/UTC--2026-03-31T14-05-58.107593532Z--19f1609855626d5551a589497acb92966c69c778

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

8. Create a file called genesis.json in the same directory and copy paste these contents

```
{
  "config": {
    "chainId": 12345,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0,
    "berlinBlock": 0,
```



```

GNU nano 7.2                                genesis.json
"config": {
  "chainId": 12345,
  "homesteadBlock": 0,
  "eip150Block": 0,
  "eip155Block": 0,
  "eip158Block": 0,
  "byzantiumBlock": 0,
  "constantinopleBlock": 0,
  "petersburgBlock": 0,
  "istanbulBlock": 0,
  "berlinBlock": 0,
  "londonBlock": 0,
  "clique": {
    "period": 5,
    "epoch": 30000
  }
},
"difficulty": "1",
"gasLimit": "8000000",
"extradata": "0x0000000000000000000000000000000000000000000000000000000000000000c99cf64ef49425bf0e814c9d51364281a61747d200000000000000000000000000000",
"alloc": {
  "0xc99cf64ef49425bf0e814c9d51364281a61747d2": {
    "balance": "1000000000000000000"
  },
  "0x19f1609855626D5551A589497aCb92966c69C778": {
    "balance": "1000000000000000000"
  }
},
"config": {
  "chainId": 12345,
  "homesteadBlock": 0,
  "eip150Block": 0,

```

Read 59 lines

9. Initializing geth for node1 using the genesis.json file

geth init --datadir node1 genesis.json

```

pranav@DESKTOP-USHSLJ1:~/private_ethereum_setup$ geth init --datadir node1 genesis.json
INFO [03-31|14:14:02.677] Maximum peer count                ETH=50 LES=0 total=50
INFO [03-31|14:14:02.677] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|14:14:02.680] Set global gas cap                  cap=50,000,000
INFO [03-31|14:14:02.680] Initializing the KZG library         backend=gokzg
INFO [03-31|14:14:02.729] Defaulting to pebble as the backing database
INFO [03-31|14:14:02.729] Allocated cache and file handles     database=/home/pranav/private_ethereum_setup/node1/genesis.json
th/chaindata cache=16.00MiB handles=16
INFO [03-31|14:14:02.805] Opened ancient database              database=/home/pranav/private_ethereum_setup/node1/genesis.json
th/chaindata/ancient/chain readonly=false
INFO [03-31|14:14:02.805] Writing custom genesis block
INFO [03-31|14:14:02.805] Persisted trie from memory database  nodes=3 size=411.00B time="33.822µs" gcnodes=0 gcsiz
=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|14:14:02.826] Successfully wrote genesis state      database=chaindata hash=08f6ea..15522b
INFO [03-31|14:14:02.826] Defaulting to pebble as the backing database
INFO [03-31|14:14:02.826] Allocated cache and file handles     database=/home/pranav/private_ethereum_setup/node1/genesis.json
th/lightchaindata cache=16.00MiB handles=16
INFO [03-31|14:14:02.899] Opened ancient database              database=/home/pranav/private_ethereum_setup/node1/genesis.json
th/lightchaindata/ancient/chain readonly=false
INFO [03-31|14:14:02.899] Writing custom genesis block
INFO [03-31|14:14:02.900] Persisted trie from memory database  nodes=3 size=411.00B time="22.46µs" gcnodes=0 gcsiz
=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|14:14:02.912] Successfully wrote genesis state      database=lightchaindata hash=08f6ea..15522b

```

10. Do the same for node2

geth init --datadir node2 genesis.json

```

pranav@DESKTOP-OSHSLJ1:~/private_ethereum_setup$ geth init --datadir node2 genesis.json
INFO [03-31|14:14:07.414] Maximum peer count          ETH=50 LES=0 total=50
INFO [03-31|14:14:07.415] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|14:14:07.418] Set global gas cap          cap=50,000,000
INFO [03-31|14:14:07.418] Initializing the KZG library backend=gokzg
INFO [03-31|14:14:07.448] Defaulting to pebble as the backing database
INFO [03-31|14:14:07.448] Allocated cache and file handles database=/home/pranav/private_ethereum_setup/node2/chaindata cache=16.00MiB handles=16
INFO [03-31|14:14:07.520] Opened ancient database      database=/home/pranav/private_ethereum_setup/node2/chaindata/ancient/chain readonly=false
INFO [03-31|14:14:07.520] Writing custom genesis block
INFO [03-31|14:14:07.520] Persisted trie from memory database nodes=3 size=411.00B time="111.483µs" gcnodes=0 gcsizes=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|14:14:07.533] Successfully wrote genesis state database=chaindata hash=08f6ea..15522b
INFO [03-31|14:14:07.534] Defaulting to pebble as the backing database
INFO [03-31|14:14:07.534] Allocated cache and file handles database=/home/pranav/private_ethereum_setup/node2/lightchaindata cache=16.00MiB handles=16
INFO [03-31|14:14:07.597] Opened ancient database      database=/home/pranav/private_ethereum_setup/node2/lightchaindata/ancient/chain readonly=false
INFO [03-31|14:14:07.597] Writing custom genesis block
INFO [03-31|14:14:07.598] Persisted trie from memory database nodes=3 size=411.00B time="32.835µs" gcnodes=0 gcsizes=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [03-31|14:14:07.610] Successfully wrote genesis state database=lightchaindata hash=08f6ea..15522b

```

11. Now we will install bootnode for configuring both the nodes.

```

sudo add-apt-repository -y ppa:ethereum/ethereum
sudo apt-get update
sudo apt-get install bootnode

```

12. Create a key for the bootnode and save it to boot.key in the folder

```
bootnode -genkey boot.key
```

```

deilsy@deilsy:~/private_ethereum_setup$ bootnode -genkey boot.key
deilsy@deilsy:~/private_ethereum_setup$ ls
boot.key genesis.json node1 node2
deilsy@deilsy:~/private_ethereum_setup$

```

13. Copy this command and copy the enode://.... This will be required later

```
bootnode -nodekey boot.key -verbosity 9 -addr :30305
```

```

pranav@DESKTOP-OSHSLJ1:~/private_ethereum_setup$ bootnode -nodekey boot.key -addr :30305
enode://65150fcfbf2bac27c639de2d15c6265f8135751d09041a248806606fef4a44206a0bf0673458a5dcb1146bda9aa54d511cbcabfd26c807257191ec1c77b464720127.0.0.1:0?discport=30305
Note: you're using cmd/bootnode, a developer tool.
We recommend using a regular node as bootstrap node for production deployments.
INFO [03-31|14:15:43.440] New local node record      seq=1,774,966,543 id=c51b56f7e246677f ip="invalid IP" udp=0 tcp=0

```

14. Now on another terminal we will start mining node1

```

geth --datadir node1 --port 30306 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af554ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid 123454321
--unlock 0x563A5a670260417B163AD1696fCBfA6849906d06 --password
node1/password.txt --authrpc.port 8551 --miner.etherbase
0x563A5a670260417B163AD1696fCBfA6849906d06 --mine

```

Replace the address with your own created

```

pranav@DESKTOP-0SHSLJ1: /private_ethereum_setup$ geth --datadir node1 \
--port 30306 \
--networkid 12345 \
--bootnodes "enode://65150fcfbf2bac27c639de2d15c265f8135751d09041a24880606fef4a44206a0bf073458a5dcb1146bda9aa54d511bcabfd26c807257191ec1c77b46472@127.0.0.1:30305" \
--mine \
--miner.etherbase 0xC99cf64ef49425Bf0e814c9d51364281a61747d2 \
--http --http.port 8545 --http.api "eth,net,web3,personal,miner" \
--authrpc.port 8551 \
--allow-insecure-unlock \
--unlock 0xC99cf64ef49425Bf0e814c9d51364281a61747d2
INFO [03-31|14:17:28.316] Maximum peer count          ETH=50 LES=0 total=50
INFO [03-31|14:17:28.317] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|14:17:28.320] Set global gas cap          cap=50,000,000
INFO [03-31|14:17:28.320] Initializing the KZG library backend=gokzg
INFO [03-31|14:17:28.355] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [03-31|14:17:28.355] Using pebble as the backing database database=/home/pranav/private_ethereum_setup/node1/geth/chaindata cache=512.00MiB handles
INFO [03-31|14:17:28.355] Allocated cache and file handles database=/home/pranav/private_ethereum_setup/node1/geth/chaindata/ancient/chain readonly=
=524,288
INFO [03-31|14:17:28.395] Opened ancient database
false
INFO [03-31|14:17:28.396] Initialising Ethereum protocol network=12345 dbversion=<nil>
INFO [03-31|14:17:28.396] -----
INFO [03-31|14:17:28.396] Chain ID: 12345 (unknown)
INFO [03-31|14:17:28.396] Consensus: Clique (proof-of-authority)
INFO [03-31|14:17:28.396] Pre-Merge hard forks (block based):
INFO [03-31|14:17:28.396]   - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/homestead.md)
INFO [03-31|14:17:28.396]   - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrad
es/tangerine-whistle.md)

Unlocking account 0xC99cf64ef49425Bf0e814c9d51364281a61747d2 | Attempt 1/3
Password:
INFO [03-31|14:17:35.536] Unlocked account address=0xC99cf64ef49425Bf0e814c9d51364281a61747d2
INFO [03-31|14:17:35.536] Transaction pool price threshold updated price=0
INFO [03-31|14:17:35.536] Transaction pool price threshold updated price=1,000,000,000
INFO [03-31|14:17:35.536] Commit new sealing work number=1 sealhash=2afec2..f3e4eb uncles=0 txs=0 gas=0 fees=0 elapsed="136.626µs"
INFO [03-31|14:17:35.536] Commit new sealing work number=1 sealhash=2afec2..f3e4eb uncles=0 txs=0 gas=0 fees=0 elapsed="395.7µs"
INFO [03-31|14:17:35.536] Successfully sealed new block number=1 sealhash=2afec2..f3e4eb hash=bc1863..339be8 elapsed="430.403µs"
INFO [03-31|14:17:35.537] "⚡ mined potential block" number=1 hash=bc1863..339be8
INFO [03-31|14:17:35.537] Commit new sealing work number=2 sealhash=973592..5f5a4f uncles=0 txs=0 gas=0 fees=0 elapsed="338.367µs"
INFO [03-31|14:17:35.537] Commit new sealing work number=2 sealhash=973592..5f5a4f uncles=0 txs=0 gas=0 fees=0 elapsed="637.7µs"
INFO [03-31|14:17:39.308] Looking for peers peercount=0 tried=1 static=0
INFO [03-31|14:17:40.001] Successfully sealed new block number=2 sealhash=973592..5f5a4f hash=ae892f..415799 elapsed=4.463s
INFO [03-31|14:17:40.001] "⚡ mined potential block" number=2 hash=ae892f..415799
INFO [03-31|14:17:40.001] Commit new sealing work number=3 sealhash=4e5126..8e1a45 uncles=0 txs=0 gas=0 fees=0 elapsed="358.797µs"
INFO [03-31|14:17:40.001] Commit new sealing work number=3 sealhash=4e5126..8e1a45 uncles=0 txs=0 gas=0 fees=0 elapsed="631.994µs"
INFO [03-31|14:17:45.001] Successfully sealed new block number=3 sealhash=4e5126..8e1a45 hash=ace1d8..8d69c0 elapsed=4.999s
INFO [03-31|14:17:45.001] "⚡ mined potential block" number=3 hash=ace1d8..8d69c0
INFO [03-31|14:17:45.001] Commit new sealing work number=4 sealhash=394b55..a0be7f uncles=0 txs=0 gas=0 fees=0 elapsed="351.696µs"
INFO [03-31|14:17:45.001] Commit new sealing work number=4 sealhash=394b55..a0be7f uncles=0 txs=0 gas=0 fees=0 elapsed="600.929µs"
INFO [03-31|14:17:49.330] Looking for peers peercount=0 tried=0 static=0
INFO [03-31|14:17:50.000] Successfully sealed new block number=4 sealhash=394b55..a0be7f hash=783ee0..2a21ec elapsed=4.999s
INFO [03-31|14:17:50.000] "⚡ mined potential block" number=4 hash=783ee0..2a21ec
INFO [03-31|14:17:50.002] Commit new sealing work number=5 sealhash=0b3313..f3a620 uncles=0 txs=0 gas=0 fees=0 elapsed=1.601ms
INFO [03-31|14:17:50.003] Commit new sealing work number=5 sealhash=0b3313..f3a620 uncles=0 txs=0 gas=0 fees=0 elapsed=2.294ms
INFO [03-31|14:17:55.001] Successfully sealed new block number=5 sealhash=0b3313..f3a620 hash=7fd986..df3592 elapsed=4.999s
INFO [03-31|14:17:55.001] "⚡ mined potential block" number=5 hash=7fd986..df3592
INFO [03-31|14:17:55.002] Commit new sealing work number=6 sealhash=1bdf88..5e952b uncles=0 txs=0 gas=0 fees=0 elapsed="509.252µs"
INFO [03-31|14:17:55.002] Commit new sealing work number=6 sealhash=1bdf88..5e952b uncles=0 txs=0 gas=0 fees=0 elapsed="999.701µs"
INFO [03-31|14:17:59.347] Looking for peers peercount=0 tried=0 static=0
INFO [03-31|14:18:00.000] Successfully sealed new block number=6 sealhash=1bdf88..5e952b hash=d6641b..dd3338 elapsed=4.998s
INFO [03-31|14:18:00.001] "⚡ mined potential block" number=6 hash=d6641b..dd3338
INFO [03-31|14:18:00.002] Commit new sealing work number=7 sealhash=667df4..412fc9 uncles=0 txs=0 gas=0 fees=0 elapsed=1.081ms
INFO [03-31|14:18:00.002] Commit new sealing work number=7 sealhash=667df4..412fc9 uncles=0 txs=0 gas=0 fees=0 elapsed=1.667ms

```

15. Now do same for node2

```

geth --datadir node2 --port 30307 --bootnodes
enode://3f690f88c00ce563d9dd609b0674ad97df9fc6b77673bd86917
f51333118e81f54e75919746a331d55af7f6edba870a5ef5ed12b60af55
4ccb922bc77ab41f27@127.0.0.1:0?discport=30305 --networkid
123454321 --unlock
0x6d04eA66725A40F5669E2d6f4f77eb4Ecf756a3f --password
node2/password.txt --authrpc.port 8552

```



```

pranav@DESKTOP-OSHSLJ1:~/private_ethereum_setup$ geth --datadir node2 \
--port 30307 \
--networkid 12345 \
--bootnodes "enode://65150fcfbf2bac27c639de2d15c6265f8135751d09041a248806606fef4a44206a0bf0673458a5dcb1146bda9aa54d511cbcabfd26c807257191ec1c77b46472@127.0.0.1:30305" \
--http --http.port 8546 --http.api "eth,net,web3" \
--authrpc.port 8552
INFO [03-31|14:18:32.384] Maximum peer count ETH=50 LES=0 total=50
INFO [03-31|14:18:32.385] Smartcard socket not found, disabling err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [03-31|14:18:32.387] Set global gas cap cap=50,000,000
INFO [03-31|14:18:32.387] Initializing the KZG library backend=gokzg
INFO [03-31|14:18:32.433] Allocated trie memory caches clean=154.00MiB dirty=256.00MiB
INFO [03-31|14:18:32.433] Using pebble as the backing database
INFO [03-31|14:18:32.433] Allocated cache and file handles database=/home/pranav/private_ethereum_setup/node2/geth/chaindata cache=512.00MiB handles=524,288
INFO [03-31|14:18:32.479] Opened ancient database database=/home/pranav/private_ethereum_setup/node2/geth/chaindata/ancient/chain readonly=false
INFO [03-31|14:18:32.479] Initialising Ethereum protocol network=12345 dbversion=<nil>
INFO [03-31|14:18:32.480] -----
INFO [03-31|14:18:32.480] Chain ID: 12345 (unknown)
INFO [03-31|14:18:32.480] Consensus: Clique (proof-of-authority)
INFO [03-31|14:18:32.480] Pre-Merge hard forks (block based):
INFO [03-31|14:18:32.480] - Homestead: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/homestead.md)
INFO [03-31|14:18:32.480] - Tangerine Whistle (EIP 150): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/tangerine-whistle.md)
INFO [03-31|14:18:32.480] - Spurious Dragon/1 (EIP 155): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/spurious-dragon.md)
INFO [03-31|14:18:32.480] - Spurious Dragon/2 (EIP 158): #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/spurious-dragon.md)
INFO [03-31|14:18:32.480] - Byzantium: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/byzantium.md)
INFO [03-31|14:18:32.480] - Constantinople: #0 (https://github.com/ethereum/execution-specs/blob/master/network-upgrades/mainnet-upgrade/constantinople.md)
INFO [03-31|14:18:32.515] Started P2P networking
INFO [03-31|14:18:32.516] IPC endpoint opened url=/home/pranav/private_ethereum_setup/node2/geth.ipc
INFO [03-31|14:18:32.517] Generated JWT secret path=/home/pranav/private_ethereum_setup/node2/geth/jwtsecret
INFO [03-31|14:18:32.517] HTTP server started endpoint=127.0.0.1:8546 auth=false prefix= cors= vhosts=localhost
INFO [03-31|14:18:32.518] WebSocket enabled url=ws://127.0.0.1:8552
INFO [03-31|14:18:32.518] HTTP server started endpoint=127.0.0.1:8552 auth=true prefix= cors=localhost vhosts=localhost
INFO [03-31|14:18:43.349] Block synchronisation started
INFO [03-31|14:18:43.365] Syncing: state download in progress synced=31.92% state=196.00B accounts=1@196.00B slots=0@0.00B codes=0@0.00B eta=14.833ms
INFO [03-31|14:18:43.365] Imported new chain segment number=14 hash=e2f217..cce6a9 blocks=14 txs=0 mgas=0.000 elapsed=1.095ms mgasps=0.000
INFO [03-31|14:18:43.365] Syncing: chain download in progress dirty=0.00B
INFO [03-31|14:18:43.365] Syncing: state download in progress synced=+Inf% chain=18.00B headers=14@6.00B bodies=14@6.00B receipts=14@6.00B eta=-2.290ms
INFO [03-31|14:18:43.365] Syncing: state download in progress synced=43.80% state=392.00B accounts=2@392.00B slots=0@0.00B codes=0@0.00B eta=9.446ms
WARN [03-31|14:18:43.365] Synchronisation failed, retrying err="sync cancelled"
WARN [03-31|14:18:43.366] Unexpected account range packet peer=2d9ec9f1 reqid=4,747,147,368,783,890,389
INFO [03-31|14:18:43.373] Looking for peers peercount=1 tried=2 static=0
WARN [03-31|14:18:45.003] Snap syncing, discarded propagated block number=15 hash=446164..829702
INFO [03-31|14:18:53.368] Syncing: state download in progress synced=43.80% state=392.00B accounts=2@392.00B slots=0@0.00B codes=0@0.00B eta=12.843s
INFO [03-31|14:18:53.370] Syncing: chain download in progress synced=100.00% chain=18.00B headers=16@6.00B bodies=14@6.00B receipts=14@6.00B eta=0s
INFO [03-31|14:18:53.371] Syncing: state download in progress synced=75.10% state=392.00B accounts=2@392.00B slots=0@0.00B codes=0@0.00B eta=3.320s
WARN [03-31|14:18:53.371] Unexpected account range packet peer=2d9ec9f1 reqid=7,480,776,852,441,583,095
INFO [03-31|14:18:53.374] Syncing: state download in progress synced=100.00% state=392.00B accounts=2@392.00B slots=0@0.00B codes=0@0.00B eta=-216ns
INFO [03-31|14:18:53.374] Syncing: state healing in progress accounts=0@0.00B slots=0@0.00B codes=0@0.00B nodes=0@0.00B pending=0
INFO [03-31|14:18:53.385] Rebuilding state snapshot
INFO [03-31|14:18:53.385] Committed new head block number=14 hash=e2f217..cce6a9
INFO [03-31|14:18:53.385] Resuming state snapshot generation root=8ce5ed..df499b accounts=0 slots=0 storage=0.00B dangling=0 elapsed="436.508µs"
INFO [03-31|14:18:53.386] Generated state snapshot accounts=2 slots=0 storage=96.00B dangling=0 elapsed="730.995µs"
INFO [03-31|14:18:53.385] Imported new chain segment number=16 hash=d4eae5..ed7afd blocks=2 txs=0 mgas=0.000 elapsed="300.199µs" mgasps=0.000
INFO [03-31|14:18:53.392] Looking for peers peercount=1 tried=0 static=0

```

16. Now we will interact with the blockchain using javascript console
 geth attach node1/geth.ipc

```

pranav@DESKTOP-OSHSLJ1:~/private_ethereum_setup$ geth attach node2/geth.ipc
Welcome to the Geth JavaScript console!

instance: Geth/v1.12.0-stable-e501b3b0/linux-amd64/go1.20.3
at block: 32 (Tue Mar 31 2026 14:20:10 GMT+0000 (UTC))
datadir: /home/pranav/private_ethereum_setup/node2
modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 rpc:1.0 txpool:1.0 web3:1.0

To exit, press ctrl-d or type exit
> net.peerCount
1
>

```

17. Interacting with the blockchain

```
"0x9f724e5b7bd747a7815130974af859a25ed67984d1869c9372a4707f451edc6f"
```




**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Name: Pranav Pol

Class : D20A

Roll no: 44

Batch: C

EXPERIMENT 10

Aim: Explore the Cryptocurrency Landscape

Theory:

Cryptocurrency Landscape – Theory

Cryptocurrencies are decentralized digital assets that operate on blockchain technology, enabling secure, transparent, and immutable transactions without the need for a central authority. These systems rely on distributed ledgers where transactions are validated through consensus mechanisms and recorded across a network of nodes.

Data Sources

Reliable and diverse data sources are essential for understanding cryptocurrency markets:

- **Financial Data Platforms** – Services like CoinGecko and CoinMarketCap provide structured data such as market capitalization, circulating supply, trading volume, and historical price trends.
- **News Aggregators** – Platforms such as CryptoPanic collect real-time news from multiple sources, helping track major events impacting the market.
- **Social Sentiment Analytics** – Tools like LunarCrush analyze social media activity to measure public sentiment and engagement.
- **Regulatory Sources** – Official portals like the U.S. Securities and Exchange Commission and frameworks such as MiCA provide insights into legal policies and compliance requirements.

Types of Cryptocurrencies

The cryptocurrency ecosystem consists of various categories based on functionality:

- **Store of Value (SoV)** – Bitcoin (BTC), known for limited supply and long-term value preservation.
- **Utility Tokens** – Ether (ETH) and Chainlink (LINK), used for network operations and smart contract execution.
- **Stablecoins** – Tether (USDT) and USD Coin (USDC), designed to maintain price stability by being pegged to fiat currencies.
- **Governance Tokens** – Uniswap (UNI) and Aave (AAVE), enabling users to participate in decentralized decision-making.

Key Characteristics

Cryptocurrencies exhibit the following features:

- **Decentralization** – No central authority controls the system.

- **Transparency** – Transactions are publicly recorded on the blockchain.
- **Immutability** – Data cannot be altered once confirmed.
- **Security** – Cryptographic algorithms ensure data integrity and safety.

Market Dynamics

The cryptocurrency market is influenced by multiple factors:

- **Volatility** – Prices fluctuate significantly due to demand, sentiment, and external events.
- **Liquidity** – Depends on trading volume and market participation.
- **Market Sentiment** – News, social media, and investor behavior impact price trends.
- **Regulation** – Government policies and frameworks influence adoption and stability.

Blockchain Technology Advancements

Modern blockchain systems are evolving to overcome limitations:

- **Consensus Mechanisms** – Shift from Proof of Work (PoW) to Proof of Stake (PoS) for efficiency.
- **Scalability Solutions** – Layer-2 technologies and sharding improve transaction speed.
- **Interoperability** – Enables communication between different blockchain networks.
- **Privacy Enhancements** – Zero-Knowledge Proofs (ZKP) allow secure verification without revealing sensitive data.

Adoption and Emerging Trends

Cryptocurrency adoption continues to grow across sectors:

- **Digital Payments** – Companies like Stripe and Shopify support crypto transactions.
- **Decentralized Finance (DeFi)** – Financial services operate without intermediaries.
- **Non-Custodial Wallets** – MetaMask and Phantom give users full asset control.
- **Asset Tokenization** – Real-world assets are increasingly represented on blockchain networks.

Utilizing Api for analysis and visualization :

Code:

```
import requests
import matplotlib.pyplot as plt import pandas as pd
import datetime

# CONFIGURATION
# Replace 'YOUR_API_KEY' with the key you got from CryptoCompare

API_KEY = 'YOUR_API_KEY' SYMBOL = 'LINK' COMPARISON_SYMBOL = 'USD'
LIMIT = 180 # Number of days

# ----- Fetch Data Function -----
def fetch_cryptocompare_data(symbol, comparison_symbol, limit, api_key): url =
f"https://min-api.cryptocompare.com/data/v2/histoday"
params = {
"fsym": symbol,
"tsym": comparison_symbol, "limit": limit,
"api_key": api_key
}

response = requests.get(url, params=params) data = response.json()

if data['Response'] == 'Success':
# CryptoCompare returns OHLCV data in 'Data' -> 'Data' return data['Data']['Data']
else:
print(f"Error fetching data: {data.get('Message')}") return None

# ----- Dashboard Function -----
def cryptocompare_dashboard(symbol, comparison_symbol, limit, api_key): raw_data =
fetch_cryptocompare_data(symbol, comparison_symbol, limit,
api_key)
if not raw_data: return

# Create DataFrame
df = pd.DataFrame(raw_data)

# Convert 'time' (unix timestamp) to datetime df['timestamp'] = pd.to_datetime(df['time'],
unit='s') df.set_index('timestamp', inplace=True)
```

```
# Standardize column names to match your previous logic # 'close' is used as the daily price
```

```
df['price'] = df['close']
```

```
df['volume'] = df['volumeto'] # Total volume in USD (volumeto)
```

```
# Calculate metrics
```

```
df['returns'] = df['price'].pct_change() df['MA7'] = df['price'].rolling(window=7).mean()
```

```
df['MA30'] = df['price'].rolling(window=30).mean() df['cumulative'] = (1 +  
df['returns']).cumprod() df['volatility'] = df['returns'].rolling(window=7).std() * 100
```

```
# Plotting
```

```
fig, axes = plt.subplots(3, 2, figsize=(16, 12)) fig.suptitle(f'CryptoCompare: {symbol} Analysis  
Dashboard',
```

```
fontsize=16, fontweight='bold')
```

```
# 1. Price Trend
```

```
axes[0, 0].plot(df.index, df['price'], color='tab:blue', linewidth=2) axes[0, 0].set_title('Historical  
Price Trend (USD)')
```

```
axes[0, 0].grid(True, linestyle='--', alpha=0.7)
```

```
# 2. Price vs Volume
```

```
axes[0, 1].plot(df.index, df['price'], color='tab:blue', label='Price')
```

```
axes[0, 1].bar(df.index, df['volume'], color='gray', alpha=0.3, label='Volume (USD)')
```

```
axes[0, 1].set_title('Price vs Trading Volume') axes[0, 1].legend()
```

```
# 3. Daily Returns Distribution
```

```
axes[1, 0].hist(df['returns'].dropna(), bins=40, color='tab:purple', edgecolor='white')
```

```
axes[1, 0].set_title('Daily Returns Distribution')
```

```
# 4. Moving Averages
```

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', alpha=0.5, label='Price')
```

```
axes[1, 1].plot(df.index, df['MA7'], color='tab:orange', label='7-Day MA')
```

```
axes[1, 1].plot(df.index, df['MA30'], color='tab:red', label='30-Day MA')
```

```
axes[1, 1].set_title('Trend Analysis (Moving Averages)') axes[1, 1].legend()
```

```
# 5. Cumulative Returns
```

```
axes[2, 0].fill_between(df.index, df['cumulative'], color='tab:green', alpha=0.2)
```

```
axes[2, 0].plot(df.index, df['cumulative'], color='tab:green') axes[2, 0].set_title('Cumulative  
Growth')
```

6. Volatility

```
axes[2, 1].plot(df.index, df['volatility'], color='tab:red') axes[2, 1].set_title('7-Day Rolling  
Volatility (%)')
```

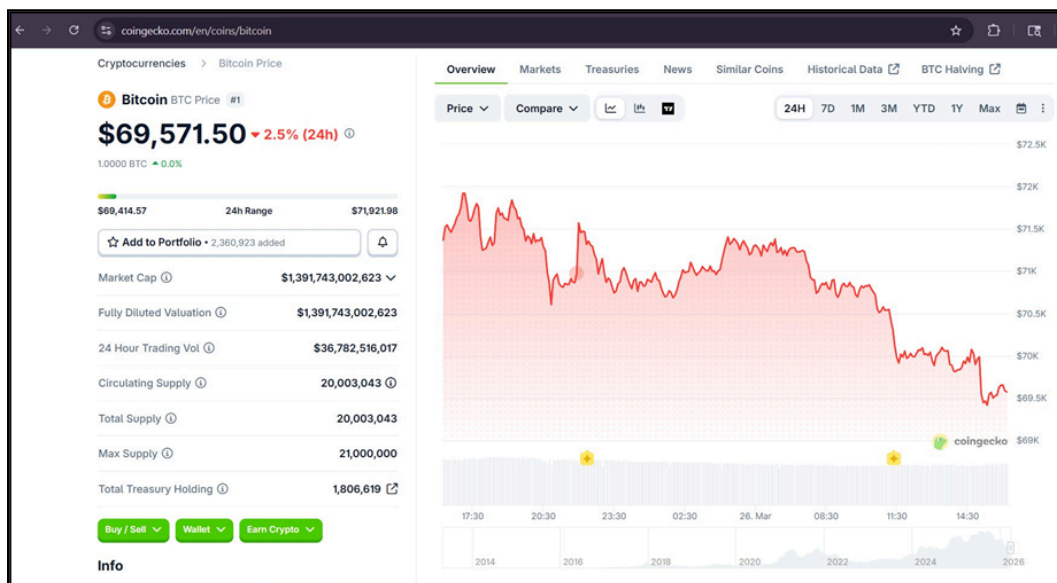
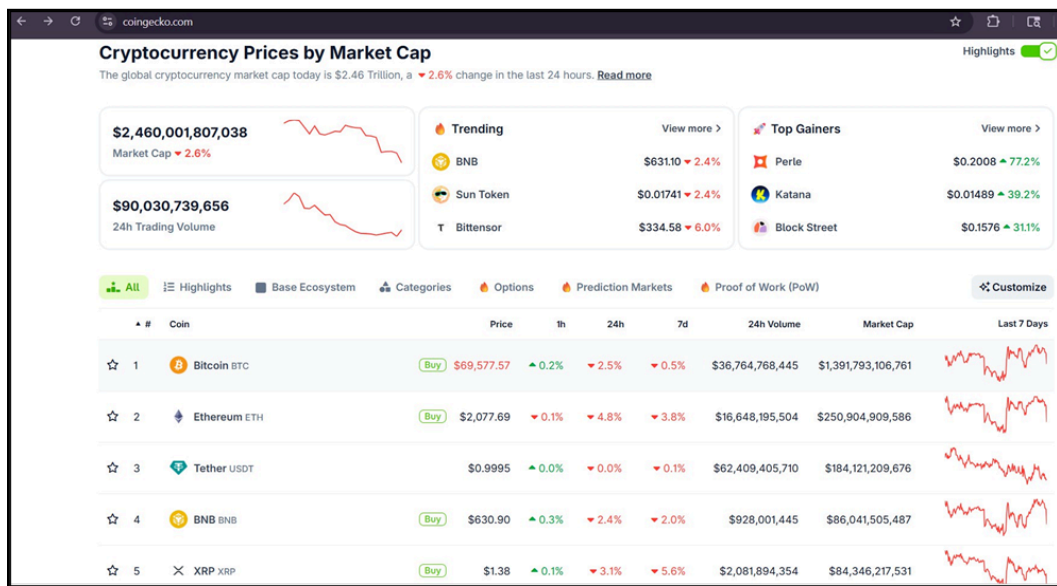
```
plt.tight_layout(rect=[0, 0.03, 1, 0.95]) plt.show()
```

Run

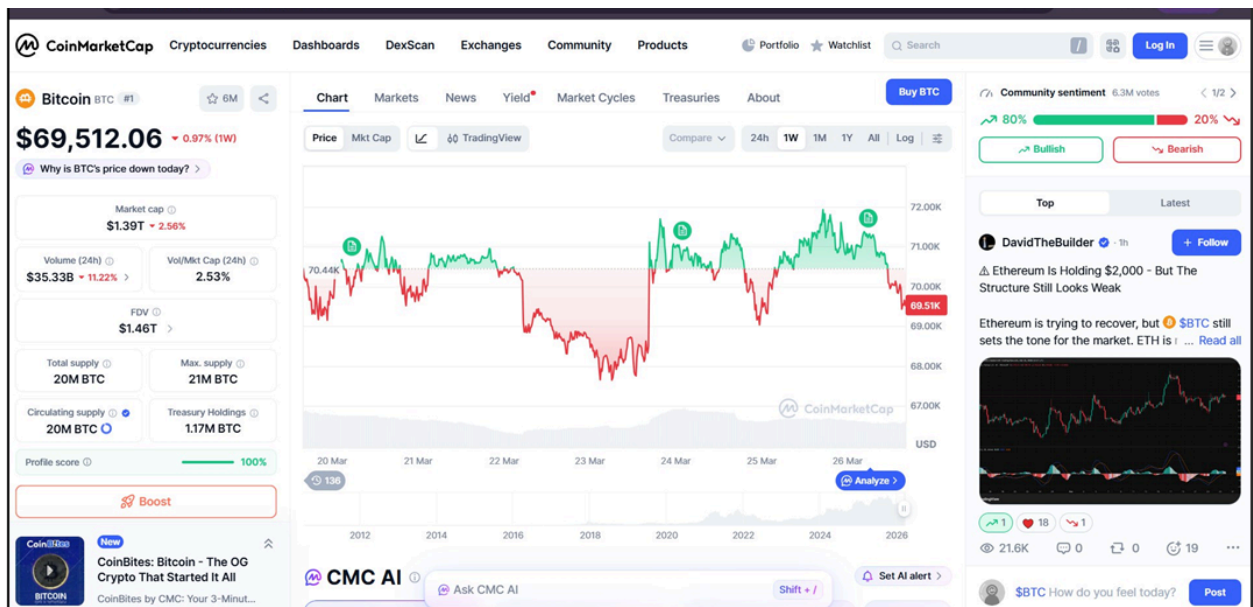
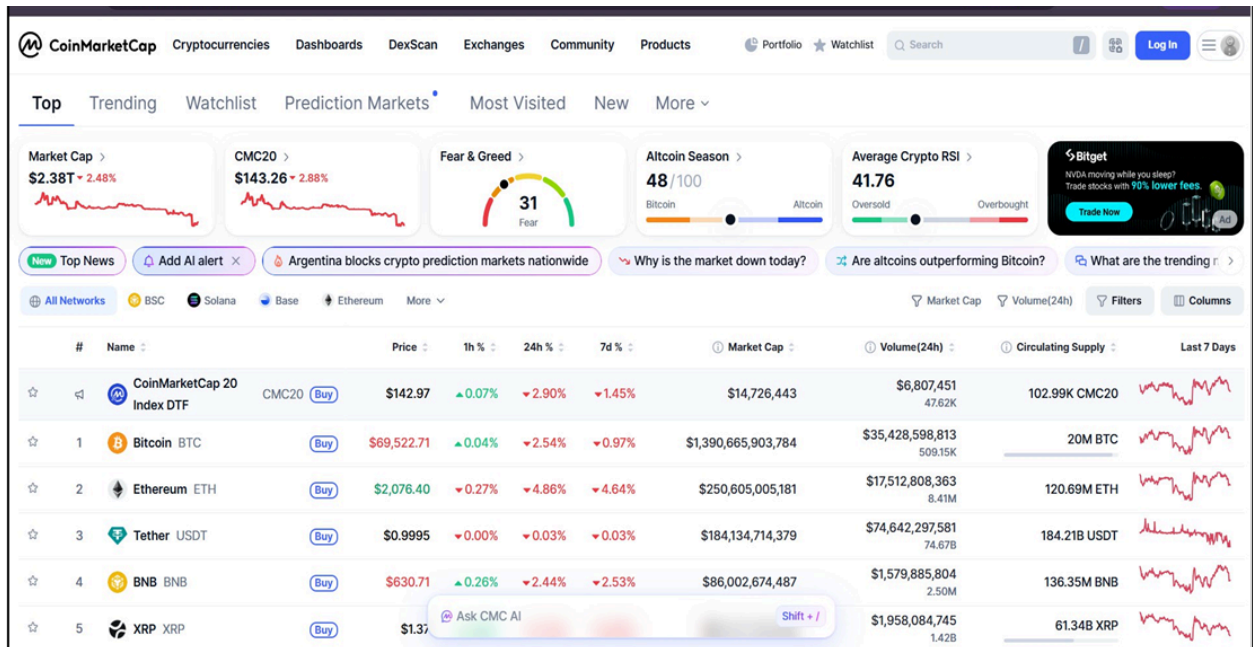
```
cryptocompare_dashboard(SYMBOL, COMPARISON_SYMBOL, LIMIT, API_KEY)
```

Output:-

Coingecko Website

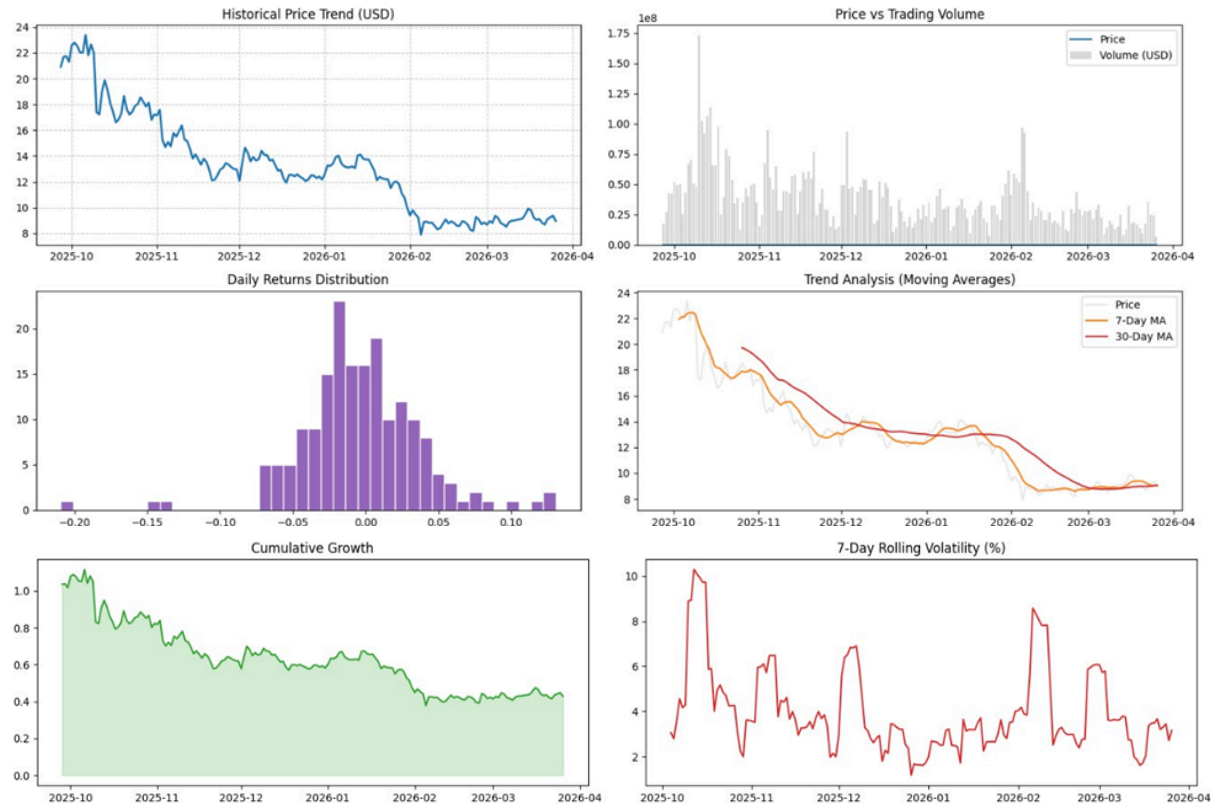


CoinMarketCap website



Dashboard

CryptoCompare: LINK Analysis Dashboard



Conclusion:-

By integrating automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, a comprehensive view of the cryptocurrency ecosystem can be achieved. This approach highlights the relationship between blockchain's underlying technical foundations and real-world market behavior, offering valuable insights into current trends as well as potential future developments in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

EXPERIMENT 11

Aim - Implementation of Hyperledger Fabric Network and Asset Management using Chaincode.

Theory -

Hyperledger Fabric Network Setup

- Consortium & Membership Service Provider (MSP): Fabric is permissioned, meaning only authorized organizations can join. MSPs define identities and roles.
- Peers: Nodes that maintain the ledger and endorse transactions. Each organization typically runs peers.
- Orderer: The ordering service packages endorsed transactions into blocks and delivers them to peers.
- Ledger: Composed of a blockchain (immutable transaction log) and a state database (current values).
- Channels: Private sub-networks where a subset of organizations transact confidentially.

Chaincode (Smart Contracts)

- Chaincode defines the business logic for asset management.
- Written in Go, Java, or Node.js.
- Deployed to peers, then instantiated on a channel.
- Functions typically include createAsset, queryAsset, transferAsset, and deleteAsset.

Transaction Flow

1. Proposal Creation: A client application submits a transaction proposal (e.g., create asset).
2. Endorsement: Peers simulate the transaction using chaincode and return signed results.
3. Ordering: The orderer sequences endorsed transactions into blocks.
4. Validation & Commit: Peers validate endorsements, check for conflicts, and commit the block to the ledger.

Asset Operations

- Asset Creation:
 - Chaincode function createAsset stores asset attributes (ID, owner, value) in the ledger's state database.
- Asset Querying:
 - queryAsset retrieves asset details by key. Rich queries are possible if CouchDB is used as the state database.
- Asset Transfer:
 - transferAsset updates the owner field of an asset. The transaction is endorsed, ordered, and committed, ensuring immutability and consensus.

Key Principles

- Immutability: Once committed, transactions cannot be altered.

- Consensus: Endorsement policies ensure multiple organizations agree before a transaction is valid.
- Privacy: Channels and private data collections allow selective visibility.
- Trust: Identities are cryptographically verified via MSPs.

In essence, Hyperledger Fabric provides a modular, permissioned blockchain framework where chaincode governs asset lifecycles. The theory emphasizes identity-driven trust, modular architecture, and consensus-based validation — ensuring secure creation, querying, and transfer of assets across organizations.

Prerequisite: Check if WSL is Installed

Open **PowerShell (Run as Administrator)** and run:

wsl --list --verbose

If it shows **no distributions**, you must install Ubuntu.

Install Ubuntu for WSL

Run this command in **PowerShell (Admin)**:

wsl --install -d Ubuntu

This installs:

- Ubuntu
- WSL2 kernel

The installation may take a few minutes.

Restart Your Computer

After installation finishes, **restart your PC**.

Open Ubuntu

Open:

Start Menu → Ubuntu

It will ask you to create:

Username

Password

Now Ubuntu is ready.

Implementation

Step 1 — Update the system

code :

sudo apt update

sudo apt upgrade -y

Explain: `apt update` refreshes the package index — it tells your system what versions of software are available. `apt upgrade` installs the newer versions. Fabric's dependencies (Docker, curl, git) can fail or behave unexpectedly on outdated system libraries. Always do this first on a fresh machine.

```
pranav@DESKTOP-OSHSLJI:~$ sudo apt update
sudo apt upgrade -y
[sudo] password for pranav:
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1540 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [246 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.6 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.6 kB]
Get:10 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [976 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe Translation-en [219 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [20.8 kB]
Get:14 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [2683 kB]
Get:15 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [623 kB]
Get:16 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [208 B]
Get:17 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 c-n-f Metadata [544 B]
Get:18 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [28.8 kB]
Get:19 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [6732 B]
Get:20 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:21 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [396 B]
Get:22 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:24 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:25 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:26 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:27 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:28 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:29 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1837 kB]
Get:30 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [336 kB]
Get:31 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:32 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.7 kB]
```

Step 2 — Verify prerequisites

code :

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
pranav@DESKTOP-OSHSLJI:~$ node -v
v18.19.1
pranav@DESKTOP-OSHSLJI:~$ npm -v
11.8.0
pranav@DESKTOP-OSHSLJI:~$ git --version
git version 2.43.0
pranav@DESKTOP-OSHSLJI:~$ docker --version
docker ps
Docker version 29.2.0, build 0b9d198
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
--------------	-------	---------	---------	--------	-------	-------

Explain: Fabric requires Docker (to run peer/orderer containers), Git (to clone the repo), and Node.js + npm (to build JavaScript chaincode). Running `docker ps` with no output confirms the Docker daemon is running and your user can access it. If `docker ps` requires sudo, you need to add your user to the docker group.

Step 3 — Clone fabric-samples

code :

```
git clone https://github.com/hyperledger/fabric-samples
```

```
cd fabric-samples
```

```

pranav@DESKTOP-OSHSLJI:~$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15527, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (60/60), done.
remote: Total 15527 (delta 57), reused 38 (delta 38), pack-reused 15429 (from 3)
Receiving objects: 100% (15527/15527), 24.16 MiB | 6.18 MiB/s, done.
Resolving deltas: 100% (8722/8722), done.
pranav@DESKTOP-OSHSLJI:~$ cd fabric-samples

```

Explain: `fabric-samples` is the official repository containing the test-network scripts, ready-made chaincode examples (including `asset-transfer-basic`), and configuration files. Everything else in this guide runs from inside this directory.

Step 4 — Download the install script and run it

code :

`curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh`

`chmod +x install-fabric.sh`

`./install-fabric.sh docker samples binary`

`./install-fabric.sh docker`

```

pranav@DESKTOP-OSHSLJI:~/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ chmod +x install-fabric.sh
pranav@DESKTOP-OSHSLJI:~/fabric-samples$ ./install-fabric.sh docker samples binary

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

====> Downloading version 2.5.15 platform specific fabric binaries
====> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
====> Will unpack to: /home/pranav/fabric-samples
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0     0      0     0      0     0      0  0:00:00  0:00:00  0:00:00     0
100 120M  100 120M    0     0 6272k      0  0:00:19  0:00:19  0:00:00 6483k
==> Done.
====> Downloading version 1.5.17 platform specific fabric-client binary

```

```

====> List out hyperledger images
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17          453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15       86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15     9972c209be47      178MB      49.5MB
ghcr.io/hyperledger/fabric-peer:2.5.15        3e25adc31a75      230MB      66.2MB
hyperledger/fabric-baseos:2.5                 654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest              654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                     453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                  453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                   453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                  86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15               86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest               86dded7eda92      1GB        255MB
hyperledger/fabric-orderer:2.5                9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB
hyperledger/fabric-orderer:latest             9972c209be47      178MB      49.5MB
hyperledger/fabric-peer:2.5                   3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:2.5.15                3e25adc31a75      230MB      66.2MB
hyperledger/fabric-peer:latest                3e25adc31a75      230MB      66.2MB

```

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ ./install-fabric.sh docker

Pull Hyperledger Fabric docker images

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Image is up to date for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Image is up to date for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15
====> ghcr.io/hyperledger/fabric-ccenv:2.5.15
2.5.15: Pulling from hyperledger/fabric-ccenv
Digest: sha256:86dded7eda9268e2cbf3691cb952edf545dc5faea1a79bfc9b47ec47d3ecfa9e
Status: Image is up to date for ghcr.io/hyperledger/fabric-ccenv:2.5.15
ghcr.io/hyperledger/fabric-ccenv:2.5.15

```

Explain: This one script does three things simultaneously. **docker** pulls all Fabric Docker images — **fabric-peer**, **fabric-orderer**, **fabric-ccenv**, **fabric-baseos**, **fabric-ca**, and critically **fabric-nodeenv** (needed to build Node.js chaincode). **binary** downloads the CLI tools — **peer**, **configtxgen**, **configtxlator** — into **~/fabric-samples/bin**. **samples** ensures the chaincode sample directories are present. Without **fabric-nodeenv**, chaincode deployment fails later.

Step 5 — Verify Docker images are present

code :

docker images | grep fabric

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      654bd4554bf9      250MB      80.4MB
ghcr.io/hyperledger/fabric-ca:1.5.17         453a0a727e82      381MB      123MB
ghcr.io/hyperledger/fabric-ccenv:2.5.15      86dded7eda92      1GB        255MB
ghcr.io/hyperledger/fabric-orderer:2.5.15    9972c209be47      178MB      49.5MB   U
ghcr.io/hyperledger/fabric-peer:2.5.15      3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-baseos:2.5                654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:2.5.15            654bd4554bf9      250MB      80.4MB
hyperledger/fabric-baseos:latest             654bd4554bf9      250MB      80.4MB
hyperledger/fabric-ca:1.5                   453a0a727e82      381MB      123MB
hyperledger/fabric-ca:1.5.17                453a0a727e82      381MB      123MB
hyperledger/fabric-ca:latest                 453a0a727e82      381MB      123MB
hyperledger/fabric-ccenv:2.5                 86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:2.5.15              86dded7eda92      1GB        255MB
hyperledger/fabric-ccenv:latest              86dded7eda92      1GB        255MB
hyperledger/fabric-nodeenv:2.5               17e2d447ca0d      570MB      146MB
hyperledger/fabric-orderer:2.5               9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:2.5.15            9972c209be47      178MB      49.5MB   U
hyperledger/fabric-orderer:latest            9972c209be47      178MB      49.5MB   U
hyperledger/fabric-peer:2.5                  3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:2.5.15               3e25adc31a75      230MB      66.2MB   U
hyperledger/fabric-peer:latest               3e25adc31a75      230MB      66.2MB   U

```

Explain: Confirm **hyperledger/fabric-nodeenv:2.5** is in the list. This image is the Node.js build environment that Fabric uses to compile and run JavaScript chaincode inside a container. If it is missing, **deployCC** will fail with a "No such image" error.

Step 6 — Set PATH and config path

code :

export PATH=\$PATH:\$HOME/fabric-samples/bin

export FABRIC_CFG_PATH=\$HOME/fabric-samples/config

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export PATH=$PATH:$HOME/fabric-samples/bin
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
pranav@DESKTOP-OSHSLSJI:~/fabric-samples$ peer version
peer:
Version: v2.5.15
Commit SHA: 83c7930
Go version: go1.26.0
OS/Arch: linux/amd64
Chaincode:
Base Docker Label: org.hyperledger.fabric
Docker Namespace: hyperledger

```

Explain: The `peer` binary lives in `~/fabric-samples/bin`, which is not on your shell's PATH by default. Without this export, the shell cannot find the command and returns "peer not found". `FABRIC_CFG_PATH` tells the peer CLI where to find `core.yaml`, the main configuration file. Without it, the peer errors with "cannot init crypto, path does not exist".

code :

peer version

You should see **Version: v2.5.15**.

Step 7 — Start the network and create the channel

code :

cd ~/fabric-samples/test-network

./network.sh up createChannel

```

pranav@DESKTOP-OSHSLSJI:~/fabric-samples/test-network$ cd ~/fabric-samples/test-network
./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org1.example.com      Removed
✓Container orderer.example.com         Removed
✓Container peer0.org2.example.com      Removed
✓Network fabric_test                  Removed
✓Volume compose_orderer.example.com   Removed
✓Volume compose_peer0.org1.example.com Removed
✓Volume compose_peer0.org2.example.com Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59f864cfa1398b52d341cb:latest
Deleted: sha256:27056269ed9759828700d7bfc21149ac76c17d725ae6f9a12c07cc42a04f539f
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035bee15cbcc7f59163835f604aa885d60dc:latest
Deleted: sha256:f7c74fe5fd5a6a73117e649e923a5de0a595771f793ee8e1b0047fc35d2d315c

```



```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ ./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using dat
Bringing up network
LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/home/pranav/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created
✓Volume compose_orderer.example.com Created
✓Volume compose_peer0.org1.example.com Created
✓Volume compose_peer0.org2.example.com Created
✓Container peer0.org1.example.com Created
✓Container orderer.example.com Created
✓Container peer0.org2.example.com Created
CONTAINER ID IMAGE COMMAND CREATED STATUS

```

Explain: This single command does everything needed to have a running network with a channel. Internally it uses **cryptogen** to generate TLS certificates and identity material (MSP) for Org1, Org2, and the Orderer. It starts three Docker containers — **orderer.example.com** on port 7050, **peer0.org1.example.com** on port 7051, and **peer0.org2.example.com** on port 9051. It then runs **configtxgen** to create **mychannel.block** (the channel genesis block), submits it to the orderer, joins both peers to the channel, and sets anchor peers so the two organizations can discover each other via gossip. The default channel name is **mychannel**.

Step 8 — Deploy the chaincode

code :

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ ./network.sh deployCC \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccl javascript
Using docker and docker-compose
Deploying chaincode on channel 'mychannel'
Executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
Executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1

```

```

2026-03-17 16:28:58.033 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:7051
2026-03-17 16:28:58.037 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [b424caca912525c6970bb1da73f421e25a00030499eab767783fa7e
atus (VALID) at localhost:9051
Chaincode definition committed on channel 'mychannel'
Using organization 1
Querying chaincode definition on peer0.org1 on channel 'mychannel'...
Attempting to Query committed status on peer0.org1, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org1 on channel 'mychannel'
Using organization 2
Querying chaincode definition on peer0.org2 on channel 'mychannel'...
Attempting to Query committed status on peer0.org2, Retry after 3 seconds.
+ peer lifecycle chaincode querycommitted --channelID mychannel --name basic
+ res=0
Committed chaincode definition for chaincode 'basic' on channel 'mychannel':
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscc, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required

```

Explain: `-ccn basic` is the name your chaincode will be called by in all future commands. `-ccp` is the path to the chaincode source code. `-ccl javascript` tells Fabric to use the Node.js runtime. Internally this runs the full Fabric chaincode lifecycle — package the source into a `.tar.gz`, install it on both peers, have Org1 approve the definition, have Org2 approve the definition (both approvals are required since majority endorsement is the default policy), and finally commit the definition to the channel. Once committed, two new Docker containers appear — one chaincode container per peer (`dev-peer0.org1...` and `dev-peer0.org2...`).

Step 9 — Set peer identity environment variables

code :

```

export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

```

pranav@DESKTOP-OSHSLJ1:~/fabric-samples/test-network$ export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051

```

Explain: The `peer` CLI has no built-in session — it does not know who you are. Every command needs to know which organization's identity to use, which peer to connect to, and how to verify TLS. These five variables provide exactly that. `CORE_PEER_LOCALMSPID` declares "I am Org1". `CORE_PEER_MSPCONFIGPATH` points to the Admin user's private signing key and certificate. `CORE_PEER_TLS_ROOTCERT_FILE` provides the CA certificate to verify the

peer's TLS identity. **CORE_PEER_ADDRESS** is which peer to send requests to. These must be re-exported in every new terminal session.

Step 10 — Initialize the ledger with sample data

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile \  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles \  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles \  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'
```

```
pranav@DESKTOP-0SHSLJ1:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscac  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.co  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.co  
--waitForEvent \  
-c '{"function": "InitLedger", "Args": []}'  
2026-03-17 16:37:35.757 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:9051  
2026-03-17 16:37:35.757 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [f982da882feb2411792ddb2c62b47591512c0269f789efe8ab9ce462  
atus (VALID) at localhost:7051  
2026-03-17 16:37:35.758 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explain: **invoke** is used for any operation that writes to the ledger. **-o localhost:7050** is the orderer address. **--tls** and **--cafile** enable TLS and provide the CA cert to verify the orderer. **-C mychannel** and **-n basic** identify which channel and chaincode. The two **--peerAddresses** and **--tlsRootCertFiles** pairs are the critical part — the default endorsement policy requires a majority of organizations to endorse. With two orgs, both must sign. Sending to only one peer results in **ENDORSEMENT_POLICY_FAILURE**. **--waitForEvent** blocks until both peers confirm the

transaction is committed (status VALID) rather than just submitted. **InitLedger** seeds the ledger with six sample assets.

Step 11 — Query all assets

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":5,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"a  
Brad","Size":5,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"Appra  
Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adrian  
cType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Explain: **query** is for read-only operations. No transaction is created, no block is written, no endorsement from multiple peers is needed. It is fast and cheap. This returns all six assets seeded by **InitLedger**:

```
[ {"ID":"asset1","Color":"blue","Size":5,"Owner":"Tomoko","AppraisedValue":300},  
  {"ID":"asset2","Color":"red","Size":5,"Owner":"Brad","AppraisedValue":400},  
  {"ID":"asset3","Color":"green","Size":10,"Owner":"Jin Soo","AppraisedValue":500},  
  {"ID":"asset4","Color":"yellow","Size":10,"Owner":"Max","AppraisedValue":600},  
  {"ID":"asset5","Color":"black","Size":15,"Owner":"Adriana","AppraisedValue":700},  
  {"ID":"asset6","Color":"white","Size":15,"Owner":"Michel","AppraisedValue":800} ]
```

Step 12 — Read a specific asset

code :

```
peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset5"]}'
```

```
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}  
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'  
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}
```

Result:

```
{"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15}
```

Explain: **ReadAsset** takes an asset ID and returns that single asset's data from the world state. The world state is a key-value database that stores the current (latest) value for each asset. The full transaction history is stored in the blockchain, but queries read from this faster world state.

Step 13 — Create a new asset

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tls  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example  
--waitForEvent \  
-c '{"function":"CreateAsset","Args":["asset10","blue","10","Pranav","1000"]}'  
2026-03-17 16:38:30.370 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:7051  
2026-03-17 16:38:30.371 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [d88abbf8b7aaff8416fb4a8914a4d01c8064d053ebc5145787034]  
atus (VALID) at localhost:9051  
2026-03-17 16:38:30.371 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200  
{"Color":"blue","Size":10,"Owner":"Pranav","AppraisedValue":1000}
```

Explain: **CreateAsset** takes five arguments in order — [ID, Color, Size, Owner, AppraisedValue]. This is a write operation so both peers must endorse. The orderer packages the endorsed transaction into a block and delivers it to both peers. Both confirm VALID and the asset is permanently recorded on the ledger.

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query \  
-C mychannel \  
-n basic \  
-c '{"Args":["ReadAsset","asset10"]}'  
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}  
  
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Pranav","Size":10}
```

The transaction flow showing what happens inside Fabric every time you run **invoke** or **query** Shows what happens inside Fabric every time you run **peer chaincode invoke** — the proposal, endorsement, ordering, and commit flow — and why both peers are always required for any write operation.

Step 14 — Transfer asset ownership

code :

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile  
$HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/  
orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/  
peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles  
$HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/  
peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'
```

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile $HOME/fabric-samples/test-network/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.c  
.pem \  
-C mychannel \  
-n basic \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles $HOME/fabric-samples/test-network/organizations/peerOrganizations/org2.example.com/peers/peer0.org2.example.com/tls/ca.crt \  
--waitForEvent \  
-c '{"function": "TransferAsset", "Args": ["asset10", "Rahul"]}'  
2026-03-17 16:41:28.692 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
atus (VALID) at localhost:9051  
2026-03-17 16:41:28.692 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [e834ab1020eac67d6bbf5759291d3c3bd92083d1a340d1f5bd35104c2e387019] committed  
atus (VALID) at localhost:7051  
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
```

Explain: **TransferAsset** takes [**assetID**, **newOwner**]. It reads the current asset from the world state, updates the Owner field, and writes it back. The function returns the previous owner ("**Pranav**") as confirmation. Both peers endorse, the orderer commits the block, and the ledger is updated permanently.

Verify the ownership change:

code :

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
```

```
# {"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

```
2026-03-17 16:41:28.692 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:"Pranav"
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset10"]}'
{"AppraisedValue":1000,"Color":"blue","ID":"asset10","Owner":"Rahul","Size":10}
```

Owner is now **Rahul**

Step 15 — To stop/bring down the Hyperledger Fabric network, run:

[./network.sh](#) down

```
pranav@DESKTOP-OSHSLJI:~/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container orderer.example.com          Removed
✓Container peer0.org2.example.com        Removed
✓Container peer0.org1.example.com        Removed
✓Network fabric_test                    Removed
✓Volume compose_peer0.org1.example.com  Removed
✓Volume compose_peer0.org2.example.com  Removed
✓Volume compose_orderer.example.com     Removed
Error response from daemon: get docker_orderer.example.com: no such volume
Error response from daemon: get docker_peer0.org1.example.com: no such volume
Error response from daemon: get docker_peer0.org2.example.com: no such volume
Removing remaining containers
Removing generated chaincode docker images
Untagged: dev-peer0.org2.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035f864cfa1398b52d341cb:latest
Deleted: sha256:aa29d3b9fb04e6316131fbf334a23f28d8a1646fa9d40090916c02d54bc5b76d
Untagged: dev-peer0.org1.example.com-basic_1.0-1d4d445573112813889f87c307bc2b5fbe664c87b24c035163835f604aa885d60dc:latest
Deleted: sha256:08d49c295856f950367880744bcacfaf56a7134bf208cacfab8a29ef9221703c
```

Conclusion

The Hyperledger Fabric network was successfully implemented, demonstrating asset creation, querying, and transfer using chaincode. The process highlights key concepts such as consensus, immutability, and secure transaction flow. Overall, it provides a clear understanding of permissioned blockchain systems and their real-world applications in asset management.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Decentralized Bug Bounty Platform

Trustless Vulnerability Reporting on the Ethereum Blockchain

Group 14 :

Pranav Pol 44

Siddhant Sathe 52

Arnav Sawant 54

Pranav Titambe 60

Overview

Bug bounty programs reward ethical hackers who discover and responsibly disclose security vulnerabilities.

Traditional platforms like HackerOne and Bugcrowd rely on centralized intermediaries, creating trust gaps between project teams and researchers.

This platform eliminates intermediaries using Ethereum smart contracts — funds are escrowed on-chain and released only after governance approval.

Built on Solidity ^0.8.20 with OpenZeppelin security primitives, it supports both ETH and ERC20 rewards.

\$2.2B+

Paid in bounties globally (2023)

70%

Researchers distrust payout systems

100%

On-chain escrow transparency

Core challenges in bug bounty management

Core Problem: No trustless mechanism exists to ensure researchers get paid after disclosing vulnerabilities to project teams.

Payment Disputes

- Teams may delay or deny payouts
- No enforceable reward guarantee
- Researchers have no recourse

Centralized Control

- Single point of failure
- Platform can freeze accounts
- Opaque decision-making

Reputation Gaps

- No verifiable researcher reputation
- Easy to spoof credentials
- No on-chain proof of work

Stakeholder Conflicts

- Researchers vs. Project teams
- Governance authority bias risk
- Lack of neutral arbitration

Existing System — Traditional Approach

4

Root Cause Analysis

How It Currently Works

- 1 Researcher finds bug
- 2 Submits to platform (HackerOne, Bugcrowd)
- 3 Platform reviews & forwards to team
- 4 Team decides payout (manual)
- 5 Platform transfers funds (may delay)
- 6 Researcher receives reward

Limitations & Challenges

- Centralized escrow with platform counterparty risk
- Manual approval — arbitrary & slow (weeks to months)
- No immutable audit trail for submissions
- Payout disputes resolved by biased intermediary
- Researchers blacklisted without transparent justification

Root Cause Analysis

- Trust deficit: funds not locked until payout guaranteed
- No smart contract enforcement of reward commitments
- Governance is centralized and non-auditable
- Reputation systems are platform-siloed and non-portable

Why Blockchain?

5

Key properties that address the core trust problem



Trustless Escrow

Smart contracts lock funds at vault creation. No human can intercept or redirect payment once approved — code is law.



Immutable Audit Trail

Every report submission, approval, rejection, and payout is permanently recorded on-chain. Nothing can be altered retroactively.



Transparent Governance

Governance logic is publicly auditable Solidity code. Authority actions are traceable with on-chain event logs.



Programmable Payouts

Severity-based rewards are encoded at vault creation. No negotiation after submission — rules are enforced automatically.



Decentralized Access

Any researcher worldwide can submit reports without gatekeeping. No KYC barriers or platform account requirements.



Portable Reputation

ERC721 NFTs mint on-chain proof of successful disclosures, carrying reputation across platforms and organizations.

Public Blockchain — Ethereum Mainnet / Testnet

SELECTED

Public

- Open participation
- Permissionless
- Censorship-resistant
- Transparent ledger
- ETH/ERC20 native

Private

- Restricted access
- Enterprise control
- Faster consensus
- Limited transparency
- Custom tokens only

Consortium

- Semi-decentralized
- Known validators
- Moderate trust
- Industry governance
- Complex setup

Justification: Public Ethereum ensures any researcher can participate without permission, all fund movements are auditable by the community, and existing DeFi security tooling (OpenZeppelin, Slither, Hardhat) integrates natively.

Key data entities in the system



Governance Decisions

Approver address, approval/rejection timestamp, decision reason string, incremental approved report count per vault



Vulnerability Reports

Vault ID, researcher address, severity level, IPFS hash (off-chain content pointer), payout amount, status (Pending/Approved/Rejected/Paid), timestamps



Reputation NFT Metadata

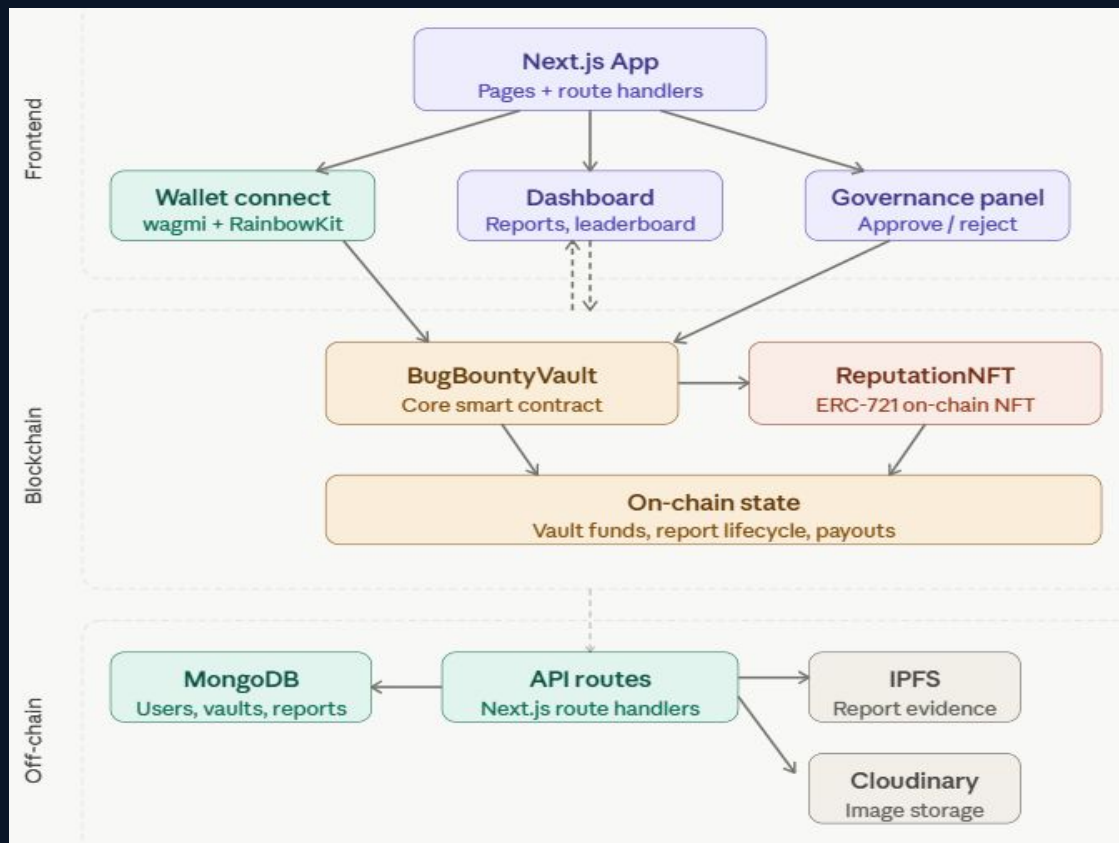
ERC721 token ID, researcher wallet address, linked report ID, IPFS tokenURI with vulnerability details, mint timestamp



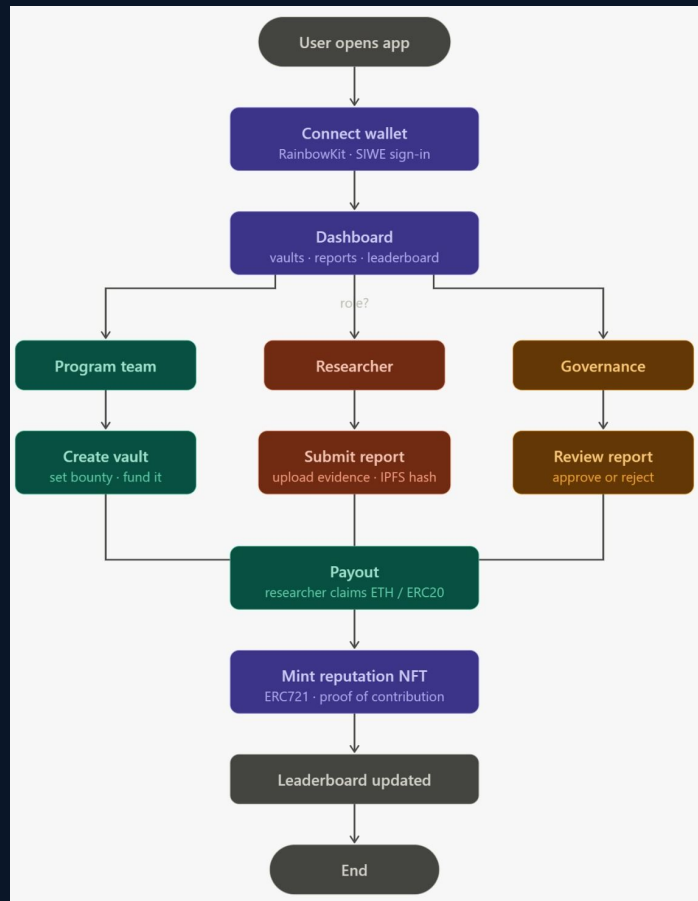
IPFS Report Content

Full vulnerability description, proof-of-concept code, remediation steps, CVSS score, environment details — stored off-chain, referenced by hash

On-chain component diagram



Step-by-step process flow



Technology stack powering the platform



Solidity ^0.8.20

Smart Contracts

Primary contract language. Overflow checks built-in, custom errors, immutable variables.



Ethereum

Blockchain Platform

Battle-tested public blockchain. EVM execution, native ETH transfers, global validator network.



OpenZeppelin

Security Library

ERC20, ERC721, Ownable, ReentrancyGuard, SafeERC20 — industry-standard audited primitives.



ERC20 / ERC721

Token Standards

ERC20 for reward tokens. ERC721 for soulbound reputation NFTs with IPFS metadata URIs.



IPFS

Decentralized Storage

Vulnerability report content stored off-chain. Content-addressed via SHA256 hash stored on-chain.



Hardhat / Foundry

Dev & Testing

Smart contract testing framework. Unit tests, fuzzing, gas profiling, deployment scripts.



Slither / MythX

Security Analysis

Static analysis and formal verification tools for detecting reentrancy, overflow, access control flaws.



Multisig / Gnosis Safe

Governance (Future)

Replace single-authority governance with DAO voting or multisig for enhanced decentralization.

Strategic evaluation of the decentralized bug bounty platform

STRENGTHS

- Trustless escrow eliminates payout disputes
- Immutable on-chain audit trail
- Permissionless global researcher access
- Automated severity-based payouts
- Portable NFT reputation system

WEAKNESSES

- Gas costs deter small bounty submissions
- Smart contract bugs = catastrophic fund loss
- Single governance authority is centralization risk
- IPFS content not guaranteed to persist
- Complex UX for non-crypto researchers

SW
OT

OPPORTUNITIES

- DAO-based governance with token voting
- ZK proofs for private vulnerability disclosure
- Cross-chain deployment (Polygon, Arbitrum)
- Integration with existing audit platforms
- DeFi protocol adoption as standard practice

THREATS

- Regulatory classification of reward tokens
- Front-running of vulnerability disclosures
- Malicious governance authority collusion
- Smart contract exploit draining vault funds
- Low researcher adoption without critical mass

Key improvements over existing centralized platforms

01

DAO-Based Governance Voting

Current: Single governance address can approve/reject arbitrarily **Proposed:** Replace with token-weighted DAO voting. Minimum quorum required for approval. Time-locked decisions with community veto window.

Impact

Eliminates single-point authority bias. Provides censorship-resistant, community-validated approval.

02

ZK-Proof Private Submissions

Current: IPFS hashes reveal metadata timing; front-running possible **Proposed:** Researchers submit ZK commitment hash first. Reveal vulnerability details only after governance locks in decision. Prevents information leakage.

Impact

Protects researchers from exploitation of their disclosures before patches are deployed.

03

On-Chain Dispute Resolution

Current: Rejected reports have no appeal mechanism **Proposed:** Kleros-style decentralized court integration. Researcher can stake and appeal to neutral jurors. Slashing for bad-faith rejections by governance.

Impact

Provides fair recourse, increases researcher trust, and disincentivizes governance abuse.

Best-fit implementation strategy

✓ Recommended: Multisig Governance + Escrow-First Architecture with Severity NFT Tiers

Efficiency

- Automated payout execution — no manual bank transfers
- Severity tiers pre-encode rewards — no negotiation needed
- IPFS offloads heavy data from expensive on-chain storage
- Batch report processing via governance multisig

Security

- ReentrancyGuard prevents double-spend exploits
- SafeERC20 handles non-standard token edge cases
- Multisig governance requires 3-of-5 signer consensus
- Timelock on vault withdrawal prevents rug pulls

Transparency

- All decisions emit on-chain events (immutable logs)
- Vault balances publicly visible on blockchain explorer
- Report status transitions are cryptographically auditable
- NFT metadata permanently links researcher to disclosure

Technical, Economic, and Scalability assessment

Feasibility Scores

Technical

- Solidity + OpenZeppelin = proven stack
- IPFS for off-chain data is production-ready
- Requires comprehensive unit & fuzz testing
- Audit by 2 independent firms recommended

Economic

- Gas costs: ~\$5–\$50 per transaction on L1
- L2 deployment (Arbitrum) reduces 10–100×
- No platform fees — only gas costs
- Initial audit budget: ~\$20K–\$50K

Scalability

- Ethereum L1: ~15 TPS (constrained)
- L2 rollups support 1000+ TPS
- IPFS decouples storage from chain capacity
- Vault architecture allows horizontal scaling

Phased delivery roadmap

Phase 1 Design	Phase 2 Development	Phase 3 Testing	Phase 4 Deployment
<i>Weeks 1–3</i> • Define vault schema & report data model • Design governance authority role model • Specify ERC721 NFT metadata standard • Map ETH + ERC20 payout flows • Threat modeling & attack surface analysis	<i>Weeks 4–9</i> • Implement BugBountyVault contract • Integrate OpenZeppelin libs (Ownable, ReentrancyGuard, SafeERC20) • Build ReputationNFT ERC721 contract • Develop frontend dApp (React + ethers.js) • IPFS integration for report storage	<i>Weeks 10–13</i> • Unit tests: all contract functions • Fuzz testing with Foundry • Integration tests (ETH + ERC20 flows) • External security audit (Tier-1 firm) • Testnet deployment + bug bounty on bounty	<i>Weeks 14–16</i> • Deploy to Ethereum mainnet + Arbitrum L2 • Publish contract source (Etherscan verify) • Community onboarding & documentation • Governance multisig setup (Gnosis Safe) • Monitor events + incident response plan

Societal, Environmental & Ethical dimensions

A. Societal Impact — User Benefits

Financial Inclusion

Researchers in emerging markets gain direct ETH payouts without bank accounts or geographic restrictions

Merit-Based Recognition

NFT credentials create verifiable proof of skills, enabling researchers to build reputation across organizations

Software Safety

Higher researcher trust → more vulnerability disclosures → safer DeFi protocols → reduced financial hacks

B. Environmental Impact — Sustainability

Post-Merge Ethereum uses Proof-of-Stake — energy consumption reduced by ~99.95% vs. Proof-of-Work.

L2 deployment (Arbitrum/Optimism) further reduces per-transaction energy footprint.

Eliminates paper/physical paperwork from traditional bug reporting workflows.

C. Ethical Considerations

Privacy

IPFS hashes stored on-chain without exposing vulnerability details. Researcher identity is pseudonymous (wallet address).

Transparency

All governance decisions are publicly auditable on-chain. No hidden rejections or undisclosed conflicts of interest.

Trust

Smart contract escrow enforces reward commitments. Neither party can unilaterally change terms after vault creation.

Barriers to adoption and deployment



Cost

- Ethereum L1 gas: \$20–\$150 per payout transaction at peak
- Smart contract audit: \$20K–\$80K for production-grade security
- IPFS pinning services (Pinata, nft.storage) have ongoing costs
- Mitigation: L2 deployment reduces gas 10–100× at same security



Technical Complexity

- Researchers need Web3 wallet (MetaMask) — unfamiliar to many
- ERC20 token approval flow adds friction before bounty claim
- IPFS content addressing requires client-side hash verification
- Mitigation: Abstract wallet management with account abstraction (ERC-4337)



Regulatory Issues

- ETH/ERC20 bounty payments may be classified as taxable income
- Some jurisdictions restrict anonymous payments above reporting thresholds
- Smart contract code-as-law conflicts with consumer protection law
- Mitigation: Optionally integrate on-chain KYC for regulated entities



Adoption Barriers

- Network effects: Platform value grows with researcher participation
- Competing platforms (HackerOne) have established brand trust
- DeFi-native researchers are minority of total security talent pool
- Mitigation: Fiat on-ramp and off-ramp integrations via Stripe or MoonPay

18 | Conclusion & Future Scope

Conclusion

The Decentralized Bug Bounty Platform solves the fundamental trust deficit in vulnerability disclosure by replacing human intermediaries with smart contract escrow.

By encoding reward commitments in immutable Solidity code, leveraging OpenZeppelin security primitives, and issuing on-chain reputation NFTs, the system delivers:

- Guaranteed payouts for researchers
- Transparent governance for project teams
- Immutable audit trails for the community

Future Scope

-  DAO voting for decentralized governance
-  ZK-proof private vulnerability submission
-  On-chain dispute resolution (Kleros)
-  Cross-chain support (Polygon, Arbitrum)
-  AI-assisted severity classification oracle
-  Researcher leaderboard with staking rewards
-  Protocol integration with Immunefi and Code4rena

Built on Solidity ^0.8.20 • OpenZeppelin • Ethereum • IPFS • ERC721

Decentralized Bug Bounty Platform

"Code is Law. Trust the Contract, Not the Counterparty."



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 13

Aim: Assignment 1

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Question : Explore following 4 websites and answer the questions bitcoin.org Blockchain.com https://coinmarketcap.com/ https://www.investopedia.com/
CO Mapped	CO1,CO2

Hard Copy Submitted



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 14

Aim: Assignment 2

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Questions ● Draw a neat diagram and explain components of Hyperledger Fabric? ● Draw a neat diagram and explain the transaction flow in Hyperledger Fabric? ● State the requirements of a consensus algorithm. Explain PAXOS and kafka consensus ● Explain ERC 20 token standard and its functions and Compare how ERC 20 token is different than ERC 721 token ● What is Tokenization? Explain its significance and challenges/limitation with example ● Compare Fungible and non Fungible tokens (Give examples of each) ● How can blockchain-based IoT networks improve device management, data monetization, and user privacy in the rapidly growing IoT ecosystem? ● In what ways can blockchain technology enhance transparency, security, and efficiency in government services and record-keeping? Discuss the challenges and benefits of implementing blockchain-based voting systems for elections.
CO Mapped	CO4,CO5,CO6

Hard Copy Submitted
ITC801: Blockchain and DLT Lab



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

Experiment 15

Aim: Assignment 3

Roll No.	44
Name	Pranav Pol
Class	D20A
Subject	ITC801: Blockchain and DLT Lab Q1 Analyze & compare different consensus mechanisms such as PoW, PoS, Delegated Proof of Stake & PBFT in terms of security, scalability, energy efficiency & fault tolerance. Illustrate your answer with examples like Bitcoin & Ethereum. Q2 Analyze different types of nodes in a blockchain network & explain their role in maintaining the system. Compare how each node contributes to validation, storage & network efficiency. (Compare UTXO model) Q3 Analyze the UTXO model & explain how it ensures transaction transparency & security in blockchain systems. Illustrate with an example such as Bitcoin. Q4 Explain crypto asset & cryptocurrency. Examine & analyze the role of different types of crypto assets (utility tokens, security tokens, crypto coins) in the blockchain ecosystem. Evaluate their impact on digital finance & decentralized applications. Q5 Analyze the role of payment scripts in blockchain transactions. Examine & analyze different types of payment scripts. Explain how they ensure secure & verifiable transfer of coins. Illustrate with an example. Q6 What is a smart contract? Explain lifecycle of smart contracts. Explain advantages, limitations, risks & opportunities of smart contracts. Q7 Explain the structure of BTC transaction & ETH transaction. Q8 Explain the difference between mining & minting and explain the process
CO Mapped	CO1- CO6

Hard Copy Submitted